

© 2024 Manasvi Saxena

A SEMANTICS-FIRST APPROACH TO SAFE GUIDELINES-BASED CLINICAL
DECISION SUPPORT

BY

MANASVI SAXENA

DISSERTATION

Submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy in Computer Science
in the Graduate College of the
University of Illinois Urbana-Champaign, 2024

Urbana, Illinois

Doctoral Committee:

Professor Grigore Roşu, Chair

Professor Jose Meseguer

Professor Lui Sha

Dr. Serdar Tasiran, Amazon Web Services (AWS)

ABSTRACT

Preventable medical errors (PMEs), characterized by misdiagnosis or mistreatment, pose a significant challenge in healthcare. In the United States, PMEs were estimated to have caused between 44,000 and 98,000 deaths in 1997, rising to more than 250,000 in 2013. Additionally, the financial burden of PMEs to the U.S. economy in 2008 was estimated at \$19.5 billion.

One strategy to reduce PMEs in medicine is through the use of clinical best practice guidelines (BPGs). BPGs are systematically developed, evidence-based statements published by medical institutions and associations that standardize diagnosis and treatment for various clinical scenarios. BPG-conformance has been linked with reduced rates of PMEs, but, following BPGs in practice can be challenging. Computerized Decision Support Systems (CDSSs) aim to improve conformance by encoding medical knowledge in BPGs and providing HCPs with situation-specific, guideline-conformant advice. Growing evidence suggests that effective CDSSs can reduce PMEs by boosting adherence to best practice guidelines.

This work presents a semantics-first approach to implementing safe clinical decision support systems. By semantics-first, we mean that i semantics of medical knowledge is accurately captured in the CDSS, and, ii the semantics of the programming language used to encode medical knowledge is formally defined. At the core of our approach is Medi $\mathbb{K}$: a novel domain specific language for expressing best practice guidelines that emphasizes comprehensibility to HCPs, enabling them to validate the accuracy of medical knowledge in its programs. Medi $\mathbb{K}$ has a complete, executable formal semantics in the $\mathbb{K}$ Framework, from which all execution and analysis tools are derived in a correct-by-construction manner.

To evaluate our approach, we collaborated with a major pediatric hospital to develop a complex real-world CDSS for the screening and management of Sepsis in pediatric cases, and validated that it satisfies desired safety properties. We outline how our approach improves upon the existing state-of-art, optimizations to address domain-specific needs of healthcare practitioners, and discuss challenges for future work.

To my parents and my sister, for their love and unconditional support, and for always believing in me, even when challenges ensued.

ACKNOWLEDGMENTS

While my journey at the University of Illinois has been long and challenging, I have been fortunate to have had the support of several wonderful people.

First, I would like to thank my advisor, Prof. Grigore Roşu, for his unwavering support and encouragement ever since I joined his research group as an undergraduate in the summer of 2014.

I am also extremely grateful to Prof. Lui Sha, with whom I have worked closely during the latter half of my PhD. His guidance, insights, and encouragement have been invaluable. I'd also like to thank him for his financial support, which enabled me to further develop my ideas.

I would like to express my sincere thanks to Dr. Serdar Tasiran for his mentorship during two summer internships at AWS's S3 Automated Reasoning Group (S3-ARG), where I also had the opportunity to collaborate with Dr. Ankush Desai and Dr. Dongyun Jin, from whom I learned a great deal. I am also thankful to Serdar for impressing upon me the importance of effectively presenting ideas—a skill I have worked hard to improve based on his advice.

I am grateful to Prof. Jose Meseguer for his valuable insights into implementing industrial-scale systems for use by non-experts in computer science.

I would also like to thank Prof. Rosu, Prof. Meseguer, Prof. Sha, and Dr. Tasiran for serving on my doctoral committee and for their time, expertise, and feedback.

I'd like to thank my labmates, both current and former, with whom I've had the pleasure of working with and learning from. Thank you, Daejun, Yi, Owolabi, Lucas, Xiaohong, Mircea, Nishant, Shuang, and Simon. A special shout-out goes to Nishant and Shuang for their companionship outside the lab, and to old friends Amulya, Neelabh, Rajiv, Dhruv, Balaji, and Aditya for always keeping me in the loop, even when I was too busy to respond.

I would like to extend my deepest gratitude to my family, both in the U.S. and in India. I'd like to thank my uncle, aunt, and cousins, who always made sure I had a home away from home. And to my parents and sister—thank you for the unconditional love, support, and unwavering belief in me, even during my moments of doubt.

TABLE OF CONTENTS

CHAPTER 1	INTRODUCTION	1
1.1	Hurdles to Wider CDSS Adoption	4
1.2	Solution	5
CHAPTER 2	BACKGROUND	9
2.1	Clinical Best Practice Guidelines	9
2.2	Clinical Decision Support Systems	13
CHAPTER 3	HURDLES TO CDSS ADOPTION	19
3.1	Addressing Adoption Hurdles	19
CHAPTER 4	RELATED WORK	23
4.1	DSL-Based Approaches	23
4.2	General Approaches	39
4.3	Discussion	45
CHAPTER 5	SEMANTICS-FIRST APPROACH TO CLINICAL DECISION SUPPORT	50
5.1	Semantics-First Approach	52
5.2	The $\mathbb{K}$ Framework	53
5.3	Pitfalls of the Semantics-First Approach	66
CHAPTER 6	EVALUATING $\mathbb{K}$	67
6.1	$\mathbb{K}$ EVM: Semantics of the Ethereum Virtual Machine	67
6.2	Ethereum ABI DSL	72
CHAPTER 7	SEPARATING CONCERNS: MODULAR AND SAFE CLINICAL DECISION SUPPORT	76
7.1	Control Theory and CDSSs	76
7.2	A Generic Treatment of Best Practice Guidelines	77
7.3	A Modular Architecture For CDSSs	78
CHAPTER 8	$\mathbb{K}$ -BASED COMPUTER INTERPRETABLE GUIDELINES	81
8.1	Advanced Cardiovascular Life Support Guidelines (ACLS)	81
8.2	The $\mathbb{K}$ -ACLS System	83
8.3	Discussion	93

CHAPTER 9	MediK: TOWARDS SAFE GUIDELINES-BASED CDSSS	94
9.1	Pediatric Sepsis Management BPG	95
9.2	Design Considerations	95
9.3	MediK Syntax	97
9.4	MediK Configuration	100
9.5	Rules	103
9.6	MediK Scheduling Semantics	108
9.7	Timer Semantics	110
9.8	MediK Pediatric Sepsis System	112
9.9	Discussion	117
CHAPTER 10	FUTURE WORK	121
10.1	Soundness of Model Checking Abstraction	121
10.2	Semantics Based Visual Representation Generation	123
10.3	Applicability to other CDSSs	125
CHAPTER 11	CONCLUSION	126
REFERENCES		128

CHAPTER 1: INTRODUCTION

Preventable medical errors (PMEs) characterized by misdiagnosis or mistreatment present a significant challenge in healthcare, both in the United States, and abroad [1]. According to a seminal report on the subject by the Institute of Medicine, in 1997, between 44,000 and 98,000 deaths were estimated to have been caused by PMEs in the United States alone [2]. A more recent study analyzed data from the eight-year period between 2000 and 2008, and estimated that in 2013, the number of deaths caused by PMEs was more than 250,000, making PMEs the third-leading cause of death in the United States [3].

The prevalence of PMEs in healthcare now finds increasing recognition. In September 2023, the President’s Council of Advisors on Science and Technology (PCAST) published a report stating patient safety to be an “urgent national public health issue”, calling for a “transformational effort” to address the problem [4]. Approximately one in four Medicare patients, the report mentioned, experienced adverse events during their hospitalization, with many catastrophic outcomes. Worryingly, 40% of these adverse events were due to PMEs. Thus, while recognizing the commitment that healthcare practitioners and their organizations towards ensuring quality care, the report deemed rates of medical errors and injuries as “alarmingly high.”

The adverse effects of PMEs extend beyond patient outcomes. PMEs have a negative effect on healthcare providers (HCPs). According to the authors of [1], PMEs caused psychological effects such as anger and guilt in healthcare providers (HCPs), adversely impacting their mental health. Moreover, there are costs associated with such errors. In 2008, the financial burden of PMEs to the United States was estimated to be \$19.5 billion, of which \$17 billion could be directly attributed to additional healthcare expenditure such as ancillary and prescription drug services, and inpatient and outpatient care. Additionally, \$1.4 billion was attributed to increased mortality and loss of productivity from missed work due to short-term disability claims [5].

In the United States, several initiatives to improve patient safety by reducing preventable errors have been undertaken at the federal government level. The September 2023 PCAST report itself included several suggestions to address patient safety, such as:

- (i) ensuring patients receive evidence-based practice for preventing harm and assessing risk, where as many measures as possible are generated in real-time from electronic health data.
- (ii) incentivizing hospitals to utilize evidence-based practices and associated novel computing tools.

- (iii) promoting research and development in areas such as computer science and health technology, including tools in prototype form, or limited deployment that hold promise to guide decision making to improve quality and safety.
- (iv) empowering hospitals with safety-critical methods, transparency, and a instilling a culture of safety.
- (v) harnessing advances in technology, such as in Artificial Intelligence, while employing best practices and engineering principles to mitigate safety and reliability of such systems.

The report highlights that technology is expected to play a significant role in addressing the problem. The emphasis on technology to reduce human errors isn't unprecedented. The report recommends the establishment of a multidisciplinary National Patient Safety Team, along the lines of the Commercial Aviation Safety Team (CAST) [6], comprising of experts from clinical, safety and computer sciences, to address challenges in patient safety. In the decade following its inception in 1997, CAST was responsible for reducing the fatality risk for commercial aviation in the United States by 83%, and aims to reduce the risk further by 50% by 2025. To put CAST's achievements into perspective, between 2008 and 2020, U.S. carriers transported more than 7.8 billion passengers. But, in the same timeframe, there were only 2 fatalities [7]. Several solutions were instrumental in achieving such remarkable results, including:

- fly-by-wire control that optimizes pilot inputs for safety, reliability, and passenger comfort [8].
- flight management and autopilot systems that automate certain navigation and flying tasks to reduce pilot workloads and make bandwidth available for monitoring and managing emergencies [9].
- diagnostic and assurance systems that support enhanced pilots' and maintenance staff's understanding of aircraft systems, faults, and mitigation strategies [9].
- traffic management systems that enhance situational awareness of pilots, air traffic controllers, and flight planners [10, 11].

CAST was instrumental in enabling the development of, and enacting policy to encourage adoption of aforementioned solutions. While solutions to address safety issues in aviation were often met with hurdles and skepticism, sustained efforts that involved stakeholders such

airline pilots, government agencies and safety-centric aviation companies eventually led to widespread adoption and deployment [12].

Unfortunately, despite similarities, improvements in aviation safety haven't been adapted in healthcare [13]. But, as evidenced by the September 2023 PCAST report, initiatives are being undertaken to address this. Addressing patient safety is a complex problem requiring multifaceted solutions, of which comprehensive use of technology is a part. This work in particular focuses on improving patient safety by enabling the development of *safe* and *cost effective* guidelines-based clinical decision support systems (CDSSs). These systems assist healthcare providers (HCPs) follow evidence-based best practice guidelines (BPGs) – evidence-based statements published by medical institutions that codify recommended treatment for various clinical scenarios [14]. High quality guidelines are routinely updated to account for results from clinical trials and advances in medicine, and make the latest diagnosis and treatment information accessible to providers [15].

Evidence suggests that BPGs can reduce medical errors, but, their effectiveness hinges on healthcare providers adhering to them in practice. For example, consider Advanced Cardiac Life Support (ACLS): a BPG published by the American Heart Association (AHA) for management of a life threatening condition called in-hospital cardiac arrest (IHCA) [16, 17]. Studies suggest that management of IHCA in 30% of adult, and 17% of pediatric cases deviates from the AHA-prescribed BPG, resulting in worse patient outcomes [18, 19, 20, 21, 22]. This can partly be attributed to the fact that BPG-adherence is difficult to achieve in practice [23, 24]. However, integrating BPGs with existing patient care-flow, and making them readily-accessible when required can improve adherence [25]. To this end, CDSSs codify BPGs and support HCPs with situation-specific advice. Such systems have been shown to improve BPG-adherence [26, 27], and evidence from multi-center clinical trials suggests that they reduce PMEs [28, 29]. Thus, guideline-based CDSSs are now considered imperative to the future of medical decision making in general [30].

Typically, a CDSS usually consists of the following components [31]:

- (a) a translation of the guideline to an executable medium, called the knowledge-base.
- (b) a user interface (UI) that healthcare providers use to interact with the system.
- (c) additional infrastructure that integrates with external data sources such as sensors and health records.

The system utilizes available data to assess patient state, and supports the HCP by providing guidelines-prescribed advice for treating assessed patient condition. As is evident, CDSSs align well with recommendations for improving patient safety put forth by the September

2023 PCAST report. They enable the use of best practice that codify evidence-based advice to optimize treatment outcomes and typically utilize patient data available via patient sensors and electronic health records to ascertain patient state. Well implemented CDSSs can aid decision-making and improve overall quality and safety at healthcare facilities that use them.

The PCAST report also stresses adapting lessons from aviation in healthcare. In aviation, systems for automation, monitoring and error mitigation that reduce workloads and enhance situational awareness of pilots have improved safety as pilots became accustomed to using and relying on such systems through training and operating procedures. CDSSs can play a similar role in healthcare, by ensuring HCPs, who often work in stressful environments, have the relevant information available to them at appropriate times to reduce workloads and improve situation awareness. CDSSs present a promising approach to addressing issues of patient safety in healthcare, and are thus the focal point of this work.

1.1 HURDLES TO WIDER CDSS ADOPTION

The Office of the National Coordinator for Health Information Technology (ONC) [32] has also recognized the potential that CDSS have to improve safety [33]. As part of its efforts to enable wider CDSS adoption, the ONC organized meetings that involved key stakeholders such as CDSS experts, policy makers and Electronic Health Record (EHR) vendors to study the state of CDSS in healthcare and barriers limiting adoption. Their findings were summarized in a National Association Medicine (NAM) special report [34]. Drawing from several publications on CDSSs, including a literature review commissioned by the Agency for Healthcare Research Quality [35], the report highlighted that despite considerable evidence that demonstrates CDSSs reduce both errors and cost while improve quality of care, implementation and actualization of such systems remains “nascent” due to the following challenges (Cs):

- C1. Absence of systematic ways of *validating content* in a *reliable*, *accessible*, and *updateable* manner.
- C2. Lack of *reliable*, *shareable* CDSS content that can be easily adopted across healthcare organizations and their (Information Technology) IT systems.
- C3. Technical difficulties of sharing due to *need for adaptation* to diverse Electronic Health Records (EHR) systems.
- C4. *Suboptimal* User Interfaces (UIs), implementation choices and workflows.

1.2 SOLUTION

Typically, CDSSs are implemented as cumbersome standalone systems. To develop a CDSS, practitioners work with developers to produce a requirements documentation that presents the BPGs’s medical knowledge in a manner amenable to software development [36]. This documentation is then utilized to develop the CDSS. Thus, the BPG serves as a functional specification for the medical knowledge encoded in the CDSS. But, this process has several limitations. First, the implementation, i.e., the medical knowledge within the CDSS, may not concur with its specification, i.e., the textual BPG. BPGs are long, complex textual documents, where the exact meaning of terms may not be explicitly stated, and recommendations may be ambiguous [37]. Capturing and communicating these complexities via requirements documentation is challenging, and incorrect or incomplete documentation has resulted in failed implementations [38]. Second, as BPGs evolve to reflect new evidence or adaptations, corresponding updates must be made to the CDSS as well. Thus, effort (and associated cost) must be expended to ensure that the medical knowledge is accurately updated in the CDSS as well.

To address these limitations, instead of a monolithic system, our approach decomposes the CDSSs into components mentioned earlier from [31], i.e., the knowledge-base, the user-interface, and additional infrastructure for integration with external data sources. To ensure that the knowledge-base accurately reflects medical knowledge, domain specific languages (DSL) that allow medical guidelines to be described in a manner that is both *computer interpretable* and *comprehensible* to healthcare practitioners are often utilized. A computer interpretable guideline (CIG) can be directly used as knowledge-base in a CDSS. But, by being comprehensible to HCPs, such a guideline can also replace its textual counterpart. Thus, such DSLs enable production of medical guidelines that can serve as both the system’s functional specification, i.e. the BPG, and implementation, i.e. the knowledge-base. This ensures that there is no gap between the BPG and the CDSS’ knowledge-base.

As CDSSs are safety-critical, the need for CIG DSLs to have a complete formal semantics that can be used to implement tools such as execution engines, model checkers, etc. has been recognized. It’s also desirable for such languages to have execution engines with formal *correctness guarantees*, and accompanying tools for formal analysis. To this end, some existing DSLs have partially defined semantics and support for verification via model-checking. However, as identified by the authors of [39, 40], existing languages lack a complete formal semantics, execution engines with correctness guarantees and a comprehensive suite of accompanying tools such, model-checkers, symbolic-execution engines, and deductive verifiers. The difficulties of formal analysis are further compounded by the fact that CDSSs

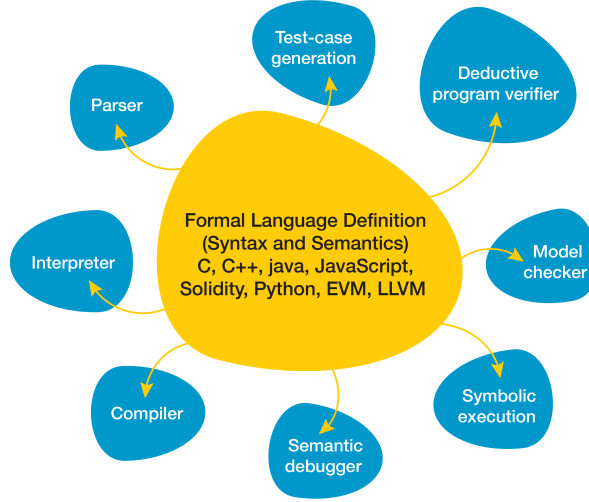


Figure 1.1: Semantics-first philosophy

are concurrent systems involving interactions with heterogeneous external agents such as sensors and HCPs, making their analysis challenging.

To address limitations of existing approaches, we developed a new language for computer interpretable guidelines called Medi $\mathbb{K}$. Medi $\mathbb{K}$ follows the *semantics-first* philosophy. As shown in Figure 1.1, this philosophy dictates that instead of implementing tools such as an interpreter, compiler, program verifier for a language from scratch in an ad-hoc way, the execution semantics of the language should be formally defined, from which said tools are derived, making them *correct-by-construction*. This also enables Medi $\mathbb{K}$ to incorporate healthcare practitioner feedback quickly and systematically, as only the semantics need to be updated. As the tools are derived from the semantics, changes to the semantics are automatically reflected across all tools.

Medi $\mathbb{K}$'s semantics have been formalized in $\mathbb{K}$ – a rewriting-based framework for defining programming languages [41]. Our choice for using $\mathbb{K}$ was motivated by (a) experience and expertise in developing $\mathbb{K}$, and defining languages in it, and, (b) the fact that $\mathbb{K}$ has been used to define other real-world languages such as C [42], Java [43] and the Ethereum Virtual Machine [44], and verify programs in them [45, 46].

The challenges associated with implementing safe CDSSs are also amplified by their inherent complexity, which stems from:

- (i) *concurrency* to account for multiple diagnoses and treatments occur simultaneously, and,
- (ii) interaction with heterogeneous *external agents* such as sensor, health records, and

practitioners.

To support formal analysis of such systems, a DSL for computer interpretable guidelines must consider said *complexity* while ensuring *comprehensibility* to healthcare practitioners.

To ensure *comprehensibility*, we ensure MediK-based guidelines *resemble* their existing textual counterparts that practitioners are already *familiar* with. We started by examining existing BPGs and the state-of-art DSLs for expressing them in a *computer-interpretable* way, and observed that BPGs are often expressed in informal flowchart-like notation. For instance, see [47] and [48] for flowchart-based BPGs for management of *cardiac arrest*, and screening, risk-reduction, treatment, and survivorship in cancer care. To tackle *complexity*, we utilized concepts from existing state-of-art in modeling and verification of large concurrent systems like P [49] and Maude [50]. In MediK, like in P, a program is a collection of concurrently executing state machines that interact by passing messages. Just as in object-oriented Maude, MediK treats external agents, such as the UI, sensors, etc., as state machines. Since BPGs are often expressed using flowcharts, we represent each flowchart in MediK as a state machine. For formal analysis, we need to model heterogeneous external components such as the UI, sensors, etc. We model such agents as *ghost machines*. These machines are only used for formal analysis, but are discarded during execution. During execution, the interface that glues all components is responsible for forwarding messages to and from external agents to MediK. This allows MediK to support a uniform way of modeling heterogeneous external agents for both execution and analysis.

In collaboration with the Children’s Hospital of Illinois at OSF St. Francis Medical Center (referred to as OSF in the remainder of this work), we used our MediK-based approach to implement a CDSS for management pediatric sepsis based on OSF’s guidelines. Sepsis is a life-threatening condition caused by the body’s extreme response to an infection [51], and is a major cause of morbidity and mortality in children [52]. Adverse outcomes can, however, be mitigated through timely identification and prompt treatment with antibiotics and intravenous (IV) fluids [53, 54]. BPGs for screening and management of sepsis in pediatric Emergency Departments (EDs) have shown effectiveness in screening and management of sepsis [52], leading to their adoption in many pediatric EDs [55, 56].

The MediK-based CDSS is a complex system, involving multiple concurrent workflows that interact with each other. As outlined earlier, the CDSS consists of three independent components:

- A UI tailored to the preferences of practitioners at OSF.
- A MediK based computer interpretable guideline based on OSF’s existing textual BPG.
- Additional infrastructure that glues said components together.

The knowledge-base of the CDSS is implemented as a MediK program. This computer interpretable guideline in MediK expresses medical knowledge *succinctly*, and allows establishing desired *safety* properties. For the sepsis management CDSS, we used MediK’s semantics-based model checker to establish *responsiveness*. We say the system is *responsive* if, under certain reasonable assumptions about the UI and interactions with external agents such as sensors, health records, all possible inputs to the system lead to a response. This ensures that the system does not “freeze” or “hang” when being used. But, beyond *responsiveness*, we need to ensure that the MediK program captures medical knowledge *accurately*. To this end, we try to ensure that MediK programs resemble existing flowcharts-based BPGs that practitioners are familiar with, enabling them to validate the semantics of the MediK-based computer-interpretable guideline. Once verified and validated, the MediK program can be combined with a customizable UI to obtain a complete CDSS with safety guarantees. To the best of our knowledge, ours is the first comprehensive CDSSs for sepsis management to provide any *safety guarantees*.

Recall from [section 1.1](#), the four challenges from the NAM special report on CDSSs. Our approach’s modular architecture and use of MediK for computer-interpretable guidelines tackles all challenges. MediK based guidelines provide a *systematic* way of *validating* medical knowledge that can be easily updated, thus addressing C1. Comprehensibility of MediK to practitioners ensures transparency. Practitioners can rely on the system to perform as expected, partly addressing C2. Our approach is agnostic to IT being utilized at the hospital. As mentioned earlier, our approach requires the implementation of a hospital-specific interface that glues the MediK-based computer interpretable guideline and the UI with the hospital’s IT infrastructure, addressing C2, C3 and partly C4. Moreover, our approach allows the UI to be safely customized, as any changes remain contained to the UI itself, and safety-critical components such as the knowledge-base remain untouched. This addresses C4, as the UI can be optimized to a hospital’s preferences. Moreover, as both the MediK-based guideline and UI are shareable, the cost for deploying a CDSS is lower than building a standalone, non-modular system from scratch.

CHAPTER 2: BACKGROUND

In [chapter 1](#), we conceptually introduced best practice guidelines (BPGs) and clinical decision support systems (CDSSs). This chapter introduces relevant background details on BPGs and CDSSs. To this end, in [section 2.1](#), we utilize a real-world BPG to explain, in detail, what such guidelines are, and motivations behind codifying treatment in the form of clinical guidelines. We then discuss certain characteristics that are common across BPGs, with the intention of describing how they influence design of domain-specific languages for expressing knowledge-base.

BPGs are usually published by hospitals, research institutions and medical associations with the aim to improve quality of care by (a) reducing medical errors due to preventable causes, (b) standardizing knowledge from latest evidence-based research, and, (c) enabling access to aforementioned knowledge at medical establishments that lack resources to conduct research.

While in theory, following BPGs should improve clinical outcomes, their effectiveness in practice is dictated by whether healthcare practitioners follow them or not. In [section 2.2](#), we discuss reasons behind non-conformance to BPGs in clinical settings. We describe how non-conformance results in worse patient outcomes, and, argue that computerized systems that utilize data from available heterogeneous sources such as electronic health records and sensors for patient parameters can improve patient outcomes by enabling BPG-conformance in practice.

2.1 CLINICAL BEST PRACTICE GUIDELINES

Clinical best practice guidelines are evidence-based statements published by hospitals and medical associations that codify recommended interventions for various clinical scenarios [\[57\]](#). It has been recognized that, if implemented correctly, BPGs have the potential to:

1. Reduce unwarranted variations in clinical practice.
2. Improve healthcare safety and quality.
3. Enhance translation of research into practice.
4. Reduce healthcare costs.
5. Enable development of performance measures for diseases [\[58, 59\]](#).

Guidelines in the form of statements and recommendations were first introduced in the 1970s, and, were mostly based on expert opinion. In the 1980s and the early part of 1990, a significant increase in evidence from proliferation of randomized controlled trials occurred. This coincided with the introduction of computers at medical institutions that enabled available evidence to be quickly accessed to aid decision making [60, 61]. These developments facilitated a shift towards more rigorous development of BPGs, where evidence was prioritized over expert opinion [58].

To be able to administer optimum care, practitioners must consider evidence from ongoing research to guide treatment. This can be particularly challenging, as available evidence can evolve rapidly. BPGs enable findings from latest research to be quickly translated to practice. Well-designed BPGs are usually developed and published by trusted medical establishments (such as medical associations and research institutions) that base recommendations on best available evidence [58]. As the body of evidence increases over time through research, BPGs must be updated accordingly to reflect latest findings.

To illustrate characteristics of BPGs, we briefly go over a BPG for managing sepsis in

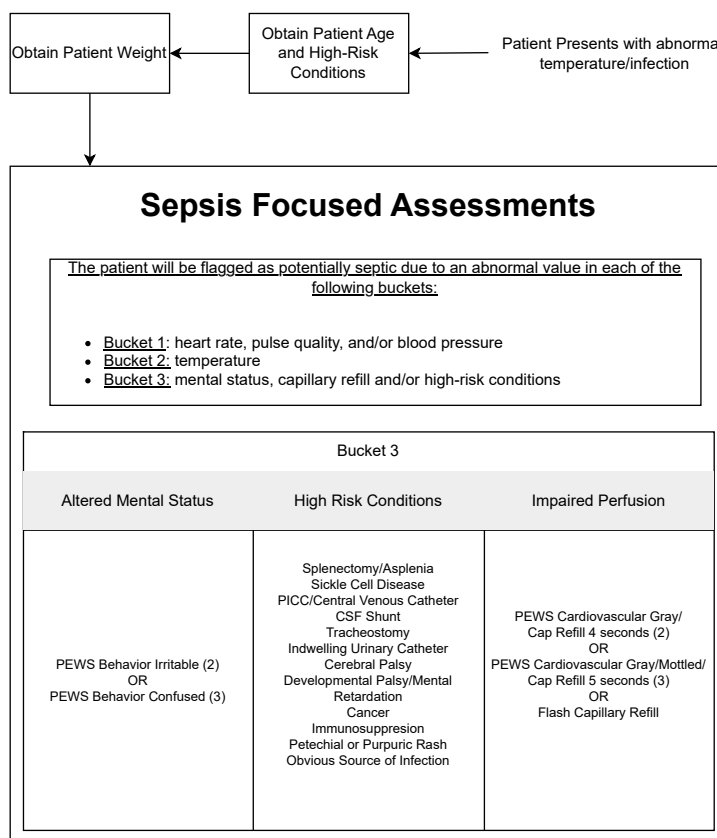


Figure 2.1: Pediatric sepsis screening BPG

Age	Heart Rate	Systolic BP	Temp
$0d - 1m$	> 205	< 60	< 36 or > 38
$\geq 1m - 3m$	> 205	< 70	< 36 or > 38
$\geq 3m - 1y$	> 190	< 70	< 36 or > 38.5
$\dots$	$\dots$	$\dots$	$\dots$
$\geq 13y$	> 100	< 90	< 36 or > 38.5

Table 2.1: Vital Signs Chart

pediatric cases used at OSF St. Francis Medical Center in Peoria, Illinois – a major pediatric hospital in the United States. Note that for brevity, we refer to said hospital simply as OSF in the remainder of this section. Sepsis is a life-threatening condition caused by the body’s extreme response to an infection [51], and is a major cause of morbidity and mortality in children [52]. Adverse outcomes can, however, be mitigated through timely identification and prompt treatment with antibiotics and intravenous (IV) fluids [53, 54]. BPGs for screening and management of sepsis in pediatric Emergency Departments (EDs) have shown effectiveness in screening and management of sepsis [52], leading to their adoption in many pediatric EDs [55, 56].

In Figure 2.1, we present a simplified version of the screening section of OSF’s sepsis management guideline. In essence, when a patient arrives at the ED with a fever or an infection, the HCP is supposed to obtain (a) the patient’s age, (b) any conditions, such as cancer, immunosuppression, etc., that increase likelihood of sepsis, and (c) the patient’s vital signs, such as heart rate, systolic blood pressure, respiratory rate, etc.

This information is then used to check for abnormalities in clusters of linked information, called “buckets”. For instance, if the patient’s heart rate is abnormal, then “bucket 1” is said to have an abnormal value. Checking for such abnormalities often involves the use of tables, such as Table 2.1 that contains normal ranges indexed by *age*. If the patient has at least one abnormal value in every “bucket”, then he/she is flagged as potentially septic.

The BPG-recommended treatment for sepsis involves multiple concurrent workflows, such as screening for septic shock, fluid resuscitation, and administering antibiotics. In Figure 2.2, we provide a version of the fluid resuscitation guideline used at OSF. Briefly, if the patient is flagged as potentially septic, the guideline suggests (i) obtaining any fluid-overload risks, (ii) administering normal saline (typically over a period of 15 minutes), where the dosage is dictated by risks determined in previous step, (iii) assessing signs of fluid-overload, (iv) evaluating patient responsiveness to normal saline upon completion of the administering process, and, (v) determining whether another fluid bolus should be administered based on information from previous steps.

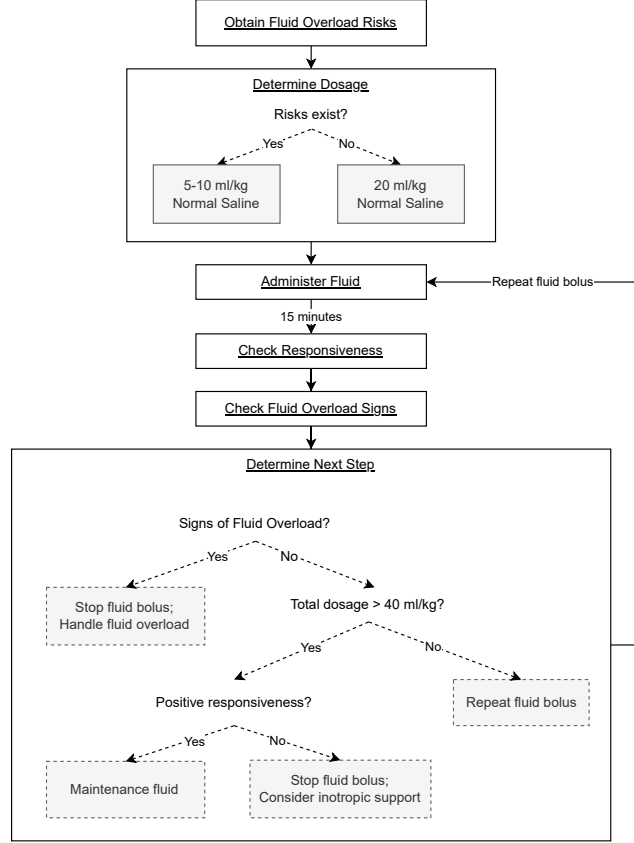


Figure 2.2: Fluid Resuscitation Guideline

This real-world BPG exhibits characteristics common across many BPGs. Specifically, BPGs typically:

- Involve *concurrent* workflows, such as administering drugs, monitoring vitals, performing treatment, etc. There may also be inter-workflow interactions. For instance, a diagnosis of sepsis during the screening may require modifications to an ongoing course antibiotics.
- Often specified in a *flowchart-like* notation. See [47] and [48] for other flowchart-based BPGs for management of *cardiac arrest*, and screening, risk-reduction, treatment and survivorship in cancer care respectively.
- Require communication between *heterogeneous agents* such as monitors and Electronic Health Records (EHRs).
- Often use *tables* indexed by parameters such as age, weight, etc. to present normal/abnormal ranges for measurements, or recommended dosages for drugs.

2.2 CLINICAL DECISION SUPPORT SYSTEMS

BPGs are developed with the intention of improving patient outcomes by reducing preventable medical errors, and their potential has resulted in them being widely adopted in daily practice [25]. However, BPG adoption in clinical settings has several challenges. BPGs are complex documents, which may also contain some vagueness. Exact meaning of terms is not always defined and recommended actions may not be clearly articulated. The long and cumbersome nature of BPGs may make them difficult to effectively apply in regular practice. Additionally, such BPGs need to be periodically updated to take into account latest evidence, and, adapted according to local needs of the setting they’re utilized in [62].

2.2.1 Deviations From BPG-Adherence

The reasons why healthcare practitioners deviate from guidelines can be hard to generalize as there may be reasons for non-compliance may differ between guidelines. In general, even though HCPs recognize the BPGs-compliance may lead to better outcomes, changing routines to adopt them may still be challenging. According to [63], reasons for non-compliance can include:

- The complexity of, or ambiguity in, the medical knowledge in the guideline. Advice within a guideline can be unclear, or inconsistent with advice from other guidelines.
- Concerns over knowledge in the guideline being obsolete.
- Overconfidence in their own ability over BPG-based advice, or, perception that guidelines were followed when they were not.
- Pressure due to resource and time constraints, especially in guidelines that deal with time-sensitive interventions.

Improving Safety through Better Adherence: To mitigate aforementioned issues, systems that utilize available patient data to assess patient state and issue guideline-based decision support to practitioners can be utilized [62]. Specifically, such systems attempt to integrate with existing patient care to administer support as seamlessly as possible. This may require either changes to existing care workflows to incorporate decision support, or, building decision support that minimizes disruptions. Such computerized clinical decision support systems (CDSSs) codify medical knowledge in BPGs and provide practitioners with situation-specific advice that “guides” them towards adherence to the underlying BPG.

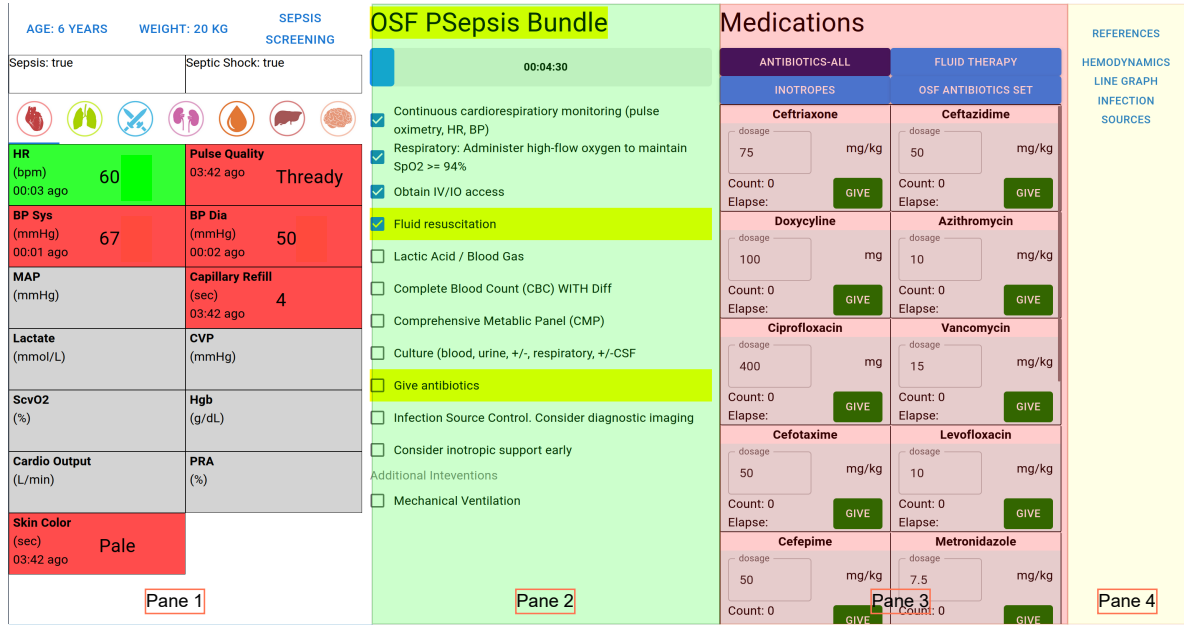


Figure 2.3: Sepsis Tool Main Screen

Well implemented CDSSs have been shown to improve BPG-adherence [26, 27], and reduce PMEs based on evidence from multi-center clinical trials [28, 29]. In the following section, we explain how such systems function through an example.

2.2.2 CDSS Example

In section 2.1, we presented a real-world BPG for screening and management of sepsis in pediatric cases. To illustrate how a CDSSs can enable better BPG adherence, we utilize an example CDSS that codifies the sepsis management BPG to administer decision support.

Figure 2.3 and Figure 2.4 show an overview of a CDSS for sepsis management. The CDSS is intended to work as tablet at the patient’s bedside in a hospital’s emergency department (ED). Figure 2.3 depicts main screen that the practitioner sees on the tablet. The main screen consists of four panes (labeled and color coded for illustration purposes). Pane 1 shows a grid with the patient’s vitals pulled from a combination of the patient’s records, sensor measurements and assessments made by practitioner through prompts that appear on the screen. Red and green depict abnormality and normality respectively, while gray indicates missing measurements, which are assumed to be normal. As more measurements become available, corresponding boxes in the grid are updated to reflect their status. The two boxes in the top left corner depict whether the patient is suspected to have sepsis, and, be in septic shock. The CDSS continually monitors these vitals to determine whether they

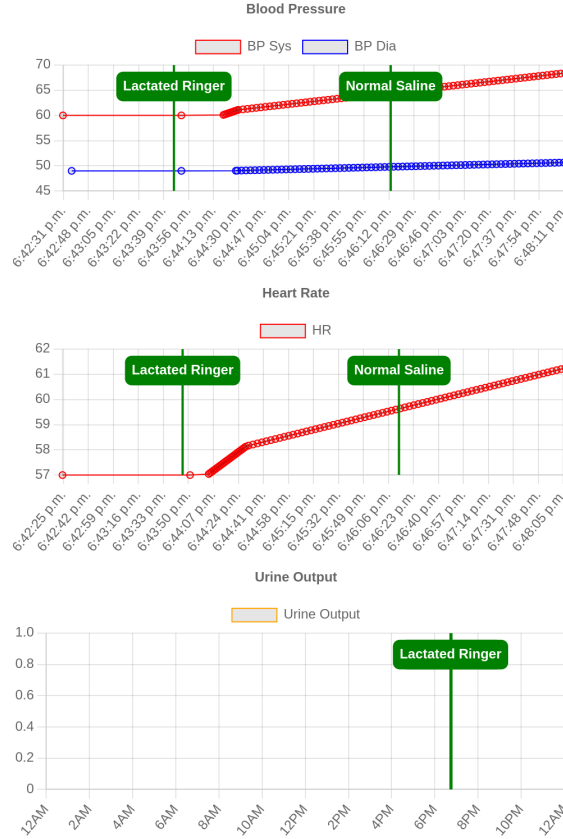


Figure 2.4: Patient Vitals vs Time

satisfy the criteria for sepsis or septic shock specified in the hospital’s guideline, where the guideline itself is a program in a computable medium that the practitioners can view and comprehend themselves. Pane 2 shows the tasks the practitioner is expected to perform once sepsis is detected. As these tasks are supposed to be performed within an hour (referred to as golden hour in sepsis treatment), the CDSS tracks time since the first task is started by clicking the corresponding checkmark. For some complex tasks, such as fluid resuscitation and antibiotics administration, an auxiliary window guides the practitioner through the task. The example scenario shows a patient with concerning vitals despite having given multiple fluid boluses, indicating septic shock—a condition that demands immediate attention. Thus, as specified in the guidelines, the system recommends urgently administering antibiotics alongside fluids, by highlighting the relevant tasks in yellow. Pane 3 contains medications grouped by type, where clicking the “GIVE” button corresponds to administering the drug. Pane 4 leads to toggleable windows with important information. For instance, clicking on “HAEMODYNAMICS LINE GRAPHS” toggles the window in [Figure 2.4](#) depicting the patient’s vitals over time punctuated by points at which drugs were administered.

2.2.3 CDSS Components

CDSSs were first introduced in the 1970s. Initial systems didn't integrate well with existing patient care workflows, and didn't find greater adoption outside academia. However, wider adoption of electronic health records and digital systems for patient parameters enabled better integration of CDSSs with workflows in everyday practice, enabling wider adoption. Modern CDSS can deliver recommendations through a diverse spectrum of devices such as systems at the patient bedsides, desktops or tablets carried by practitioners and smartphones. This versatility has further enhanced CDSSs adoption [31].

In Figure 2.5, we provide an overview of the components of a typical CDSS, and the underlying architecture. A typical guidelines-based CDSS conceptually consists of the following components [31]:

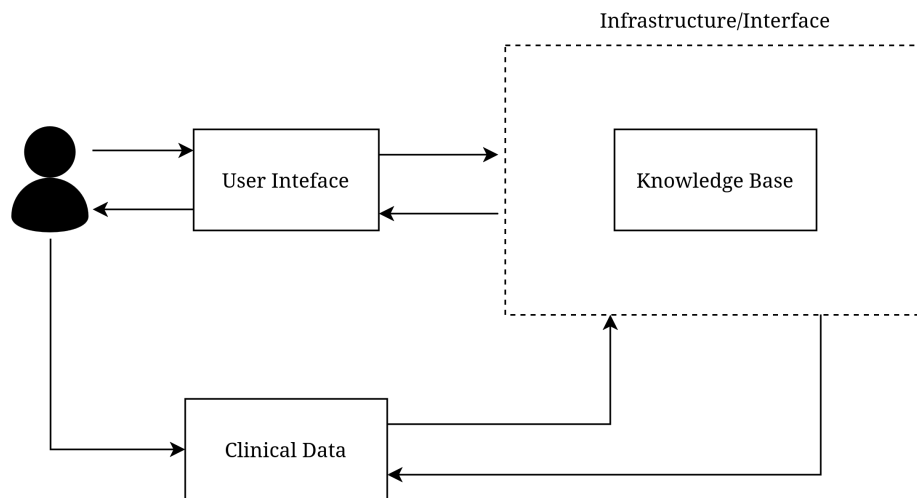


Figure 2.5: CDSS Components

Knowledge Base: The knowledge base is the encoding of the underlying guideline in a computable medium. Recall from section 2.1 that BPGs are typically developed experts in medicine and published as textual documents. To develop a CDSS, experts in medicine collaborate with computer scientists or software developers to develop requirements documentations that present the BPG's semantics in a manner amenable to software development. This is subsequently utilized by software developers to encode knowledge from the textual BPG into some programming language [36]. For instance, the CDSS for sepsis management from subsection 2.2.2 utilizes the textual BPG shown in Figure 2.1 and Figure 2.2 to diagnose and manage sepsis. Thus, medical knowledge in the textual BPG must be translated into some programming language to enable its use in a functioning CDSS.

Clinical Data: CDSSs utilize clinical to assess patient state and generate recommendations as specified in the underlying BPG. Typically, this involves utilizing available data from various sources such as electronic health records, devices that monitor various patient vitals, and inputs from HCPs.

Consider the sepsis screening BPG in [Figure 2.1](#). To screen a patient for sepsis, data and measurements such as the patient’s age, associated high-risk conditions, mental status, heart rate, etc. is required. The origins of the data are generally very diverse. Information such as the patient’s age, weight and associated high risk conditions remain static over the duration of sepsis management and can be obtained from the patient’s health records. On the other hand, information such as the patient’s mental state or the condition of the patient’s skin necessitates an assessment from an HCP at the patient’s bedside before it’s manually entered into the system. Finally, for patient parameters such blood pressure or mean arterial pressure that change dynamically during treatment, sensors or monitors that provide them in real-time should ideally be utilized. Typically, CDSS must take said diversity in data sources into account to be effective.

User Interface (UI): The UI is the interface the HCPs utilize to interact with the system. The UI typically:

- Delivers guideline-specified information (recommendations, alerts, etc.).
- Facilitates input of necessary data from the practitioner.
- Displays relevant patient information (vital signs, treatment progress).

The UIs need to account for the diversity in devices used in modern healthcare. Interaction can occur through desktops, tablets, or even cell phones. The UI of our running example CDSS for managing sepsis is meant to be displayed on tablet at the patient’s bedside and is shown in [Figure 2.3](#). The displayed information is organized into distinct “panes”: Pane 1 for patient state, Pane 2 for treatment progress, Pane 3 for medication details, and Pane 4 for textual guideline references. Time-sensitive and critical information appears as overlaid popups/prompts.

Additional Infrastructure Finally, additional infrastructure is required to glue all aforementioned components into a functioning CDSS. Said infrastructure typically consists of software to:

- Integrate with a healthcare establishment's existing Information Technology stack for access to electronic health records, and systems for services such as drug order and delivery, HCP communication, etc.
- Interface with various devices, including sensors and monitors, that track patient parameters like heart rate, blood pressure, etc.
- Control treatment-related devices such as infusion pumps, ventilators, etc.

For instance, the example CDSS for sepsis management shown in [Figure 2.3](#) integrates requires additional software for integration with electronic health, several monitors for patient parameters such as heart rate, blood pressure, mean arterial pressure, etc. and the hospital's drug order and delivery systems to be effective.

CHAPTER 3: HURDLES TO CDSS ADOPTION

There is now increasing evidence to suggest that well implemented CDSSs can significantly improve quality of care [64, 65]. However, despite several advantages, several challenges continue to inhibit wider CDSS adoption [34]. Some challenges are non-technical, i.e., require changes to legislation, incentive mechanisms and practitioner education and training, and are beyond the scope of this work. But several limitations in existing CDSS technology have also inhibited further adoption. This chapter discusses said challenges, and the progress made by existing state of art towards addressing them.

Recall, from [section 1.1](#), that in 2017, the National Academy of Medicine published a report on CDSSs that laid out a roadmap to optimize CDSS uptake in medicine. According to the report, several challenges need to be tackled for wider adoption, such as:

- C1. Absence of systematic ways of *validating content* in a *reliable*, *accessible* and *updateable* manner.
- C2. Lack of *reliable*, *shareable* CDSS content that can be easily adopted across healthcare organizations and their (Information Technology) IT systems.
- C3. Technical difficulties of sharing due to *need for adaptation* to diverse Electronic Health Records (EHR) systems.
- C4. *Suboptimal* User Interfaces (UIs), implementation choices and workflows.

3.1 ADDRESSING ADOPTION HURDLES

CDSS first appeared in the 1960s and have evolved over time to address aforementioned challenges. We describe this progress in upcoming chapters.

3.1.1 Monolithic CDSSs

Early CDSSs were developed as monolithic standalone systems that were self-contained, requiring direct user input for clinical data [66]. Such systems co-existed with primitive electronic health records (EHR) systems, and thus had to rely on manual data entry before administering support.

Several successful CDSSs implementations utilized a standalone architecture. Early CDSS implementations such as MYCIN [67] required the HCP to answer a set of questions to provide advice regarding microbial therapy. Other early CDSSs such as DXplain [68] utilized

a wide range of findings (history, data, etc.) to come up with a diagnosis, and is still in active development.

The reliance on manual entry made using such systems time-consuming. As support for EHR matured, CDSSs implementations became better integrated with EHR systems for automated clinical data retrieval. However, with monolithic systems, the integration was usually EHR-specific [66]. Migrating or sharing CDSS content across medical establishments presented significant challenges as a system designed for an establishment’s EHR system couldn’t be used with a different establishment’s EHR system [69].

3.1.2 Modular CDSS Architectures

In [subsection 2.2.3](#), we presented the components that every guidelines-based clinical decision support can conceptually be decomposed into. Early CDSSs from [subsection 3.1.1](#) were not designed with modularity that enabled sharing components between different implementations. As the need for scaling CDSSs across institutions grew, component-based architectures that enabled EHR agnostic systems to be developed became prevalent [69].

EHR-agnostic architectures represent a significant step towards addressing several challenges. Such architectures address C3 as the knowledge-base can be shared across institutions with different EHR systems. C2 is partially addressed as the knowledge-base can be independently developed, maintained and distributed. C4 is also partially addressed as decoupled components, such as the UI, are easier to adapt to HCP preferences.

Over the years, several CDSS implementations have utilized a components-based architecture. For instance, in [69], the authors utilize the service-oriented architecture [70] to build a CDSS web service that can be utilized in a completely EHR-agnostic way. Recent efforts include CDSSs platforms such as EvidencePoint that enable CDSS to be integrated closely with the hospital’s EHR without being tightly coupled [71]. EHR-agnostic architectures allow decision support to be administered using the EHR’s UI. Given their prevalence in modern medical establishments, EHR systems have become integrated into workflows, and HCPs are accustomed to using them. Dispensing clinical decision support through the EHR’s UI is vital for adoption, as better workflow integration can lead to higher adoption [72, 73].

Approaches that utilize a component-based architecture enable medical knowledge to be shared more efficiently. But, it’s possible for medical knowledge itself to be incorrectly encoded. BPGs are generally expressed as long textual documents meant to be understood by HCPs [74]. To build a CDSS, the BPG must be systematically expressed in a computable medium. This translation process, referred to as knowledge formalization, is generally ad-hoc, and can be the source of inconsistencies in encoded medical knowledge resulting in CDSSs

that render wrong advice [75].

Typically, to translate textual guidelines to a computable medium, experts in medicine collaborate with computer scientists and software developers to come up with a requirements document. This is subsequently utilized to develop the knowledge base, i.e., the computer interpretable encoding of the BPG [36] in a traditional programming language. Thus, the BPG as a functional specification for the knowledge base. For instance, in the case of EvidencePoint, the knowledge base is expressed in Javascript [71].

However, since computer scientists/software developers don't understand medicine, and experts in medicine typically don't understand programming languages, it's possible for the functional specification, i.e., the BPG, to diverge from its implementation, i.e., the knowledge base. Thus, C1 from [chapter 3](#) remains unaddressed.

3.1.3 Computer-Interpretable BPGs

CDSSs are safety-critical systems, where bugs can have serious consequences. Thus, ensuring correctness of CDSSs is vital to widespread adoption. To ensure bugs arising out of divergences between the textual BPG and its computable translation, i.e., the knowledge base, the gap between the BPG and the knowledge base must be eliminated. To this end, instead of encoding BPGs in a conventional programming language, domain-specific languages (DSLs) for directly expressing BPGs in a computer-interpretable manner can be utilized. By emphasizing comprehensibility to HCPs, these DSLs enable HCPs to validate the accuracy of encoded medical knowledge. A computer-interpretable BPGs can both as the specification, i.e. the textual BPG, and the implementation, i.e., the knowledge base, thereby eliminating the specification-implementation gap.

In 1989, the Arden Syntax was developed as a result of incompatibility of medical knowledge between medical institutions [76]. While it was designed to represent simple guidelines, such as those related to reminders, more complex treatment protocols couldn't be represented. Arden syntax had a formal Backus-Naur Form (BNF) syntax definition, but no formal semantics, or a standardized execution engine. Instead, ad-hoc execution engines for the language have been developed over time [37].

Shortcomings regarding ability of the Arden Syntax to represent complex treatment protocols were addressed by formalisms such as GLIF [77] that permit encoding complex guidelines using a multi-layer approach to represent both high-level medical knowledge and low-level implementation details. However, just like the Arden Syntax, GLIF lacks a formal semantics or formal analysis tools.

The need for formal analysis is identified by Asbru: a formalism with formally defined syntax

and semantics [40]. In Asbru, a guideline is modeled as a plan that contains: (i) intentions that define aims, (ii) conditions that specify when the plan is applicable, (iii) effects that define expected behavior during execution, and, (iv) a body containing other subplans. Apart from an execution engine, the Asbru ecosystem also contains other tools, such as a model checker for verification [78]. However, the formal semantics of Asbru have been only partially defined, and is insufficient to implement tools for the language [39]. The importance of a complete formal-semantics is identified and addressed by PROforma [39], another formalism that uses plans to model guidelines. A PROforma plan is made of a sequence of tasks. The plan defines constraints on their enactment, and circumstances for termination (for example, exceptions) [39]. But, despite having complete formal semantics, PROforma’s semantics is not executable. Therefore, an interpreter and analysis tools must be implemented in an ad-hoc manner.

Several endeavors, mainly academic, but also involving the industry towards making CDSSs safer, easier, effective and cheaper. We briefly talk about some related approaches in this chapter to provide an overview of existing progress and remaining questions but discuss them in greater detail in [chapter 4](#), along with other related work. In [subsection 3.1.1](#), the earliest CDSSs attempted to increased adherence to evidence-based best practices at medical establishments. To ensure CDSSs could be scaled better, components-based architectures, discussed in [subsection 3.1.2](#) enabled medical knowledge to be developed and maintained independently of other components (such as system UI). To ensure that medical knowledge is indeed encoded correctly, several DSLs that eliminate the gap between a textual BPG and the CDSS’ knowledge base have been developed. However, the following issues still remain:

- Interpreters and compilers for said DSLs are developed in an ad-hoc way and remain prone to bugs that may manifest during execution.
- There is a lack of formal analysis tools such as model checkers and program verifiers that can be utilized to establish that the computer-interpretable BPGs satisfy desired safety and liveness properties.
- DSLs lack complete formal semantics that can be utilized as a language model for a suite of formal analysis and execution tools.

Addressing these issues is vital to improving CDSS adoption. To have trustworthy and useable computer interpretable BPGs, it’s important to buttress HCP-friendly DSLs with a suite of formal analysis tools that can establish desired safety properties, thus comprehensively addressing C1.

CHAPTER 4: RELATED WORK

In [chapter 3](#), we provided a brief overview of existing approaches and their limitations. This chapter provides a comprehensive discussion of related approaches. Recall from [subsection 3.1.2](#) that implementing a guidelines-based clinical decision support system requires collaboration between experts in medicine and software engineers for knowledge formalization, i.e., the process of encoding medical knowledge in textual BPGs in some programming language. Using a conventional programming language for knowledge formalization can lead to an inaccurate encoding of medical knowledge, as experts in medicine, being unaccustomed to computer programming, cannot validate the accuracy of the encoding. To address this, DSLs designed specifically for expressing BPGs in a computer-interpretable format are utilized. BPGs expressed in such languages can serve as both textual guideline documents and knowledge bases in CDSSs. But medical knowledge in BPGs has also been expressed and formalized using other non-domain-specific approaches. The discussion in this chapter has also been split along the same lines. In [section 4.1](#), we discuss approaches involving DSLs for computer interpretable guidelines. In [section 4.2](#), we discuss other approaches that aren't specific to BPGs.

4.1 DSL-BASED APPROACHES

4.1.1 Arden Syntax

The Arden Syntax is among the earliest and most widely used standards for expressing medical logic, with the first draft of the standard appearing in 1989. It was also among the earliest attempts to create a domain specific language specifically for use in CDSSs [[79](#)].

In Arden Syntax, code is organized into self-contained medical logic modules (MLMs) that have a well-defined structure to separate higher-level medical logic from low-level implementation details such as variable declarations. The language continues to evolve to accommodate diverse uses and is supported by multiple execution engines [[80](#), [81](#)].

The maturity of the Arden Syntax platform is reflected in its use to implement a large and diverse set of CDSSs. These include:

- Systems for monitoring and infections surveillance [[82](#), [83](#)].
- Treatment planning and decision support [[84](#), [85](#)].

Being one of the earliest DSLs specifically designed for CDSSs, the Arden Syntax has found widespread adoption. But the language also has several limitations. Notably:

- Support on simple, independent guidelines instead of complex treatment workflows [37].
- Lack of formal semantics and clarity in the language specifications [79].

4.1.2 Dilemma

The Dilemma project was among the first attempts at standardizing BPGs to originate out of Europe. In Dilemma, a protocol describes the set of activities that must be carried out to reach an objective based on clinical indications [86]. Protocols are recursively broken into smaller elementary protocols. Each protocol has one or more associated indication that determine when it becomes applicable, and a set of objectives that define the expected outcome.

Applying a protocol to a patient state is defined as a procedure, where leaf-level procedures are specifically referred to as activities. Activities are carried out by agents (usually HCPs such as doctors, nurses, etc.), and typically involve:

- Maintaining or changing patient state through treatment steps.
- Obtaining information or confirming procedures.
- Communication or interaction with other agents.

Dilemma was utilized to implement guidelines from several European countries, such as primary care guidelines from the Netherlands and the UK and cancer treatment guidelines from France [86].

4.1.3 GEODE-CM

Guided Entry of Data Elements for Clinical Management (GEODE-CM) was an early DSL developed by the Decision Systems Group at Harvard University Medical School. The GEODE-CM DSL utilized a state machine-based architecture to represent medical knowledge. Clinical problems in GEODE-CM were broken down into clinical management states, where entry, exit and transitions between states were determined by data collected during execution [87]. Each clinical management state was represented as state machine nodes, and edges between nodes represented transitions between the various clinical management states [88].

4.1.4 EON

The EON language, developed at Stanford University, also utilizes a state machine-based architecture for representing medical guidelines. Treatment protocols are first recursively composed into smaller granular elementary protocols that cannot be decomposed any further. The protocols are then represented via directed multigraphs, where nodes represent both patient and treatment state, and transitions represent actions or changes in patient conditions [89].

EON has many similarities to GEODE-CM. Medical knowledge in both DSLs is expressed using a finite-state machine notation, where nodes are patient or treatment states and edges represent actions or changes in state. However, EON, unlike GEODE-CM:

- Has an informal, yet comprehensive operational semantics.
- Support for sequencing, looping and synchronization constructs to support guidelines with multiple concurrent actions [89].

While Arden Syntax emphasizes expressing medical logic using independent, modular medical logical modules, EON state machines are more tightly coupled, enabling representation of complex protocols and guidelines [89]. EON has been used to implement complex CDSSs for management of conditions such as AIDS [90] and Breast Cancer [89].

4.1.5 GLIF

The Guideline Interchange Format (GLIF) was introduced in 1998 through a collaboration between researchers from Columbia University, McGill University, Harvard University and Stanford [37]. Good medical guidelines improve healthcare quality but require substantial work to develop and maintain. At the time, guidelines were published through unstructured text documents that were not easily shareable. The motivation behind the GLIF project was to enable BPG between institutions sharing through the development of:

- A standardized electronic format for rapid dissemination.
- A repository of guidelines to prevent duplicated effort.
- Tools that HCPs could utilize to retrieve and execute relevant guidelines.
- Analysis tools that enabled authors to develop and publish high-quality, unambiguous guidelines.

The GLIF team comprised of research groups from various universities where other languages, including Arden Syntax ([subsection 4.1.1](#)), GEODE-CM ([subsection 4.1.4](#)), were developed. Thus, perspectives and learnings from existing efforts guided the development of GLIF, including borrowing certain useful constructs directly from EON [88]

In GLIF, guidelines are expressed using the GLIF class containing relevant attributes corresponding to treatment information. One such attribute is an unordered list of all guideline steps. The guideline specification itself is a directed graph defined over collection of said steps. A step can be one of the following:

- Action steps: Specify clinical-care actions during treatment.
- Conditional steps: Determine control flow based on logical statements.
- Branch steps: Enable flow to multiple guideline steps concurrently.
- Synchronization steps: Converge concurrent flow back to a single guideline step [88].

GLIF was used to implement several CDSSs, such as a web-based application for support in clinical consultations [91] and the Partners Computerized Algorithm Processor and Editor (P-CAPE) system for creating and deploying BPGs [92].

The experience and lessons from implementing these CDSSs in GLIF also revealed certain limitations of the language, such as:

- Integration of heterogeneous clinical systems was challenging.
- Guidelines were cumbersome to develop, and, readability was suboptimal.

To address these shortcomings, an updated version of the language called GLIF3 was introduced [77]. GLIF3 utilized different abstraction levels to present relevant information based on the user’s role. Guidelines are modeled using three hierarchical “abstraction levels”:

- i. The conceptual level, where the guideline is represented as a flowchart for consumption by HCPs.
- ii. The computable level, where information regarding execution exists to enable use in a CDSS.
- iii. The implementable level, where additional information regarding integration with institution-specific EHRs resides.

Like GLIF, GLIF3 has also been used to implement several CDSSs, including systems for depression screening and management [93] and hyperkalemia patient screening [94]. But despite its use in several systems, the semantics of the GLIF framework remain only informally defined.

4.1.6 PROforma

PROforma was initially developed at the Cancer Research UK Advanced Computation Laboratory and has been utilized to develop several CDSSs [37]. In PROforma, guidelines are modeled as tasks and data items. Tasks are hierarchically organized into plan, and may be further divided into:

- Actions: Procedures that must be performed in an external environment.
- Enquiries: Guideline points at which data must be obtained.
- Decisions: Points at which a choice determines further control flow [39].

PROforma guidelines can be visualized as directed graphs, with nodes representing actions and edges constraints on control flow. Pictorially, actions are represented as squares, enquiries as diamonds and decisions as circles. This enables PROforma guidelines’ visual representations to resemble flowcharts and medical algorithms that HCPs are already familiar with.

A PROforma task has associated properties that determine how it’s interpreted. These include:

- Captions and descriptions that improve guideline comprehensibility.
- Preconditions that specify conditions for a task’s execution to begin.
- Scheduling constraints that enable synchronization between concurrent tasks.

Additionally, plans have termination and abort conditions that specify successful end of the current plan to facilitate execution of successor plans.

Using PROforma requires a suite of tools called Tallis [95], written in Java, that include a composer for creating and viewing guidelines, a tester for debugging them, and an execution engine for running them [39].

PROforma has been used to implement several CDSSs that have demonstrated improvements in care quality in clinical settings. Among the first applications of PROforma was the CAPSULE system that assisted general practitioners prescribe medicines [96]. Similarly, the LISA system was designed to HCPs with dosages for pediatric patients with acute lymphoblastic leukemia [97]. Other applications include support with image interpretation and diagnosis [98], and support for breast cancer diagnosis and treatment [99].

PROforma is also among the first DSLs to identify the for a formally defined syntax and semantics and expend significant effort in doing so. In [39], the authors enunciate that PROforma “must have” open source syntax and semantics definitions to ensure that medical logic in PROforma is unambiguous, and to ensure tools “correctly” read and process PROforma.

4.1.7 Asbru

Asbru, developed by collaborators from Stanford, Vienna University of Technology, and Ben Gurion University, focuses on time-oriented clinical guidelines [37]. The language attempted to address limitations from existing approaches such as Dilemma (subsection 4.1.4) such as:

- Lack of human-readability.
- Inability to integrate well with electronic patient records.
- Inability to model flexibility in guideline-prescribed steps.

To address these shortcomings, especially readability and modeling flexibility, Asbru utilizes a skeletal plans-based approach. Skeletal plans are a sequence of generalized steps that are instantiated in a specific problem context to solve a given problem [100]. Thus, skeletal plans provide a broad framework for solving a class of problems, instead of fine-grained details required to solve a specific problem. This flexibility enables Asbru guidelines to have intended objectives that can be achieved in different ways. For instance, to manage diabetes, an HCP can either administer additional insulin, or, instruct the patient to consume less carbohydrates during a meal. Both actions have the intended outcome of moderating blood sugar levels. Specifically, an Asbru skeletal plan based guideline has:

- Actions to be performed by the HCP.
- An intended plan made of action described as pattern of actions.
- The intended outcome described through patterns over patient states [101].

When executed, the HCP applies the guideline by following actions, which are observed, recorded and abstracted over time into an abstract plan. Both patient state and the inferred intention are also recorded [101]. Asbru’s execution model also provides the ability to critique execution along these axes:

- Guideline-prescribed actions vs HCP followed actions.
- Guideline-intended plan vs HCP followed plan.
- Guideline-intended state vs the HCP’s intended state.
- Guideline-intended state vs actual patient state.
- HCP’s intended state vs actual patient state.

These axes provide 32 different scenarios of guideline execution [101].

Asbru provides comprehensive support for temporal aspects of guidelines by using time reference annotations. These represent uncertainty in starting, ending and duration of time intervals. An annotation can be an absolute reference point, a reference point with uncertainty (defined by a region), or a function of some previous plan. Deviation from the reference point represent uncertainty in starting. Because of these annotations, intervals have an earliest start shift, latest start shift, earliest finishing shift, latest finishing shift and minimum duration and maximum durations [101].

A guideline in an Asbru library consists of a set of plans and corresponding time annotations. During execution, the interpreter attempts to decompose the plan into subplans from the library. If a plan is non-decomposable, it is considered to be atomic, usually corresponding to actions to be performed by an external agent such as obtaining information, administering treatment, etc.

Library plans have four states: considered, possible, rejected and ready that determine whether a plan is applicable. During execution, a ready plan is instantiated. Apart from the name, arguments and time annotation, a plan also has:

- Preferences: Factors that influence the selection of a plan to achieve a goal. Examples include start conditions for the plan, heuristics (such as exact-fit) to specify a selection criterion, etc.
- Intentions: High level goals of the plan.
- Effects: Functional relations between arguments and measurable clinical parameters.
- Body: Other plans (that also have the same structure) that may be executed in parallel, in sequence, or some combination thereof [101].

The Asbru language has been used to implement several guidelines in Endocrinology, Pulmonology, Gynecology, such as diabetes mellitus, jaundice in infants, and mechanical ventilation in premature babies [102]. The language has a formal syntax in BNF, and the earlier versions of the language also have a formal structural operational semantics [103].

During the development of Asbru, it was realized that CDSSs are limited by a lack of standardized EHR formats, and in many settings, a complete lack of EHR support. To address these issues, a new Asbru-based system that enables implementing CDSSs in settings with complete, partial, or no EHR, called Spock, was developed. Spock utilizes Hybrid-Asbru, a semi-formal variant of Asbru developed for the Digital Electronic Guideline Library (DeGeL) [102, 104]. Unlike Asbru, Hybrid-Asbru lacks a complete executable semantics and an interpreter with any correctness guarantees.

KIV-based Verification

In [105], a formal-methods approach to improve Asbru-based guidelines is presented. The authors identify that using formal methods for systematic verification of guidelines and protocols requires an appropriate representation language with a clear and well-defined semantics. Two real-world guidelines were chosen and analyzed by gradually transforming them into more formal representations. First, the protocols are transformed from plain text into Asbru plans. Then, the Asbru plans are transformed into KIV programs encoding said plans, for analysis using the KIV theorem prover [106]. A crucial step in using KIV is the definition of the formal semantics of the main parts of Asbru, which can also be utilized for verification of guidelines beyond those presented in [105]. As part of formalizing the guidelines, several properties relevant to clinical guidelines were identified, formulated over Asbru code, formalized in KIV, and established against the KIV code.

To evaluate the effectiveness of their approach, the authors chose the following guidelines for the management of:

- (a) Hyperbilirubinemia (Jaundice) in normal term infants by the American Academy of Pediatrics [107].
- (b) Diabetes mellitus type 2 by the Dutch Association of General Practitioners [108].

The formalization process required significant effort, where the following problems were observed:

- **Ambiguity:** Several terms presented interpretation related challenges, where details were left to the HCP. While missing details may be overlooked in paper versions, they needed accurate descriptions in the computer-interpretable counterparts.
- **Incompleteness:** Expected guidance or information was found to be missing in some cases. For instance, conditions such as “rapid increases” in certain patient parameters lacked accurate descriptions, and interpretations were left to the HCPs.
- **Inconsistencies:** Certain guidelines were found to lead to conflicting advice for the same patient data. Such scenarios were deemed to be very serious, as following the guideline could potentially result in suboptimal treatment.
- **Redundancies:** Instances of repeated recommendation were often discovered during the process of formalizing the guidelines in Asbru.

Asbru Semantics The semantics of Asbru are defined for two purposes: (a) to formally document the language specification, and, (b) to derive a calculus for verification. To this end, a structural operational semantics (SOS) is defined [109]. Recall from subsection 4.1.7 that Asbru programs consist of hierarchical plans that execute concurrently. The semantics consist of an execution engine that encodes concurrent execution of plans, and a data abstraction unit that abstract signals to the said plans.

The semantics of plan execution are described by expressing the transition system using statecharts [110]. Statecharts are directed graphs representing state machines [111]. Plan states represent statechart nodes, and edges transitions. The transitions can then be conveniently formalized as SOS rules. For instance, the rule:

$$\frac{\llbracket \text{P.State} = \text{considered} \rrbracket_{\sigma} \quad \text{da}(\Omega, \sigma) \rightarrow * \text{da}(\text{true}, \sigma')}{\text{psm}_P(\sigma) \rightarrow \text{psm}_P(\sigma' [\text{P.State} \leftarrow \text{ready}])} \quad (4.1)$$

expresses the semantics of a plan P (expressed as a statechart psm) to transition from state “consider” to state “ready”. The SOS rule expresses that if, under the environment σ , (i) the filter condition Ω evaluates to true by the data abstraction unit da , and, (ii) the plan’s current state is consider, then the plan P advances to state ready.

Once the semantics have been formally captured, Asbru plans are translated to programs into KIV. In KIV, program behavior is specified using standard dynamic logic program constructs, such as variable assignments $v := \tau$, conditional $\text{if}(\Psi) S_1 \text{ else } S_2$, iteration $\text{while}(\psi) \text{ do } S$, parallel execution $S_1 \parallel S_2$. The translation from Asbru to KIV is performed in a structure-preserving way, as most Asbru constructs can directly be mapped into a corresponding KIV one. For certain features of Asbru, additional encoding is required. The KIV encoding makes KIV’s theorem proving capabilities available for formal verification, where properties are stated as Interval Temporal Logic (ITL) formulas [112]. Notably, plan intentions can be encoded as logical formulas and verified against the plan’s KIV translation. However, the verification may require experience with theorem proving when dealing with properties of interest.

The outlined based approach demonstrates that formal methods can successfully be utilized for improving real BPGs. The analysis revealed several issues with both protocols, some of which were deemed very serious. Moreover, the corresponding Asbru plans were verified to satisfy their stated intentions, improving system safety. But the approach has the following limitations:

- While the formal semantics of Asbru intends to serve as a manual for implementing execution engines for Asbru, existing engines are not derived from it. This leaves it

possible for the execution engine to implement semantics that diverge from the SOS semantics used for verification, leaving it possible for the verification results to not hold on. As newer versions of the language are introduced, effort has to be expended into keeping the semantics and all tools in sync.

- Only a subset of the language has a direct mapping into KIV constructs. Manual effort and expertise is required for encoding constructs without direct mappings. Expressing intentions as logical formulas also requires expertise, and, may be error prone as the property isn't directly written over the Asbru program, but its translation in KIV.

4.1.8 Guideline Acquisition, Representation, and Execution (GLARE)

The GLARE system attempts to decouple the representation of knowledge in BPGs to their execution [113]. The GLARE representation language represents guidelines using actions, that may be either atomic or composite. Atomic actions involve elementary steps that cannot be broken down further, and can be one of the following:

- Work actions: Steps or procedures that must be performed at a particular point during guideline execution.
- Query actions: Obtaining information from the outside world, for instance, from doctors, or databases.
- Decision actions: Selecting alternate actions in the guideline based on guideline-specified criteria.
- Conclusion actions: Finalizing the outcome of executing the guideline.

While atomic actions provide the elementary primitives for modeling medical knowledge, specifying a guideline requires additional mechanisms for expressing relations over said actions. Relations in GLARE can either be structure-related or control-related. Structural relations enable building trees from actions, where nodes represent composite actions, and leaves atomic ones.

Guideline execution requires specifying temporal ordering on actions. This is supported through control-related relations, which can be one of:

- Sequence relations that enforce a strict ordering on actions, where, if any one of the actions in the sequence fails, then the entire sequence is considered to have failed.
- Concurrent relations that allow actions to run in any order, including in parallel.

- Decision relations that allow execution of alternative actions based on the conclusion of a decision action.
- Iteration relations that allow looping behavior, where an action may be repeated for a given duration, or, until a particular condition is met.

To enhance usability, actions in GLARE must further follow a particular structure. Work actions are further refined into clinical procedures and pharmacological prescriptions. Additionally, work actions may additionally have properties such as:

- Preconditions that specify the context under which the action may apply.
- Cycles that specify temporal behavior [113].

Guideline Acquisition and Consistency Checking

The GLARE framework provides a graphical tool for authoring guidelines that enables HCPs to specify GLARE-based guidelines. The interface has distinct “windows” that deal with different guideline aspects.

The structure window is used to represent relations between different actions through a tree. Each node represents an action, and its children represent sub-actions that must be performed as a part of executing the main action. A separate control window is utilized to enforce execution ordering on actions. For a given action, its sub-actions are represented as nodes in a control graph, where edges specify control relations between said nodes.

GLARE framework provides mechanisms for ensuring modeled guidelines do not have inconsistencies [113]. GLARE supports the following two types of checks:

Standardization Inconsistencies: Writing guidelines in GLARE occurs in one of two modes: safe and advanced. In safe mode, when a new term is introduced as a part of an action’s description, GLARE checks if the term has already been defined in its existing database and disallows the introduction if the term is not found. This ensures standard terminology and range of values is used throughout the system. In advanced mode, new terms can be introduced, and a warning is issued, under the assumption that the reason for ignoring existing checks is intentional.

Logical Inconsistencies: Checks for logical inconsistencies ensures that certain “logical design criteria” are followed. The following inconsistencies are disallowed:

- Cycles: Introduction of cyclic behavior through control arcs is disallowed, as repetitiveness can be specified through attributes.
- Alternatives without decisions: An action that models available choices must follow a decision action. This ensures that if a guideline provides multiple paths to reaching a goal, then there exists an explicit way of discriminating between them.
- Queries before decisions: When a decision action is reached, all data must have already been obtained. This ensures that all data necessary to make a decision is already available when control reaches a decision action.

Temporal Inconsistencies: These checks try to ensure semantic correctness from a temporal standpoint [113]. Recall that sequences of actions and sub-actions form graph structures. Minimum and maximum durations of actions and minimum and maximum delays between actions form logical constraints. These constraints are utilized in the control graph to infer new constraints, that can be checked for consistency using the LaTeR temporal manager [113, 114] .

Execution of GLARE Guidelines

Execution in GLARE corresponds to an instantiation of the guideline with a specific patient’s data. GLARE’s execution module is responsible for:

- 1) Interacting with the patient’s EHR to generate required SQL [115] queries to obtain relevant data.
- 2) Maintaining the set of applicable guidelines for each patient, and their status (such as active, suspended, etc.).
- 3) Maintaining the list of patients undergoing treatment.
- 4) Storing logs for each guideline execution to append patient history.

The execution module also supports different “modalities” to enable use in diverse settings, such as clinical practice, or training. For example, in actual practice, time delays between actions are modeled using wall-clock time, while in training, a simulated time might be used instead.

The GLARE framework has been used to implement real-world guidelines in different domains, such as bladder cancer, reflux esophagitis and heart failure [116]. The approach has

been instrumental in demonstrating viability of standardized executable guidelines. However, despite use in safety-critical settings, the execution module has an ad-hoc Java implementation with no correctness guarantees, and utilizes an informal and incomplete paper-based execution semantics [113].

4.1.9 Guideline PRocess cOnformance VERification Framework (GPROVE)

GPROVE is a set of tools that allow users to specify guidelines and reason about compliance of observed behavior. Specifically, the framework allows a-posteriori verification of workflow executions [117]. The GPROVE framework consists of:

- The Guideline prOcess Specification Language (GOSpeL) language for graphically representing guidelines.
- A translation to the Sciff language [118].
- Verification of compliance of an execution trace to the guideline.

Next, we briefly describe each component.

GOSpeL: GPROVE’s modeling language uses flowchart-derived concepts of blocks and relations between blocks to represent medical guidelines. Blocks belong to one of three families:

- Activity blocks that contain various guideline actions that HCPs must perform. Said actions are expressed through activities. Atomic activities represent actions that cannot be further decomposed and can be combined to form more complex composite activities.
- Gateway blocks that dictate divergence and convergence of control flow. These blocks enable fork and join constructs for modeling concurrent behavior.
- Start/end blocks that dictate terminal flowchart nodes.

Sciff Translation and Verification: The modeled guideline is mechanically translated to Sciff – a constraint logic programming framework. This translation consists of various constraints describing expectations of what should or should not happen. During runtime, a log of the observed events is generated from guideline execution. The log is then checked against the generated constraints, and a violation is raised if the log does not satisfy the said constraints [119].

The GPROVE framework is motivated by a different objective than other frameworks described in this chapter. Notably, GPROVE focuses on proving a-posteriori compliance to guidelines, and intends to serve as a complementary tool, instead of a standalone one [120]. But the approach has some limitations. First, the GPROVE modeling language does not have a formally defined semantics. Instead, the semantics are embedded in the Sciff translator and execution trace generator. Second, there exist no proofs of correctness of the Sciff translator, or methods of checking equivalence between the GOSpeL code and its Sciff translation, leading to a gap that can potentially result in the Sciff model to diverge from the underlying GOSpeL code.

4.1.10 HELEN

The HELEN-Project was implemented and tested at the Heidelberg University Medical Center’s Neonatology Department, and specifically focused on the following:

- Standardizing guidelines from diverse sources and origins.
- Support for convenient local adaptations to existing guidelines.
- Direct utilization in decision support for use in actual practice [121].

In HELEN, guidelines are organized into classes, that may have attributes and subclasses. The subclasses represent an “is a” relation, i.e., the subclass inherits attributes of all its superclasses. For knowledge acquisition, HELEN uses Protégé framework [122]. To improve comprehensibility, the HELEN framework includes a graphical interface that allows HCPs to view the guideline logic.

The framework also includes the Guideline Execution Engine (GEE) that facilitates BPGs to be used in CDSSs. The GEE traverses the guideline to:

- Communicate with HCPs about recommendations and scheduled actions.
- Evaluate guideline logic to determine execution flow.
- Perform administrative tasks such as logging performed actions and manage HCP information (such as roles, access, etc.) and patient data [121].

As HELEN was developed at the Heidelberg University Medical Center’s Department of Neonatology, it was used to implement diverse BPGs for use at the department. In particular, the following BPGs were modeled:

- Management of hyperbilirubinemia in healthy newborns, published by the American Academic of Pediatrics (AAP) [123].
- Management of apnea in pre-term newborns [124].

The BPG for hyperbilirubinemia management is largely a textual document with an accompanying non-executable flowchart, and is utilized mainly for illustration and training purposes. The BPG for apnea management, on the other hand, forms a complex CDSSs that utilizes nested algorithms to administer both advice and timing support [121]. This highlights HELEN’s versatility in adapting to needs of diverse BPGs.

While the framework can be used to implement and execute BPGs, it does not provide a way to systematically analyze them for semantic inconsistencies – limitations were recognized in [121]. The language does not have a formal syntax and semantics, and the main execution engine in Java has been implemented using an ad-hoc semantics. Thus, while HELEN moves the needle towards wider BPG adoption, as acknowledged by the developers themselves, more work is needed to address challenges for formal analysis and execution with correctness guarantees.

4.1.11 Prodigy

The Prodigy initiative was an effort by the UK’s National Health Service (NHS) to develop CDSSs for use by general practitioners (GPs). There have been several iterations of the Prodigy system to address limitations observed in practice. The Prodigy I and Prodigy II were closely coupled with the EHR systems and were not amenable to sharing. Prodigy II found wide use, demonstrating benefits of such systems to GPs, but was found to be inadequate to express important guidelines for chronic diseases. As a result, Prodigy III was developed to support modeling of such diseases [125].

In Prodigy III, guidelines are expressed through scenarios and action steps. Scenarios are easily recognizable patient states dictating if the guideline is applicable. Each scenario can have one or more outcome assessments with associated action steps that specify guideline-coded recommendations to manage the outcome. The action steps can:

- Start, stop or modify activities that occur over time, such as administering time-metered drug dosages).
- Specify instantaneous actions, such as giving an immediate drug dose, or recording a patient assessment.

In cases when a single action is insufficient to describe recommended treatment, a sub-guideline step that specifies additional treatment in a structured way can be utilized. This hierarchical representation methodology makes Prodigy guidelines resemble their paper-based counterparts.

Prodigy, like most CDSSs frameworks, depends on patient data to be available through EHRs or HCP assessments. In real clinical settings however, it is unreasonable to expect perfect data to be consistently available. It is often the case that (a) decision support is started after some treatment has already been performed, leading to an inconsistency between the perceived digital state and actual real-world state, and, (b) patient data is unreliable, obsolete, or simply unavailable. To address this, in Prodigy III:

- Scenarios can be broad and flexible and serve as intuitive access points for the clinicians in situations where perceived digital state may be inaccurate.
- Default values for patient parameters and other assessments are utilized if patient data is unavailable or inconsistent [125].

Prodigy III differs in its approach towards decision support, in comparison to alternative approaches such as GLIF (subsection 4.1.5), EON (subsection 4.1.4) or Asbru (subsection 4.1.7). Unlike said alternatives, that offer complex concurrency and synchronization primitives to capture treatment details, Prodigy employs a simpler action plans-based approach that its developers argue is sufficiently expressive for most guidelines. Other approaches emphasize HCP-system interactions and access to good quality and up-to-date clinical data to make complex assessments about patient state and dispense detailed advice. Prodigy, on the other hand, leaves the complex assessment of patient state to the HCP, and provides more abstract advice in the form of choices, expected outcomes and associated steps to reach said outcomes [125].

Prodigy, as an early CDSS framework, enabled greater adoption among HCPs. The focus of the Prodigy initiative was to standardize guidelines and improve uptake and adoption. Thus, efforts weren't expended into ensuring that the guidelines had a well-defined semantics, and tools that were based on it.

4.1.12 SAGE

The Standards-based Active Guidelines Environment (SAGE) was the result of a collaboration between groups at University of Nebraska Medical Center, GE Healthcare, Stanford University and Mayo Clinic. The project focused on creating relevant infrastructure for

shareable, standards-based guidelines amenable to use in decision support, across diverse EHR environments.

Integration with care workflow has been shown to be critical to CDSS effectiveness. SAGE, therefore, focuses on ensuring that the language and associated infrastructure support this goal. Instead of waiting for the user to control the workflow, SAGE uses available information to suggest applicable best-practices in the form of events. To this end, the language itself is expressive enough to specify workflow contexts that decide when guidelines become applicable [126].

In SAGE, guidelines are represented using recommendation sets, or, collections of recommendations applicable in one or more computable contexts. A context formalizes the clinical settings under which the recommendation is applicable. A recommendation further comprises of action specifications and decision logic that specify guideline-intended HCP behavior. The recommendation set itself is organized as a directed graph that captures relation between recommendations [126].

SAGE guidelines are interpreted by the SAGE execution engine. The engine interprets the context, action specifications and recommendation-relations and dispatches decision support to the EHR. The engine’s event listener module is responsible for receiving any state changes from the EHR [127].

SAGE’s design is motivated by: (i) using native EHR to dispense support, (ii) integrating with workflows as seamlessly as possible, and, (iii) enabling interoperability between different EHR systems. Its architecture and design choices enable interaction with heterogeneous sources of information, and, de-duplicating replication across EHR implementations [126]. This is reflected in the fact that SAGE has been used to implement guidelines at EHR systems at the University of Nebraska Medical Center and the Mayo Clinic [126]. But SAGE does not provide tools for validating guideline content, and, execution engine doesn’t have any correctness guarantees either. While the execution engine does provide some validation checks [127], SAGE does not have dedicated validation or verification tools.

4.2 GENERAL APPROACHES

4.2.1 T/Gen

The T/Gen is a lisp-based project that enables validation of CIGs through test-case synthesis. While primarily developed for GLIF (subsection 4.1.5), it can be made to work with any guideline modeling formalism.

The T/Gen tool takes as input a GLIF guideline encoded as a Lisp program, and an initial condition set that defines possible clinical scenarios for a program, and additional logical constraints that define when said clinical scenarios are feasible. T/Gen takes these initial conditions and constraints and generates a set of meaningful clinical scenarios under which the guideline can be tested [128].

For instance, consider the guideline in Figure 4.1 with initial conditions $\varphi_1, \varphi_2, \varphi_3$. A naive way to test the system would result in 8 distinct conditions $\varphi_i = Y/N$ for $i = 1, 2, 3$. T/Gen, however, would only generate 4 conditions $\{\varphi_1 = Y, \varphi_2 = Y\}$, $\{\varphi_1 = Y, \varphi_2 = N\}$, $\{\varphi_1 = Y, \varphi_3 = Y\}$, $\{\varphi_1 = Y, \varphi_3 = N\}$ that result in firing of $\alpha_1, \dots, \alpha_4$ under which the system should be tested to ensure complete coverage. Additionally, T/Gen further allows the user to specify constraints on the condition set both that can further reduce the number of test cases [128].

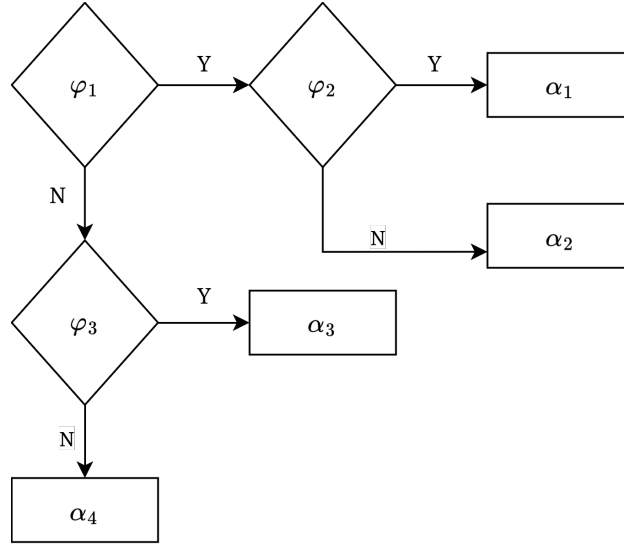


Figure 4.1: T/Gen Example

T/Gen’s white-box testing approach allows significant reductions in the number of test cases required for high test-coverage and has demonstrated effectiveness in real-world use cases [128]. But it still has many limitations. Notably:

- T/Gen can work with any CIG language and supports a simplified version of GLIF. But this requires the CIG to be re-expressed in T/Gen’s simplified language, which requires additional effort, especially in cases of languages that differ significantly from GLIF. Moreover, the translation effort itself may introduce bugs, or, may semantically be different from the original source.
- Uncovering additional constraints requires both efforts and expertise. As acknowledged

in [128], a compromise must be made between finding constraints and allowing some redundancy in tests. Moreover, introducing constraints can omit certain scenarios, such as ones dependent on ordering of actions, from being tested. Thus, using the tool effectively may require significant effort, depending on the complexity of the system under test.

- T/Gen utilizes an ad-hoc semantics of GLIF, which may differ from that used by GLIF execution engines. Moreover, changes to the GLIF language over time would also necessitate corresponding updates to T/Gen.

4.2.2 Verification of Hierarchical Plans

In [129], formal verification of hierarchical plans-based guidelines is discussed. Although the work specifically focuses on guidelines in Asbru, it is applicable to other languages that utilize a plan-based approach to representing BPGs. Recall from subsection 4.1.7 that Asbru guidelines consist of hierarchical plans, where a plan may contain other sub-plans, thus creating a tree of plans. Leaves of said trees are plans that cannot be decomposed any further into sub-plans.

A set of eight plan states describe the actual state of the plan during planning and execution. The following seven conditions define transitions between aforementioned states:

- 1) filter-preconditions that hold if an applicable plan cannot be achieved.
- 2) setup-preconditions that must hold for an applicable plan to start.
- 3) activate-condition that specifies whether plan starts automatically or manually.
- 4) suspend-conditions that specify when a plan must temporarily halt execution.
- 5) abort-conditions that specify when a plan must be stopped unsuccessfully.
- 6) complete-conditions that specify successful termination.
- 7) reactivate-conditions that enable suspended plans to resume.

The technique described in [129] utilizes static analysis to ensure plans are *meaningful*, or, verified to not contain well defined anomalies described in [130]. Given the hierarchical nature of plans, abnormalities are classified into: (a) level 1 that occur within a plan, (b) level 2 that occur within linked plans, (c) level 3 that occur at the scale of BPGs.

In [130], a general foundation for reasoning for reasoning about knowledge bases is provided. A knowledge-base $\mathcal{K} = \mathcal{R} \cup \mathcal{D}$, where:

1. $\mathcal{R}$ is a set rules $\{R_1, \dots, R_n\}$. Rule R_i is an expression of the form of the form $L_{i_1} \wedge L_{i_2} \wedge \dots \wedge L_{i_m} \rightarrow R_i$, where $L_{i_1}, \dots, L_{i_m}, R_i$ are First-Order Logic literals. antecedent (R_i) = $L_{i_1} \wedge L_{i_2} \dots L_{i_m}$ and consequent (R_i) = M_i .
2. The set of declarations $\mathcal{D} = \mathcal{G} \cup \mathcal{L} \cup \mathcal{C}$ representing the set of goal literals, input literals and constraints respectively.

An environment E is defined as a subset of input literals s.t. $E \rightarrow \neg(C\rho)$ for some $c \in \mathcal{C}$ and any substitution ρ . The set of inferable hypotheses $\mathcal{H}$ is defined as literals that can be inferred using rules, i.e., for $H \in \mathcal{H}$, $\exists R \in \mathcal{R}$ s.t. $H = \text{consequent}(R)\rho$ for any substitution ρ . $(\mathcal{R} \cup E) \vdash H$ denotes that H is inferable from rule base $\mathcal{R}$ if there is some environment E s.t. H is the logical consequence of supplying E to rule R .

In [129], a methodology to transform an Asbru plans-based guidelines in Asbru's model of plan states to a rule base MPS is described. The rules specify the sequence in which the conditions of a plan are considered. Note that since MPS encodes the semantics of Asbru, the translation has to be specified only once.

Next, each plan P_i in the guideline is also encoded as a rule base PC_i , and the guideline is defined as the rule base $\text{PC} = \text{PC}_1 \cup \text{PC}_2, \cup \dots \cup \text{PC}_n$. Additionally, the following sets are defined:

- PlanSet – the set of all plan
- PatientSet – the set of all patients
- ConditionSet = {filter, setup, $\dots$, complete} corresponding to plan conditions.
- FinalStateSet = {rejected, aborted, completed}.

The rule base MPS consists of rules encoding the generic model of plan states. Given parameters pl and pa for a plan and patient respectively, MPS has rules encoding states such as:

- Considered(pl) encodes the starting state of each plan.
- Considered(pl, pa) $\wedge$ filter(pl, pa) $\rightarrow$ possible(pl, pa) encodes the condition that a plan with filter conditions satisfied is possible for execution.

Next, Asbru plan i in the guideline need to be transformed into its corresponding rule base PC_i . For instance, a plan for treatment of non-insulation dependent gestation diabetes mellitus, in patients with normal blood glucose levels, called GDM-TYPE-II, contains rules of the form:

- $\text{Female}(pa) \wedge \text{Pregnant}(pa) \rightarrow \text{filter}(\text{GDM-TYPE-II}, pa)$ encodes that pregnant female patients should be considered for the plan.
- $\text{Activate}(\text{GDM-TYPE-II}, pa)$ encodes that the plan should be automatically activated.
- $\text{State}(\text{blood-glucose-high}, pa) \rightarrow \text{suspend}(\text{GDM-TYPE-II}, pa)$ encodes that if the plan be suspended in case of high blood sugar.
- $\text{State}(\text{blood-glucose-normal}, pa) \rightarrow \text{reactivated}(\text{GDM-TYPE-II}, pa)$ similarly encodes that the guideline be reactivated when blood sugar returns to normal.

Once the guideline has been encoded as a rule base using the above mentioned transformation, important properties can be checked against the rule base. For instance, conditions in a plan must be satisfiable to influence the plan. Formally, this is equivalent to checking all rules in the rule base are *fireable*. If a condition, such as $\text{Male}(pa) \wedge \text{Pregnant}(pa) \rightarrow \text{filter}(\text{plan}, pa)$ contains a conjunction that cannot be satisfied, then it does not contribute to the plan's execution, and, is not *meaningful*.

The work methodology describe above represents significant progress towards verification of meaningful properties on real guidelines. But the methodology also has limitations, such as:

- It only works for certain pre-defined anomalies.
- Expressing hierarchical plans in Asbru as knowledge-bases from [130] utilizes an ad-hoc semantics of Asbru.
- Any update to Asbru would necessitate corresponding updates to the verification tool.

4.2.3 Model-driven Development and Verification

In [131], the authors present a comprehensive approach to development and verification of guidelines, called the Model Driven Architecture (MDA). Instead of utilizing a domain specific language, they choose to model BPGs as UML Statecharts [132]. The choice is motivated by the fact that Statecharts have been used to build systems in diverse domains such as Air Traffic Control [133] and biology [134].

The development and verification methodology has several steps. First, software engineers and experts in medicine collaborate express the guideline using Statecharts. Next, patterns representing common errors in guidelines are utilized to write specifications for the guidelines. The the Statecharts code is automatically translated to PROMELA, the language of the

SPIN model checker [135] for verification. To turn the verified Statechart guidelines into a CDSS, the MDA-based approach includes tools that translate the Statecharts into executable Java code that is combined with additional infrastructure to obtain a functioning CDSS.

Property Specification

In [131], the authors deal with medical properties, or properties that are not implementation-related, dealing with aspects such as medical parameters, HCP actions, and overall guideline intentions. As specifying properties requires expertise and a mathematical background. The MDA approach includes several common patterns applicable to all guidelines that can be used as templates for writing actual properties. Thus, by using these patterns, the authors attempt to enable even non-experts to use the MDA verification methodology.

Property patterns in the MDA verification methodology are based on existing work on simplifying verification for concurrent and reactive systems through safety and liveness patterns that contain informal descriptions of the property and well-defined translations to widely used temporal logics such as computation tree logic (CTL) and linear temporal logic (LTL) [136, 137, 138, 139].

To ensure that that patterns presented under the MDA framework cover a wide spectrum of possible properties, the authors collected properties relevant to BPGs from literature and attempted to represent them using specification patterns proposed by Dwyer et al. in [136]. However, they realized that Dwyer patterns were insufficient to represent all properties of interest. To address this the authors: (a) utilized an expanded pattern representation schema proposed by Ryndina et al in [139], and, (b) proposed a new patterns to encompass all desired properties. This expanded representation schema enables the authors to represent useful and complex properties, at both local and global scopes, and dealing with both safety (something bad never occurs) and liveness (always something desirable eventually occurs).

In [140], MDA is used to implement and analyze the IRC guideline for management of infections due to intravenous catheters. The verification process revealed several inconsistencies, which were fixed. Moreover, the CDSS was shown to satisfy several important properties. Several important patterns were identified during the verification process, which are also applicable to other guidelines. For instance, it is important to ensure that if an empirical treatment has not been ordered, then it is not removed later. This is identified as an instance of the absence-after pattern, and can be represented by the LTL formula $\diamond \neg (\text{removeTreatment} == \text{Empirical} \rightarrow \text{beginTreatment} == \text{Empirical})$. Another pattern of interest is possible-existence, stating that, starting from a certain patient state leads to a desired guideline-based outcome. For instance, in case of the IRC guideline, the authors

verified that starting from clinical test results that were not indicative of IRC, it is never possible to reach a state where IRC is indicated. Thus, the verification process revealed both inconsistencies in the guidelines and ensured that other desired properties are satisfied [131].

The MDA approach has several strengths. Notably:

- The approach has been used to implement real-world systems and verify important properties against them, as demonstrated in [140].
- Utilizes existing work in modeling large concurrent systems as statecharts, instead of implementing a DSL from scratch, allowing existing tools for visualization to be used.
- Unlike other approaches, emphasis is given to ensuring that expertise in mathematics is not needed for verification, using property patterns for property specification, and automated translations of the BPG and property to PROMELA for verification using SPIN.

However, the approach also has some limitations. First, to execute the Statecharts-based BPG in a CDSS, a translation from the OMG Systems Modeling Language [132] to Java is utilized, and for verification via SPIN, a translation to PROMELA is utilized. Thus, it is possible for the translations to diverge from the underlying BPG in the OMG Systems Modeling Language. This can have serious consequences, as verification results may not be satisfied by the actual implementation. Thus, even though utilizing specification patterns may enable non-experts in verification to verify properties PROMELA model, expertise in formal reasoning may be required to ensure that the model corresponds to the original BPG and the corresponding Java code. Second, only a subset OMG Systems Modeling Language is supported by the tools in [131, 140], and the translation are tied to a particular version of the language. Supporting newer versions of the language and updates to the language would require corresponding updates to the entire toolchain. Finally, while several important properties can be established using model checking in SPIN, supporting other techniques such as symbolic execution or deductive verification would require similar translation to other languages, which would further require increase the chances of the many translations diverging from the underlying BPG.

4.3 DISCUSSION

In [chapter 3](#), we discussed several challenges that limit wider CDSSs uptake in practice. The approaches discussed in this chapter make significant headway into addressing said challenges. These challenges (Cs) were concisely described in [section 1.1](#) as:

- C1. Absence of systematic ways of *validating content* in a *reliable*, *accessible* and *updateable* manner.
- C2. Lack of *reliable*, *shareable* CDSS content that can be easily adopted across healthcare organizations and their (Information Technology) IT systems.
- C3. Technical difficulties of sharing due to *need for adaptation* to diverse Electronic Health Records (EHR) systems.
- C4. *Suboptimal* User Interfaces (UIs), implementation choices and workflows.

In [section 3.1](#), we outlined major *solution themes* behind addressing challenges mentioned above. We briefly recap those themes here:

Implementation-Specification Gap Textual BPGs are often written by medical experts, but the translation into an executable computer program is provided is performed by software developers and knowledge engineers who typically lack the necessary expertise to ensure medical knowledge is correctly encoded. The textual BPG is utilized as a functional specification to develop the CDSSs. This makes it possible for the specification to diverge from the implementation, leading to an implementation-specification gap.

DSLs for computer-interpretable guidelines aim to eliminate this gap by emphasizing HCP comprehensibility, thereby enabling the CIG to also serve as the guideline’s textual description, eliminating the implementation-specification gap, and are vital to developing trusted guidelines with validated content.

Formal Semantics In order to utilize formal methods to ensure CDSS correctness, the underlying language utilized to express guidelines must have a complete, well-defined formal semantics [40]. The semantics can then be utilized to implement tools for execution and analysis, to support a development of verified guidelines.

Formal Analysis Tools A comprehensive suite of formal analysis tools—model checkers, symbolic execution engines, and deductive verifiers—is essential for developing systems with desired safety properties. Additionally, it is crucial to keep these tools updated as the language evolves. As DSLs for computer interpretable guidelines adapt to lessons from implementing best practice guidelines (BPGs), mechanisms must be in place to update all relevant tools, and not just ones needed for immediate use, such as execution engines.

Holistic Safety Ensuring that computer interpretable guidelines meet desired correctness properties is crucial, but it is equally important to ensure that tools used to execute and

analyze said guidelines themselves are correct, as bugs can compromise patient safety. We use the term “holistic safety” to refer to approaches that have a complete formal semantics and a comprehensive suite of execution and formal analysis tools derived using them, each with correctness guarantees.

Next, we discuss where approaches discussed in this chapter fall under *solution themes*. In Table 4.1, we show whether each approach fully addressed (✓), partly addresses ◦ or doesn’t address (✗) issues related to said theme. For example, all DSLs specifically designed with comprehensibility to HCPs in mind eliminate the implementation-specification gap, as guidelines expressed using these DSLs can serve as both the HCP-readable textual representation and its computable translation as a CDSS’ knowledge-base.

The outlined languages can be categorized into either DSLs for CIGs, or, general ones for building concurrent systems. For instance, the Arden Syntax (subsection 4.1.1) was among the first languages specifically designed to be comprehensible to HCPs and is still widely used for expressing medical knowledge. But it’s designed for simpler, independent guidelines, and is unsuitable for complex workflows. Other approaches such DILLEMA (subsection 4.1.2) and GEODE-CM (subsection 4.1.3) enabled expressing more complex guidelines, and modular architectures that integrate with diverse systems. However, since these were among the earliest attempts at building modular CDSSs, functionality was prioritized over formal correctness.

	Implementation- Specification Gap	Complete Formal Semantics	Formal Analysis Tools	Holistic Safety
Arden Syntax	✓	✗	✗	✗
DILLEMA	✓	✗	✗	✗
GEODE-CM	✓	✗	✗	✗
EON	✓	✗	✗	✗
GLIF	✓	✗	✗	✗
Asbru	✓	◦	✓	✗
PROforma	✓	✓	✗	✗
GLARE	✓	◦	◦	◦
GPROVE	✓	✓	✗	✗
HELEN	✓	✗	✗	✗
PRODIGY	✓	✗	✗	✗
SAGE	✓	✗	✗	✗
Hierarchical Plans	✗	◦	✓	◦
MDA	✗	✗	✓	✗

Table 4.1: Comparison of Existing Approaches

EON (subsection 4.1.4) switched from small, modular and independent guidelines from Arden Syntax to more complex, tightly coupled state machines-based representation to enable modeling interdependent workflows. Similarly, GLIF (subsection 4.1.5) borrowed experiences from EON, GEODE-CM and Arden Syntax to facilitate modeling complex workflows. The design and implementation of GLIF was particularly influenced by the motivation to allow GLIF-based guidelines to be easily shared between medical establishments. Similarly, Prodigy (subsection 4.1.11) tackled CDSSs challenges encountered in practice, such as diversity in data sources, and their unreliability, while SAGE (subsection 4.1.12) attempted to integrate with existing digital workflows as seamlessly as possible. It's important to note that these approaches demonstrated that computerized decision support systems can both (a) manage complex guidelines with interdependent concurrent tasks, and, (b) be modular and portable to accommodate hospital-specific customizations, and integrate with their EHR systems. But system safety and correctness wasn't a challenge that these approaches were directly attempting to address. Thus, they lack a complete formal semantics and a suite of tools accompanying analysis tools.

The need for a complete formal semantics to enable development of tools for execution and analysis of guidelines has already been recognized by several approaches. For instance, the lack of ways to systematically analyze guidelines in HELEN (subsection 4.1.10), and an execution engine based on ad-hoc semantics has already been recognized as a limitation of the framework. PROforma (subsection 4.1.6), and Asbru (subsection 4.1.7) attempt to address this by having formal semantics, albeit incomplete semantics, that are intended to guide tool development. The Asbru language has an incomplete, but formal SOS paper-based semantics. But according to the authors of [39], it is insufficient for developing tools for the language. Despite its incompleteness, Asbru language's semantics has served as a calculus for deductive verification using KIV, as described section 4.1.7. Along similar lines, a formal but incomplete paper-based semantics of the PROforma language is presented in [39], with the aim of serving as a manual for developing PROforma tools. Both approaches take significant steps towards using recognizing formal semantics and using them in some ways to establish CDSS safety. But the support for formal analysis is very limited. Moreover, the tools aren't directly derived from the semantics, and need to be updated every time the language evolves, which can potentially lead to a divergence between the official semantics, and the ones utilized by said tools.

Approaches such as GLARE (subsection 4.1.8) and GPROVE (subsection 4.1.9), also have tools that allow complex guidelines to be both modeled and analyzed, but said tools are based on ad-hoc language semantics. For instance, GLARE provides mechanisms to check for inconsistencies in the guidelines such as unintended cyclic behavior, choices without

alternative options, and actions with unsatisfiable temporal constraints. Notably, GPROVE has capabilities to enable a-posteriori compliance checking, i.e., checking whether a log generated during execution indicates adherence (or lack thereof) to the guideline. To this end, GOSpeL (the language for expressing guidelines in GPROVE) is translated to Sciff – a constraint logic programming framework. But both GLARE and GPROVE lack formal semantics. GLARE’s analysis and GPROVE’s Sciff translator and execution trace generator are based on ad-hoc semantics of the underlying languages, making it possible for the analysis results to not hold on the actual programs.

Next, we consider approaches that rely on general solutions for building concurrent systems for building and analyzing CDSSs. The verification methodology defined in [subsection 4.2.2](#) utilizes foundational work in reasoning with knowledge bases to reason about Asbru guidelines. But it can work with any language that utilizes a plan-based approach to represent guidelines. Similarly, the MDA-based approach relies on guidelines expressed as statecharts in the OMG systems modeling language. The OMG statecharts are then translated to PROMELA for verification using the SPIN, and into executable Java code for use as a CDSS. For convenience, we refer to the representation that guidelines are directly encoded in as “HCP-friendly”, and the translated representation used for analysis as “analysis-friendly”. For instance, for the MDA-based approach described in [subsection 4.2.3](#), we refer to Statecharts as the “HCP-friendly”, and the translated PROMELA code as “analysis-friendly”. Thus, for both these approaches: (a) There exists a specification-implementation gap, where the “HCP-friendly” guideline serves as the specification, and the translated PROMELA or Java code serves as the implementation. (b) the translation from the “HCP-friendly” representation to an “executable” one, or a “analysis-friendly” is typically based on an ad-hoc semantics of the “HCP-friendly” language. Thus, even though the “analysis-friendly” representation may have well-defined formal semantics, the results of analysis may not hold on the “HCP-friendly” notation. Moreover, in case of the MDA-based approach, the “HCP-friendly” notation is translated to Java for execution. But as the Java and PROMELA translations are not proven to be equivalent, it is possible for the results of model checking through SPIN may not be applicable to the corresponding Java code.

CHAPTER 5: SEMANTICS-FIRST APPROACH TO CLINICAL DECISION SUPPORT

In [chapter 1](#), we explained that, despite advances in medicine, mortality and costs associated with preventable medical errors (PMEs) remain unacceptably high. In [chapter 2](#), we explained how systems that assist healthcare practitioners (HCPs) with situation-specific advice based on evidence-based best practice guidelines (BPGs), called clinical decision (CDSSs) can reduce both mortality and costs associated with PMEs. But, despite their potential, the uptake of such systems in practice is hindered by challenges that were introduced in [section 1.1](#), and discussed in depth in [chapter 3](#). In brief, the following challenges (Cs) were outlined:

- C1. Absence of systematic ways of *validating content* in a *reliable*, *accessible* and *updateable* manner.
- C2. Lack of *reliable*, *shareable* CDSS content that can be easily adopted across healthcare organizations and their (Information Technology) IT systems.
- C3. Technical difficulties of sharing due to *need for adaptation* to diverse Electronic Health Records (EHR) systems.
- C4. *Suboptimal* User Interfaces (UIs), implementation choices and workflows.

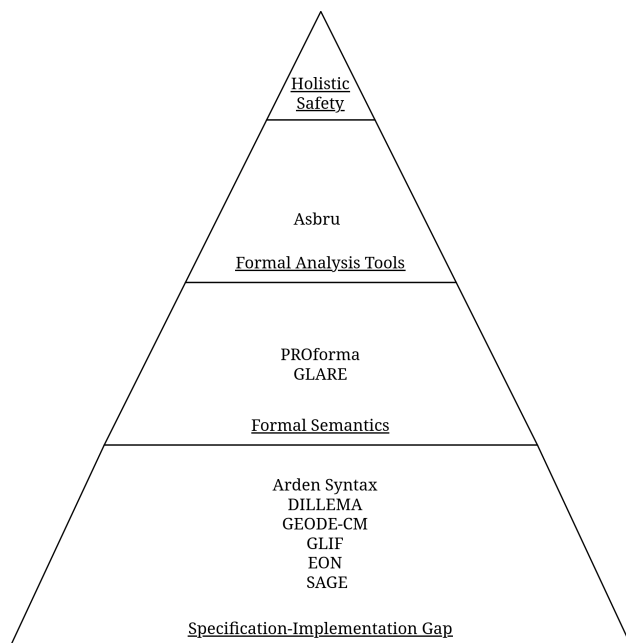


Figure 5.1: Existing DSLs for Computer Interpretable Guidelines

Over the years, significant progress has been made towards addressing these challenges. In [chapter 4](#), we discussed how existing approaches have attempted to address said challenges,

and their limitations. Specifically, in [section 4.3](#), we outlined major themes that these approaches adopt to tackle these challenges. This is further illustrated by the pyramid diagram in [Figure 5.1](#), with *solution themes* making up the pyramid’s various rungs. As is typical, approaches that appear in higher rungs also have characteristics of ones below them. For example, while guidelines expressed in the Arden Syntax eliminate the specification-implementation gap by being both HCP-comprehensible and interpretable, they cannot be formally analyzed due to lack of analysis tools in the ecosystem. Asbru-based guidelines on the other hand not only eliminate the specification-implementation gap but can also be formally analyzed using support for KIV-based verification in the Asbru ecosystem (see [section 4.1.7](#)).

As is evident in [Figure 5.1](#), no existing approach covers the “holistic safety” rung of the pyramid. Recall from [section 4.3](#) that we say an approach tackles “holistic safety” if, besides support for analyzing guidelines, the underlying language has a formal semantics, on which analysis and execution tools are based, leading to some correctness guarantees said tools themselves. In this work, we argue that such guarantees are necessary for trustworthy CDSSs. We attempt to address “holistic safety” systematically by developing a *semantics-first approach* for building clinical decision support systems. In this context, by semantics-first we mean that:

- The semantics of the programming language for defining said knowledge is formally defined, from which execution and analysis tools are derived in a correct by construction manner, leading to holistic safety.
- The semantics of medical knowledge are expressed accurately.

At the core of our approach is a novel domain-specific language for expressing medical knowledge called Medi $\mathbb{K}$ (pronounced Medi-Kay). By being comprehensible to domain experts in medicine, Medi $\mathbb{K}$ -based computer interpretable guidelines can serve both as a guideline’s non-executable HCP-comprehensible description, i.e., the specification, and its encoding in a computable medium, i.e., the implementation, thereby eliminating any specification-implementation gap.

The remainder of this chapter is structured as follows: [section 5.1](#) briefly describes the semantics-first philosophy. Next, [section 5.2](#) describes $\mathbb{K}$ – the language semantic framework that Medi $\mathbb{K}$ ’s are expressed in. Finally, [section 5.3](#) describes potential pitfalls of following the semantics-first philosophy.

5.1 SEMANTICS-FIRST APPROACH

The semantics-first approach prescribes a systematic way of developing programming languages. Instead of implementing tools for a language, such as interpreters, compilers and model checkers in ad-hoc manner, the approach states that the first step in developing said tools must be to formally define the language’s semantics. As show in [Figure 1.1](#), once defined, all tools for the language can then be automatically derived from the semantics. Moreover, since the tools utilize the semantics, they are, by definition, correct-by-construction.

While following the semantics-first philosophy might seem like an obvious choice in language design, its adoption in practice is far from ideal. Conventional practice in the programming language and formal methods community is still to develop analysis and execution tools for each programming language from scratch [\[141\]](#), as illustrated in [Figure 5.2](#) from [\[141\]](#). But, this approach has several disadvantages:

- Implementing tools that perform the same function for different languages incurs unnecessary development and maintenance cost. As shown in [Figure 5.2](#), if there are l languages, where each has t tools, then a total of $l \times t$ tools have to be developed and maintained over time.
- Tools are often based on informal descriptions of language semantics, leaving developers to extrapolate finer details of the language’s semantics, leading to inconsistencies. For instance, in [\[142\]](#), it was found that ECMAScript 5.1-compliant JavaScript engines in mainstream web browsers behaved differently from each other for certain complex JavaScript programs.
- As newer versions of a language are introduced, each tool for the language must be updated to ensure support for the latest version. This again results in duplicated work.

5.1.1 Why build CDSSs using Semantics-First?

In [section 5.1](#), we described benefits of using the semantics-first approach for developing regular programming language. But these differences become starker when semantics-first is compared against the conventional approach shown in [Figure 5.2](#) in context of domain-specific language for expressing medical guidelines. Specifically, as such a language will be utilized in safety-critical settings, it is vital that the language:

- Has an *unambiguous, formal* semantics that can serve as a reference for developing tool support for it. This is necessary to ensure that tools are free of behavioral inconsistencies due to ambiguities in the semantics.

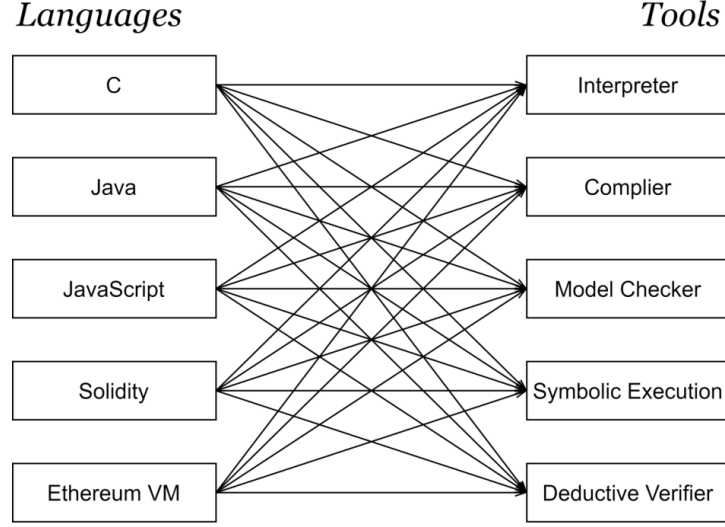


Figure 5.2: State-of-Art in Programming Language Design

- Is supported by a rich ecosystem of formal analysis tools that can be used to analyze programs. Implementing such tools from scratch would require significant effort.
- Can evolve quick to incorporate (i) lessons from expressing medical guidelines in it, and, (ii) HCP feedback, specifically regarding comprehensibility. This can be challenging when using the approach shown in [Figure 5.2](#), as every change to the language’s semantics would require corresponding changes to all relevant tools, and additional effort to maintain different versions, making the development process extremely tedious.

5.2 THE $\mathbb{K}$ FRAMEWORK

In this section, we introduce $\mathbb{K}$: a rewrite-based executable semantics in which programming languages can be defined through configurations and rules [41]. Once the semantics of a programming language has been defined, $\mathbb{K}$ automatically generates all tools depicted in [Figure 1.1](#), such as an interpreter, compiler, model-checker and deductive verifier for the language. $\mathbb{K}$ has been successfully utilized to formalize semantics of large real-world languages, such as C [42], Java [43] and Javascript [142], and analyze non-trivial programs [45, 46].

The remainder of this section introduces relevant features of $\mathbb{K}$ by describing the $\mathbb{K}$ semantics of an example language called Imp. This introduction to $\mathbb{K}$ provides necessary background for upcoming chapters that discuss the Medi $\mathbb{K}$ DSL using $\mathbb{K}$ notation and concepts.

5.2.1 Defining Languages in $\mathbb{K}$

A typical $\mathbb{K}$ definition of a language consists of the following components:

- **Syntax:** Defined in BNF-like notation and utilized by $\mathbb{K}$ to generate a parser for the language.
- **Configuration:** Organizes the program execution state into units called *cells* that may be nested.
- **Rules:** Operate over configuration segments and define program evolution via rewrites.

We now explain these components using an example language called *Imp*. *Imp* is a simple imperative programming language inspired by C and Java that supports arithmetic and boolean expressions and statements such as variable declaration and assignment, branching (*if*) and looping (*while*).

[Listing 5.1](#) and [Listing 5.2](#), define syntax and semantics of *Imp* respectively. $\mathbb{K}$ code must be placed inside an organizational unit called a **module** that has a name and can import other **module**(s). For example, **module IMP-SYNTAX ... endmodule** between [Line 1](#) and [Line 23](#) of [Listing 5.1](#) defines a $\mathbb{K}$ **module** named **IMP-SYNTAX** containing *Imp*'s grammar. $\mathbb{K}$ provides builtin support for domains such as natural numbers, integers, booleans and program identifiers under a **module** named **DOMAINS**. [Line 2](#) **imports** the syntax definition of $\mathbb{K}$'s **DOMAINS** module to enable parsing integers and booleans in *Imp* programs.

Syntax in $\mathbb{K}$ is defined using BNF-like notation; terminals are enclosed in quotes, and non-terminals begin with an uppercase. For example, consider the declaration of *Imp* arithmetic expressions between [Line 3](#) and [Line 7](#). On [Line 3](#) $\mathbb{K}$'s builtin support for integers (**Int**) and program identifiers (**Id**) is used to support arithmetic expressions over program variables.

Beyond simply defining the syntax, $\mathbb{K}$ also allows assigning semantics to constructs through the use of *attributes* specified as a comma-separated list inside square brackets (`[...]`) placed immediately after the BNF production. For example, on [Line 5](#) and [Line 7](#), the attribute **left** is used to declare the corresponding operator as left associative. The **strict** attribute is used to assign *evaluation strategies*. Note its use on [Line 5](#) and [Line 7](#), signifying that both operand sub-expressions must be completely evaluated before the corresponding operator (`/`, `+`) is evaluated. Note that the order in which the arguments are evaluated is non-deterministic, and, all operands will be evaluated before the operator is evaluated. To enforce a particular order, or, to choose a subset of the operands, a list of operand positions specifying the desired order can be supplied to **strict**. For example, **strict(2, 1)** would make $\mathbb{K}$ evaluate the second argument before the first. This is particularly useful for defining

```

1 module IMP-SYNTAX
2   imports DOMAINS-SYNTAX
3   syntax AExp  ::= Int | Id
4                   | "-" Int
5                   | AExp "/" AExp           [left, strict]
6                   | "(" AExp ")"           [bracket]
7                   > AExp "+" AExp          [left, strict]
8   syntax BExp  ::= Bool
9                   | AExp "<=" AExp           [seqstrict]
10                  | "!" BExp                [strict]
11                  | "(" BExp ")"            [bracket]
12                  > BExp "&&" BExp            [left, strict(1)]
13   syntax Block ::= "{" "}"
14                  | "{" Stmt "}"
15   syntax Stmt  ::= Block
16                  | Id "=" AExp ";"         [strict(2)]
17                  | "if" "(" BExp ")"
18                    Block "else" Block       [strict(1)]
19                  | "while" "(" BExp ")" Block
20                  > Stmt Stmt               [left]
21   syntax Pgm   ::= "int" Ids ";" Stmt
22   syntax Ids   ::= List{Id, ",", ""}
23 endmodule

```

Listing 5.1: Imp Syntax in $\mathbb{K}$

constructs like short-circuit ($\&\&$) boolean expression on [Line 12](#), where `strict(1)` ensures that the left operand is evaluated first, allowing the right to only be evaluated if the left evaluates to `true`. Similarly, for an assignment statement on [Line 16](#) the `strict(2)` indicates that the second argument, i.e., the expression to the right of the `=` sign must be evaluated before the identifier on the left of the `=` is updated.

$\mathbb{K}$ also allows operator precedence to be specified as a part of the syntax definition. On [Line 7](#), the `>` signifies that all preceding productions have higher precedence, i.e., bind tighter, than the production for addition (`+`). Similarly, [Line 20](#) defines statement composition. Thus, preceding `Stmt` productions ([Lines 15-19](#)), that define blocks (`{. . .}`) and standalone statements such as variable assignment and while loop have higher precedence, indicated by `>` on [Line 20](#).

On [Line 21](#), an Imp program is defined to start with a list of program variable declarations (`"int" Ids ";"`) followed by other statements. Note the definition of a list of identifiers (`Ids`) on [Line 22](#). In $\mathbb{K}$, `List{...}` is used to define syntactic-lists, where the first argument is the production of list elements, and the second the list de-limiter. Thus, `List{Id, ",", ""}` defines

```

1 module IMP
2   imports IMP-SYNTAX
3   imports DOMAINS
4   syntax KResult ::= Int | Bool
5
6   configuration <T>
7       <k> $PGM:Pgm </k>
8       <state> .Map </state>
9   </T>
10
11 // AExp
12 rule <k> X:Id => I ...</k> <state>... X |-> I ...</state>
13 rule I1 / I2 => I1 /Int I2   requires I2 /=Int 0
14 rule I1 + I2 => I1 +Int I2
15 rule - I1 => 0 -Int I1
16 // BExp
17 rule I1 <= I2 => I1 <=Int I2
18 rule ! T => notBool T
19 rule true && B => B
20 rule false && _ => false
21 // Block
22 rule {} => . [structural]
23 rule {S} => S [structural]
24 // Stmt
25 rule <k> X = I:Int; => . ...</k> <state>... X |-> (_ => I) ...</state>
26 rule S1:Stmt S2:Stmt => S1 ~> S2 [structural]
27 rule if (true) S else _ => S
28 rule if (false) _ else S => S
29 rule while (B) S => if (B) {S while (B) S} else {} [structural]
30 // Pgm
31 rule <k> int (X,Xs => Xs);_ </k>
32   <state> Rho:Map (.Map => X|->0) </state>
33   requires notBool (X in keys(Rho))
34 rule int .Ids; S => S [structural]
35
36 endmodule

```

Listing 5.2: $\mathbb{K}$ Semantics of Imp

a comma-separated list of program identifiers.

Once the *syntax* has been defined, $\mathbb{K}$ can utilize it to generate a parser, which can be used to generate abstract syntax trees (ASTs) for programs. The $\mathbb{K}$ grammar declares an order-sorted algebra over AST terms, where the non-terminals are the sorts. The program's AST forms a part of the larger program state, over which semantics are defined through rules. The $\mathbb{K}$ semantics of any language has two components:

- A *Configuration* that organizes the state into units called *cells*, that may be nested.
- *Rules* that operate over *configuration*-segments to describe evolution of state during

execution.

For Imp, [Listing 5.2](#) has these components under `module IMP`. Note that the syntax and semantics exist in separate modules, where the semantics module `imports` the syntax module. This is by convention, and has the following advantages:

- a) Rules can operate directly over the language’s syntax, making them easier to specify and comprehend.
- b) With the syntax and semantics residing in separate modules, $\mathbb{K}$ can be instructed to use only the syntax module for parsing programs. This allows users to define additional syntactic constructs in the semantics module for use in rules. If a combined syntax and semantics module is used instead, any constructs defined solely for use in rules would unintentionally become part of the language’s syntax and be accepted by the parser.

Configurations

The configuration is defined using the keyword `configuration`, followed by an unordered list of *cells*, which may be nested and are specified using an XML-like notation. For instance, `<foo> <bar> ... </bar> </foo>` corresponds to $\mathbb{K}$ *cells* named `foo` and `bar` respectively, where `bar` is nested under `foo`. Imp’s configuration is defined between [Line 6](#) and [Line 9](#), and consists of a top-level cell `<T>` containing cells `<k>` and `<state>` on [Line 7](#) and [Line 9](#) respectively. In $\mathbb{K}$, the `<k>` cell typically contains the AST of the executing program, indicated by its contents `$PGM:Pgm`. During execution, $\mathbb{K}$ will attempt to parse the program being executed using the production `Pgm` defined on [Line 21](#) of [Listing 5.1](#). If parsing is successful, $\mathbb{K}$ will replace `$PGM` with the parsed AST. The `<state>` cell, as the name suggests, will hold a map of program identifiers and values they acquire during execution. Said map is initially empty, denoted by `.Map`. The dot (`.`) in $\mathbb{K}$ denotes *nothing* or *empty*. Thus, `.Map` denotes an empty map.

Rules

$\mathbb{K}$ *rules* operate over configuration segments and define evolution of program state. When specifying a rule $\mathbb{K}$, only relevant parts of the configuration need to be mentioned— $\mathbb{K}$ completes the rest of the configuration through a mechanism called *configuration abstraction*. This allows rules to be concise, enhancing readability and making them easier to write. For example, consider the rule for addition on [Line 14](#). Simply put, the rule specifies that if

the term at the top of the `<k>` cell is an addition expression, then it should be *rewritten* to addition in the integer domain `+Int`. But even though the rule is intended to simplify the top of the `<k>` cell, no cells are explicitly mentioned. This is because:

- If no cell is mentioned, $\mathbb{K}$ assumes that rule applies at the top of the $\mathbb{K}$ cell.
- All other parts of the configuration are assumed to remain unchanged.

Now we describe rules in greater depth. A rule begins with the keyword `rule` and is a statement of the form $\varphi \Rightarrow \psi$, where φ, ψ are *patterns* over configuration terms and $\mathbb{K}$ variables. We say φ is the LHS and ψ is the RHS of the rule. We define a *substitution* θ to be a map from $\mathbb{K}$ -variables to terms. Given pattern φ and *substitution* θ , we say $\varphi\theta$ is the pattern obtained by replacing every variable v in φ with $\theta(v)$. We say pattern φ matches term τ iff there exists a substitution θ s.t. $\tau = \varphi\theta$. During execution, if the current configuration term τ *matches* the LHS φ of rule $\varphi \Rightarrow \psi$ with substitution θ , then C is rewritten to $\psi\theta$. For example, the addition rule on [Line 14](#). $\mathbb{K}$ -variables always begin with an uppercase, and may be suffixed with `:S`, where `S` is the variable's sort. In the case of the addition rule, $\mathbb{K}$ can *infer* that the sort of `I1` and `I2` is `Int`, as operands of the builtin operator `+Int` can only be integers (`+Int` is addition in the domain of integers). Thus, the LHS `I1 + I2` *match* term `2 + 3` (of sort `AExp`) with substitution $\theta = (I1 \mapsto 2, I2 \mapsto 3)$ and rewrite it `2 +Int 3`, i.e. `5`, as `+Int` is $\mathbb{K}$ -builtin for integer addition.

We now discuss execution of an entire program to introduce $\mathbb{K}$ nuances relevant to later chapters. Imp's grammar dictates that an Imp program (denoted by production `Pgm` on [Line 21](#) of [Listing 5.1](#)) must declare all variables at the start of execution. Consider the simple program `int x; x = 2 + 3; .` At the start of execution, $\mathbb{K}$ replaces the `$PGM` in the configuration declaration with the program's AST. Thus, we get the following initial configuration shown in [Listing 5.3](#), where `.Ids` is the identity element for the user-defined list of program identifiers, and `.Map` is the map identity from the initial configuration declaration on [Line 8](#).

```
<T>
  <k> int x, .Ids; x = 2 + 3; </k>
  <state> .Map </state>
</T>
```

Listing 5.3: Initial Execution Configuration

Now the rule for variable declaration on [Lines 31-32](#) can match with substitution $(X \mapsto x, Xs \mapsto .Ids, Rho \mapsto .Map)$ to rewrite the configuration from [Listing 5.3](#) to the configuration shown in [Listing 5.4](#).

```

<T>
  <k> int .Ids; x = 2 + 3; </k>
  <state> x |-> 0 </state>
</T>

```

Listing 5.4: Configuration with Variables Declared

Note the following conveniences that $\mathbb{K}$ offers to make writing rules easier:

- i) The use of parenthesis limits scope of the rewrite to the list of program identifiers. Localized rewriting reduces redundancy as only relevant parts of a term must be mentioned on the RHS of the rule.
- ii) The underscore `_` following `int (X, Xs => Xs);` on Lines 31-32 is an *anonymous* variable that matches the remainder to the program. Since it's not anywhere on the RHS, there is no need to provide an explicit variable name.

Also that the rule has a side condition, expressed through the keyword `requires` on Line 33 of Listing 5.2. This forces the rule to only apply if the program identifier has not already been declared. For instance, a program that starts with `int x, x;` will not execute to completion, as the side condition prohibits already declared variables from being re-declared. Next, the rule on Line 34 of Listing 5.2 will eliminate `int .Ids;`, leaving the configuration shown in Listing 5.5 after the rewrite.

```

<T>
  <k> x = 2 + 3; </k>
  <state> x |-> 0 </state>
</T>

```

Listing 5.5: Configuration Post `.Ids` Elimination

Also note that the rule on Line 34 does not explicitly mention the cell on which it is intended to operate. In $\mathbb{K}$, a rule that doesn't mention any cells is implicitly assumed to apply on top of the `<k>` cell. As the `<k>` cell typically contains the program being executed, not having to explicitly mention the cell allows the rule to only show how constructs in the language affect execution. For instance, the rules on Line 27 and Line 28 depict that for an `if(condition) {...}` statement, if the `condition` is true, the `if` block is taken, else whatever comes after is taken.

Next, consider the assignment rule on Line 25 of Listing 5.2. The rule uses two variables: `x` and `I` with sorts `Id` and `Int` respectively. Since the `<k>` cell in our running example

has $x = 2 + 3$; , we expect the assignment rule to apply at some point and update the `<state>` accordingly. But the rule cannot apply as the RHS of the assignment, i.e., the expression $2 + 3$, is not of sort `Int`, which needs to be evaluated first. This in $\mathbb{K}$ is enabled by evaluation strategies. Recall that the attribute assignment statement has the attribute `strict(2)` (Line 16 of Listing 5.1), denoting that the second argument of the construct must be evaluated first. Thus, $\mathbb{K}$ will *heat*, or pull-out the second argument for evaluation. This results in the configuration show in Listing 5.6.

```
<T>
  <k> 2 + 3 ~> x = []; </k>
  <state> x |-> 0 </state>
</T>
```

Listing 5.6: Post *Heating* Configuration

In $\mathbb{K}$, $\sim>$ means *followed-by*, i.e., the evaluation of $2 + 3$ must occur *before* the evaluation of $x = []$; . `[]` denotes a *hole* left in place of the argument that was *heated*. `<k> 2 + 3 ~> x = []; </k>` is re-written to `<k> 5 ~> x = []; </k>` by an application of the arithmetic addition rule on Line 14 of Listing 5.2. On Line 4, we specify any term of sort `Int` to be a `KResult`. This signifies that the term can no longer be evaluated, and can be *cooled* or plugged-back into its corresponding hole, resulting in the configuration shown in Listing 5.7.

```
<T>
  <k> x = 5; </k>
  <state> x |-> 0 </state>
</T>
```

Listing 5.7: Post *Cooling* Configuration

This enables the LHS of the assignment rule can *match* with substitution $\theta = (X \mapsto x, I \mapsto 5)$, to update the value of x in the `<state>` cell to 5, resulting in the final configuration shown in Listing 5.8.

```
<T>
  <k> . </k>
  <state> x |-> 5 </state>
</T>
```

Listing 5.8: Final Configuration

Note how the use of such evaluation strategies contributes to making the definition concise, as they eliminate the need for manually specifying related *heating* and *cooling* rules, enabling the definition to only contain relevant rules assigning semantics to various constructs. Also note the use of $\dots$ in the assignment rule. In $\mathbb{K}$, the $\dots$ is used to refer to parts of the term not relevant to the rule. For instance, in the `<state>` cell, it signifies that there may be other identifier-value pairs in the map, but we are only concerned with the binding that has the program identifier on the LHS of the rule. Similarly, the $\dots$ after the assignment statement in the `<k>` cell signifies that there may be other statements to evaluate, but they are irrelevant to the assignment rule. To better illustrate the significance of the $\dots$ in $\mathbb{K}$, we slightly modify the imp program to have two simple statements after the variable declaration instead of one. Assume the program has the form `int x; x = 5; x = x + 2.` variable declaration, we would have the configuration shown in [Listing 5.9](#).

```
<T>
  <k> x = 5; x = x + 2; </k>
  <state> x |-> 0 </state>
</T>
```

Listing 5.9: Multi-Statement Imp Program

Now, we would expect the assignment rule from [Line 25](#) of [Listing 5.2](#) to apply, since the RHS of the assignment `x = 5` doesn't need any further evaluation via heating and cooling. But, the rule cannot apply, as the top of the `<k>` cell is a composite statement as dictated by the production on [Line 20](#) of [Listing 5.1](#), and the assignment rule can only match a standalone statement. Thus, the rule on [Line 26](#) will apply with substitution $\theta = (S \mapsto x = 2 + 3; , Ss \mapsto x = x + 2;)$ to give us the configuration shown in [Listing 5.10](#).

```
<T>
  <k> x = 5; ~> x = x + 2; </k>
  <state> x |-> 0 </state>
</T>
```

Listing 5.10: Configuration Containing `~>` (*followed-by*)

As mentioned before, since `~>` means *followed-by*, $\mathbb{K}$ will evaluate `x = 2 + 3`; *followed-by* `x = x + 2`; . Now, the assignment rule on [Line 25](#) can apply, where $\dots$ after `x = x + 2`; in the `<k>` abstracts away the remaining computation, which in our example is `~> x = x + 2`; . The rule can then rewrite the assignment `x = 5`; to nothing (denoted by `.`), which leads to the configuration shown in [Listing 5.11](#).

```

<T>
  <k> x = x + 2; </k>
  <state> x |-> 5 </state>
</T>

```

Listing 5.11: Configuration Post $\sim$ (*followed-by*) Evaluation

We had earlier mentioned that, in $\mathbb{K}$, any rule that doesn't mention a cell by default, is explicitly assumed to operate on top of the `<k>` cell. We can now explain this in greater detail. Essentially, $\mathbb{K}$ considers any rule $\varphi \Rightarrow \psi$ where φ, ψ don't mention any cells equivalent `<k> ( $\varphi \Rightarrow \psi$ ) ... </k>`. This both simplifies the process of specifying semantics in $\mathbb{K}$, and, greatly improves readability of $\mathbb{K}$ definition. Moreover, the ability to use strictness attributes further reduces the need to write rules to specify evaluation strategies, enabling the semantics to serve as language specifications that are both concise and mathematically precise. This is demonstrated well by the example `Imp` definition as:

- Only the rules for variable lookup ([Line 12](#)), variable declaration ([Lines 31-32](#)) and variable assignment ([Line 25](#)) explicitly mention cells. All other rules use terms *only involving* constructs whose semantics they define. For instance, the semantics of short circuit boolean and (`&&`) is concisely captured by two rules depicting: (a) the need to evaluate the second argument only if the first is `true`, and, (b) discarding the second argument when the first is `false` (see [Line 19](#) and [Line 20](#) of [Listing 5.2](#)). The `strict(1)` attribute on [Line 12](#) of [Listing 5.1](#) eliminates the need to write additional rules to ensure the first argument is already evaluated when the rules for `&&` apply.
- The entire executable semantics is fewer than 100 lines, indicating $\mathbb{K}$'s suitability for semantics-first language design.

Note the difference between program identifiers and $\mathbb{K}$ variables. While program variables are simply terms belonging to sort `Id`, $\mathbb{K}$ variables have logical meaning. If multiple rules can match the configuration term, then one rule is non-deterministically chosen. Execution is a sequence of rule applications that continues until no rule can match the configuration. Since in the running example the $\mathbb{K}$ the `<k>`-cell becomes empty after all statement are evaluated, indicating successful completion.

5.2.2 Matching Logic: Theoretical Foundations of $\mathbb{K}$

In [section 4.3](#), we discussed how the need for a formal semantics for DSLs for expressing computer interpretable guideline has already been recognized. One of the motivations

mentioned is the ability to utilize semantics for formal methods-based reasoning and analysis. In this section, we briefly introduce the theoretical foundations of $\mathbb{K}$ that provide a sound framework for formal reasoning.

The semantics of a $\mathbb{K}$ definition are given in Matching Logic (ML), which we briefly describe in this section. We only present concepts that are relevant to this proposal, and direct the reader to [143, 144] for a more thorough introduction to ML.

Traditionally, program verification approaches have relied on an axiomatic semantics for deduction, i.e., a proof system with *language-specific* proof rules, such as Hoare logic [145]. In $\mathbb{K}$ however, the semantics are defined in an operational style, as rewrite rules in Matching Logic, and reasoning can be performed through the language-independent proof system of ML [146]. ML as a logic was developed specifically to reason about programs compactly and modularly, and is expressive to enable other logics, such as First-Order Logic with least fixed-points, modal logic and temporal logics to be captured as ML theories [143, 144, 147, 148]. Next we present a brief introduction to Matching Logic. For a more thorough reading, we refer the reader to [147].

Syntax

The *syntax* of matching logic is parametric in two sets of variables EV and SV , called *element variables* and *set variables*. We use a lowercase letter x (uppercase letter X) to denote $x \in EV$ ($X \in SV$). A (*matching logic*) *signature* Σ is the tuple (EV, SV, Σ) where Σ is a set of symbols denoted $\sigma_1, \sigma_2, \dots$. The set of Σ -patterns is inductively defined by the grammar:

$$\varphi ::= x \mid X \mid \sigma \mid \varphi_1 \varphi_2 \mid \perp \mid \varphi_1 \rightarrow \varphi_2 \mid \exists x. \varphi \mid \mu X. \varphi \quad (5.1)$$

where in $\mu X. \varphi$ it's required that φ has no negative occurrences of X . $\varphi_1 \varphi_2$ is an *application* pattern representing application of φ_1 to φ_2 . All other constructs can be derived as follows:

$$\begin{aligned} \neg \varphi &\equiv \varphi \rightarrow \perp \\ \top &\equiv \neg \varphi \\ \varphi_1 \wedge \varphi_2 &\equiv \neg (\varphi_1 \vee \varphi_2) \\ \varphi_1 \vee \varphi_2 &\equiv \neg \varphi_1 \rightarrow \varphi_2 \\ \forall x. \varphi &\equiv \neg \exists x. \neg \varphi \\ \nu X. \varphi &\equiv \neg \mu X. \neg \varphi [\neg X / X] \end{aligned} \quad (5.2)$$

Note that $\varphi[\psi/x]$ (respectively $\varphi[\psi/X]$) is the result of substituting ψ for x (respectively X), where bound variables in ψ are renamed to avoid capture.

Semantics

A Σ -model $\mathbb{M}$ is the tuple $(M, \dots, \{M_\sigma\}_{\sigma \in \Sigma})$, where, M is a non-empty *carrier set*, $\dots : M \times M \rightarrow \mathcal{P}(M)$ is a binary function called *application*, and $M_\sigma \subseteq M$ is an *interpretation* of symbol $\sigma \in \Sigma$ in M . For brevity, we abuse $\dots : \mathcal{P}(M) \times \mathcal{P}(M) \rightarrow \mathcal{P}(M)$ to also mean the pointwise extension of the *application* function, i.e., $A.B = \bigcup_{a \in A, b \in B} a.b$ for $A, B \in \mathcal{P}(M)$.

Given a model $\mathbb{M} = (M, \dots, \{M_\sigma\}_{\sigma \in \Sigma})$, an M -valuation of variables $\rho : (SV \cup EV) \rightarrow (M \cup \mathcal{P}(M))$, define interpretation $|\varphi|_\rho$ of a pattern φ in M as:

- $|x|_\rho = \{\rho(x)\}$ for $x \in EV$.
- $|X|_\rho = \rho(x)$ for $X \in SV$.
- $|\sigma|_\rho = M_\sigma$ for $\sigma \in \Sigma$.
- $|\varphi_1 \varphi_2|_\rho = |\varphi|_{\rho_1} \cdot |\varphi|_{\rho_2}$
- $|\perp|_\rho = \emptyset$
- $|\varphi_1 \rightarrow \varphi_2|_\rho = M \setminus (|\varphi_1|_\rho \setminus |\varphi_2|_\rho)$.
- $|\exists x. \varphi|_\rho = \bigcup_{a \in M} |\varphi|_{\rho[a/x]}$, where $\rho[a/x]$ and ρ agree on all variables except x , and $\rho[a/x](x) = a$.
- $|\mu X. \varphi|_\rho = \mu \mathcal{F}_{X, \varphi}^\rho$ where $\mathcal{F}_{X, \varphi}^\rho = |\varphi|_{\rho[A/X]}$ for every $A \subseteq M$, where $\rho[A/X]$ and ρ agree on all variables except X and $\rho[A/X](X) = A$. For proof of existence of a unique least-fixed point for $\mathcal{F}_{X, \varphi}^\rho$, referred to as $\mu \mathcal{F}_{X, \varphi}^\rho$, see [144].

A pattern φ is said to hold in model $\mathbb{M}$, denoted $\mathbb{M} \models \varphi$ iff $|\varphi|_\rho = M$ for any $\mathbb{M}$ evaluation ρ . Similarly, set of patterns Γ holds in $\mathbb{M}$, i.e. $\mathbb{M} \models \Gamma$ iff $\mathbb{M} \models \varphi$ for all $\varphi \in \Gamma$. Finally, $\Gamma \models \varphi$ holds iff for any model $\mathbb{M}$, if $\mathbb{M} \models \Gamma$ then $\mathbb{M} \models \varphi$. In [149], the authors present a Hilbert-style proof system for ML that is amenable to automated fixed-point reasoning for working with inductive proofs [150]. The provability relation, denoted $\Gamma \vdash \varphi$, is defined to mean φ can be deduced using ML's proof system utilizing Γ as additional axioms. This provability relation is *sound*, i.e., $\Gamma \vdash \varphi \implies \Gamma \models \varphi$.

Next, we give an intuitive description of the semantics of ML. Given signature Σ , a Σ -model $\mathbb{M} = (M, \dots, \{M_\sigma\}_{\sigma \in \Sigma})$, Σ -patterns evaluate to sets of elements in M that *match* them. For

instance, variable $x \in EV$ evaluates to a *singleton* containing *element*, while $X \in SV$ evaluates to some subset of M . $\perp$ evaluates to the $\emptyset$. $\varphi_1 \rightarrow \varphi_2$ is matched by all elements that match φ_2 if they also match φ_1 . $\exists x. \varphi$ allows abstracting away irrelevant parts (i.e. x) of the pattern φ . $\varphi_1 \varphi_2$ allows for application of a function/operation/constructor, i.e. φ_1 , to its arguments, i.e. φ_2 . $\mu X. \varphi$ evaluates to the *smallest* set X s.t. φ evaluates to X . Other patterns such as $\neg \varphi$, $\varphi_1 \vee \varphi_2$ can all be derived from the basic ones mentioned above and have expected semantics. For instance, $\neg \varphi$ matches everything *not* matched by φ . $\varphi_1 \vee \varphi_2$ matches everything that either φ_1 or φ_2 matches. We refer the reader to [147] for a detailed discussion on the semantics of matching logic.

$\mathbb{K}$ in Matching Logic

Given a language L defined in $\mathbb{K}$, the $\mathbb{K}$ tool compiles the $\mathbb{K}$ -definition of L into a matching logic theory Γ^L . Note that this compilation involves generation of additional axioms to capture $\mathbb{K}$ concepts such as definedness, order-sortedness, etc. For an in-depth discussion, see [149]. The key idea behind ML is that it is simple, yet expressive to formally capture the semantics of a $\mathbb{K}$ definition. Element variables can be used to *abstract* away parts of terms through existential quantification ($\exists$). Set variables range over sets and behave as modal logic propositional variables be by fixed-point quantification μ . Symbols can represent functions, constructors and modal operators in a uniform way. Together with application ($\varphi_1 \varphi_2$), symbols can construct complex patterns from a simple signature. For example, a symbol f applied to arguments $a_1, a_2, \dots$ (see [144] for encoding tuples as patterns) can behave as a function. Similarly, a symbol $\langle k \rangle$ can be used to construct **Configuration** terms. Recall from section 5.1 that a $\mathbb{K}$ -definition consists of rewrite rules of the form $\varphi \Rightarrow \psi$. These rules define a *transition system* over computation configurations. This is captured in ML by adding a symbol $\bullet \in \Sigma$, called *one-path-next*. Intuitively, for a given configuration γ , $\bullet \gamma$ is *matched* by all states that can transition to γ in one step [147]. We can then define $\diamond \varphi \equiv \mu X. \varphi \vee \bullet X$, which equals $\varphi \vee \bullet \varphi \vee \bullet \bullet \varphi \dots$ or $\bigvee_{i \in \mathbb{N}} (\bullet^i \varphi)$. Intuitively, the $\diamond$ operator behaves exactly as its modal-logic equivalent. It matches all states that can *reach* φ in zero or more rewrite steps. Thus, a rewrite $\varphi \Rightarrow \psi$ is expressed in ML as $\varphi \rightarrow \diamond \psi$, or the set of all configurations that if match φ , can also reach ψ in zero or more steps. As mentioned earlier, ML has a sound proof system that is amenable to automated deduction. Thus, reasoning in $\mathbb{K}$ for a language L boils down to formal reasoning using its ML encoding Γ^L . Thus, reasoning that an initial configuration φ_{init} reaches a final configuration φ_{final} is showing $\Gamma^L \vdash \varphi_{\text{init}} \Rightarrow \varphi_{\text{final}}$.

5.3 PITFALLS OF THE SEMANTICS-FIRST APPROACH

In [section 5.1](#), we outlined advantages of the semantics-first approach. In this section, we outline potential pitfalls of the approach, applicable specifically in context of clinical decision support systems.

5.3.1 Performance of Semantics-based Tools

$\mathbb{K}$ has been used to define several large real-world languages such as C [\[42\]](#), Java [\[43\]](#), and Javascript [\[142\]](#). However, all these languages, had existing highly optimized language-specific compilers and interpreters for optimal performance, such as GCC [\[151\]](#), OpenJDK’s compiler and java virtual machine for Java, and Javascript engines such as the V8 engine [\[152\]](#) that runs Chrome and Node, and firefox’s SpiderMonkey [\[153\]](#). Thus, even though $\mathbb{K}$ -derived tools helped discover issues with either the language semantics or documentation, the $\mathbb{K}$ -derived execution engines for these languages didn’t replace their language-specific counterparts. This can be attributed to: (i) legacy use of existing toolchain, and, (ii) concerns of inferior performance of semantics-derived tools.

For the semantics-first approach, we intend to rely *solely rely* on semantics-derived tools for both execution and analysis. This leads to the question—are $\mathbb{K}$ semantics-derived tools performant enough to serve as execution and analysis engines in a production setting?

5.3.2 Developing a New Domain Specific Language from Scratch

$\mathbb{K}$ has been used to analyze programs in real-world languages, but these languages had existing informal language descriptions and reference test-suites to assist in development and testing of the semantics. The typical process for developing $\mathbb{K}$ semantics for such languages involves (i) coming up with the grammar in $\mathbb{K}$ using existing syntax definitions and programs as tests, and, (ii) developing the semantics rules using the informal description, until all tests pass.

Unlike existing $\mathbb{K}$ -semantics, in our case, we intend to implement the language from scratch and come up with reference tests for it at the same time, where both the definition and the corresponding tests need to be comprehensive enough to enable independent development of tools utilizing the semantics as a standard and tests as measures of compliance. Thus, it remains an open question whether $\mathbb{K}$ can serve as an appropriate foundational framework for this application.

CHAPTER 6: EVALUATING $\mathbb{K}$

In [chapter 5](#), we introduced the semantics-first approach to building systems, and discussed how $\mathbb{K}$, a framework for defining semantics of programming languages and type systems, enables it. Specifically, [subsection 5.1.1](#) discusses the rationale for using the semantics-first approach over the traditional state-of-art for programming language design that leads to tools based on ad-hoc language semantics and duplicated effort. In [section 5.3](#), we also touched upon some additional considerations for using $\mathbb{K}$ in the context of developing a language for developing a computer-interpretable-guideline DSL. We outlined the following concerns:

- (Q1) Can the $\mathbb{K}$ generated tools be performant enough to serve as the sole tools?
- (Q2) Can the $\mathbb{K}$ semantics replace an informal language description/manual?
- (Q3) Can the $\mathbb{K}$ -based semantics first approach be used to develop a semantics from scratch, without even an informal description of the language, and, comprehensive tests to go long with the same? Can the semantics serve as a comprehensive document to implement other tools for the language?

In the following sections attempt to answer these questions. Specifically, in [section 6.1](#), we describe work on modeling the formal semantics of the Ethereum Virtual Machine (EVM) in $\mathbb{K}$, where the generated interpreter’s performance compares favorably against a custom C++ implementation, providing evidence that (Q1) can be addressed. Moreover, given the executable nature of the semantics, the $\mathbb{K}$ semantics elaborate on ambiguities in the Yellow Paper—the official paper-based specification of the EVM. Not only can the $\mathbb{K}$ -semantics replace an informal language description, their detailed nature can also serve as a solid foundation to base other tools on, addressing (Q2).

In [section 6.2](#), we discuss work on implementing a DSL for calling functions in Ethereum smart contracts using the Application Binary Interface. Unlike previous $\mathbb{K}$ languages, this DSL neither has an informal semantics, nor existing test suites. But the language and tests were developed concurrently with the $\mathbb{K}$ definition, providing evidence that (Q3) can be addressed.

6.1 $\mathbb{K}$ EVM: SEMANTICS OF THE ETHEREUM VIRTUAL MACHINE

The Ethereum ecosystem is a blockchain-based platform where accounts have snippets of computer code called *smart contracts*, that enable programmatic governance of transactions

on the Ethereum network. All *smart contracts* on the blockchain must be specified in an assembly-like stack-based language called the Ethereum Virtual Machine (EVM). The semantics of the Ethereum Virtual Machine are described semi-formally in a specification called the Ethereum Yellow Paper [154].

Smart contracts execute when a transaction calls the account, and, among other features, these contracts can tally user votes, communicate with other contracts, store or represent tokens and digital assets, and send or receive money in cryptocurrencies, without requiring trust in any third party to faithfully execute the contract [155, 156]. The growing popularity of smart contracts has led to increased scrutiny of their security. Bugs in such contracts can be financially devastating to the involved parties. An example of such a catastrophe is the DAO attack [157], where 150 million USD worth of Ether was stolen, prompting an unprecedented hard fork of the Ethereum blockchain [158].

To address these issues in a principled manner, in [44]¹, we formalized the semantics of the EVM in $\mathbb{K}$, with the intention to utilize the semantics-generated tools for both execution and analysis of $\mathbb{K}$ EVM programs. Specifically, we intended to demonstrate that:

- The $\mathbb{K}$ definition can be used not only to derive semantics-based tools, but also as an *unambiguous alternative* to the informal Yellow Paper semantics specification to implement new tools. In other words, the semantics themselves are *readable* to enable one to comprehend all subtleties and nuances of the language.
- The generated tools, including the interpreter, are *performant* enough for use in place of custom EVM-specific tools.
- The $\mathbb{K}$ definition is *complete*, as in, the interpreter generated from it passes all tests in the reference test suite.
- The definition is useful, i.e., the generated tools can be used for formal analysis in some meaningful way.

For a more complete reading, please see [44]. Here, we only present arguments that help answer the questions we posed at the beginning of this chapter. The $\mathbb{K}$ semantics of EVM presented in [44] has 2644 lines of non-blank and non-literate code and passes all reference tests published by the Ethereum Foundation for testing EVM implementations. For reference, the EVM specific C++ implementation has 4588 lines of code. This measurement was taken on commit-hash ee0c6776c of <https://github.com/ethereum/cpp-ethereum> by counting non-blank lines of all *.h and *.cpp files in subdirectory `libevm`. We argue that these numbers

¹Joint work with E. Hildenbrandt et al.

are not atypical for implementing an interpreter for a small real-world programming language, not to mention the extra tools that $\mathbb{K}$ provides for analysis and security along the way.

In Table 6.1, we present a performance comparison between KEVM, the Lem semantics [159], and the C++ reference implementation distributed by the Ethereum foundation. At the time of publication of [44], the Lem semantics: the only other executable formal specification of the EVM that we are aware of at the time of comparison.

Test Set (no. tests)	Lem EVM	KEVM	cpp-ethereum
Lem (40665)	288:39	34:23	3:06
VMStress (18)	-	72:31	2:25
VMNormal (40665)	-	27:10	2:17
VMAll (40683)	-	99:41	4:42
GSNormal(22705)	-	35:00	1:30
GSQuad (250)	-	855:24	0:21
GSAll (22955)	-	889:00	1:51

Table 6.1: Lem EVM vs KEVM vs cpp-ethereum

All execution times are given as the full sequential CPU time (in MM:SS format) on an Intel i5-3330 processor (3GHz on 4 hardware threads) and 24 GB of RAM. The row Lem indicates a run of all the tests that the Lem semantics can run (a subset of the VMTests). The row VMStress indicates a run of all 18 stress tests in the test-suite, to compare the performance of KEVM with the C++ implementation. The row VMNormal is a run of all the non-stress tests in the test-suite (*not* the same set of tests as Lem). VMAll is the addition of the second and third rows and is included for completeness. The last three rows indicate a runs of the GeneralStateTests; GSNormal are the non-stress tests, GSQuad are the stress tests, and GSAll is the addition of the two. Under the GeneralStateTests, our tools performs well except in the case of QuadraticStateTests (250 out of 22955).

As shown in the comparison, the automatically extracted interpreter for KEVM outperforms the currently available formal executable EVM semantics. $\mathbb{K}$ EVM compares favorably to the C++ implementation, performing under 30 times slower on the stress tests, roughly 20 times slower on all tests, and only 11 times slower on the Lem and VMNormal tests. Note advancements to $\mathbb{K}$'s concrete execution capabilities since the publication of [44] may make the $\mathbb{K}$ EVM interpreter compare even more favorably against the C++ implementation.

Next, we look at a more holistic comparison of $\mathbb{K}$ EVM against related approaches, some of which are also semantics-based. We compare these approaches using the following metrics:

- **Spec.:** Suitable as a formal specification?
- **Exec.:** Executable on concrete tests?

- **Tests:** Passes the Ethereum test-suites?
- **Prover:** Serves as theorem prover for EVM?
- **Bugs:** Heuristic-based tools for finding bugs?
- **Gas:** Analyzes gas complexity of EVM programs?

Table 6.2 shows an overview of the results of our comparison. We briefly describe each effort and compare it to the relevant KEVM artifact. The projects fit two categories: semantic specifications and smart contract analysis tools.

6.1.1 Semantic Specifications

Ethereum Yellow Paper: The Yellow Paper is the official document describing the execution of the EVM [154], as well as other data, algorithms, and parameters required to build consensus-compatible EVM clients and Ethereum implementations. It cannot be tested against the conformance test-suite; instead, it serves as a guide for implementations to follow. Much of the machine definition is supplied as several mutually recursive functions from machine-state to machine-state. The Yellow Paper is occasionally unclear or incomplete about the exact operational behavior of the EVM; in these cases, it is often easier to simply consult one of the executable implementations.

Cpp-ethereum: Cpp-ethereum is the C++ implementation that at the time served as a de-facto semantics of the EVM [160]. The Yellow Paper and the C++ implementation were developed by the same group early in the project, so the Yellow Paper conforms mostly to the C++ implementation. In addition, the conformance test-suite is generated from the C++ implementation. This means that if the Yellow Paper and the C++ implementation disagree, the C++ implementation is favored.

Lem semantics: A Lem ([161]) implementation of EVM provides an executable semantics of EVM for doing formal verification of smart contracts [159]. Lem compiles to various interactive theorem provers, including Coq, Isabelle/HOL, and HOL4. The Lem semantics does not capture inter-contract execution precisely as it models function calls as non-deterministic events with an external (speculated) relation dictating the “allowed non-determinism”. This semantics is executable and passes all tests in the VMTests test-suite except for those dealing with more complicated inter-contract execution, providing high levels of confidence in its correctness.

GMS small-step specification: A small-step specification of the EVM inspired by the EtherLite semantics of [162]. The specification is non-executable but provides a precise guide for implementers of the EVM [163].

Tool	Spec.	Exec.	Tests	Prover	Bugs	Gas
Yellow Paper	✓	✗	✗	✗	✗	✗
cpp-ethereum	✗	✓	✓	✗	✗	✗
Lem spec	✓	✓	✓	✓	✗	✗
Oyente	✗	✓	✗	✗	✓	✓
hevm	✗	✓	✗	✗	✗	✗
Manticore	✗	✓	✗	✗	✓	✓
REMIX	✗	✓	✗	✗	✓	✓
F*	✗	✓	✗	✓	✓	✗
KEVM	✓	✓	✓	✓	✗	✓

Table 6.2: Feature comparison of EVM semantics and other software quality tool efforts.

6.1.2 Smart Contract Analysis Tools

Oyente: Oyente is an EVM symbolic execution engine written in Python supporting most of the EVM, with heuristics-based drivers of the engine for bug-finding [164].

Manticore: Manticore is symbolic execution engine for virtual machines, including models for x86, x86_64, ARMv7, and EVM [165]. This tool exports a Python API for specifying programs, driving symbolic execution, and checking assertions.

REMIX REMIX is a JavaScript implementation of the EVM with a browser-based IDE for building and debugging smart contracts [166]. Some static analysis is built into the tool, allowing it to catch pre-specified classes of bugs in smart contracts.

Hevm: Hevm [167] is a Haskell implementation of EVM including on an interactive debugger mode, which allows stepping through contract execution one opcode at a time.

F* formalization of EVM: An implementation of the EVM in the F* language [168] which passes roughly half of the VMTests at the time of writing. The same paper discusses an on-paper small-step specification of the EVM as well [169].

Discussion: At the start of this chapter, we asked whether tools generated from the semantics can be performant enough to replace their language-specific counterparts. We argue that the semantics-first treatment of $\mathbb{K}$ EVM addresses said question. Specifically, the semantics-derived interpreter compares favorably against a language-specific C++ implementation, and other tools are able to perform as well as other language-specific tools on metrics from [Table 6.2](#).

We based our $\mathbb{K}$ semantics on the Yellow Paper and found several inconsistencies that were confirmed by its developers [\[44\]](#). This stems from the fact that unlike the $\mathbb{K}$ semantics, the Yellow Paper is an informal document in natural language. Often, the Yellow Paper was found to be unclear, underspecified or inconsistent with the behavior of actual implementations. As the $\mathbb{K}$ semantics are executable, all details must be considered for our interpreter to pass all tests in the reference test-suite. We utilized the developer documentation for our semantics to generate an alternative to the Yellow Paper itself, called the Jello Paper [\[170\]](#). The Jello Paper has been well-received by the community, with discussions to designate it as the *canonical document* instead of the Yellow Paper. For more details, we refer the reader to section VII of [\[44\]](#). Thus, it demonstrates the $\mathbb{K}$ semantics can serve as a standalone language manual instead of an informal paper-based one.

6.2 ETHEREUM ABI DSL

EVM lacks high-level constructs including functions and explicit data types. Any invocation of an EVM program always begins with the VM’s program counter set to 0. Higher level languages such as Solidity and Viper, however, have functions and types, allowing users to directly call functions in contracts without dealing with low-level EVM code. This lack of high level constructs makes writing specifications for programs at the EVM level tedious and error prone.

To address this, in [\[44\]](#), we introduced a DSL modeled on the Ethereum Application Binary Support Interface ABI. The Ethereum ABI [\[171\]](#) is a mechanism for simulating a function call at the EVM level. Contracts are written in higher level languages and call functions in other contracts. The ABI facilitates this by specifying the encoding and decoding rules for a transaction’s calldata: an input field available to each transaction. The ABI is an additional protocol that higher level languages can choose to comply with to ensure compatibility with other smart contracts written in other languages. Thus, compilers for popular high-level smart contract languages often compile to ABI-compliant EVM code. As it’s not an official part of the Ethereum protocol, it’s not a part of the Yellow Paper.

Since most smart contracts needing formal verification are written in a high-level language

like Solidity [172] that produce ABI-compliant code, we created a Domain Specific Language (DSL) for interacting with ABI-compliant compiled code. By using our DSL, in [45]², we demonstrated that verification of complex smart contracts at the level of the EVM bytecode, was feasible. Operating at the EVM-level had many benefits, namely:

- Our methodology worked for any high-level language, as long as it compiled down to ABI-compliant code.
- As our DSL is built atop the EVM semantics, updates to the semantics are automatically accommodated.

6.2.1 Verifying Contracts Using Our DSL

Specifying the ERC20 Standard in $\mathbb{K}$: The ERC20 standard [173] is a standard for implement tokens within smart contracts, essentially enabling trade of third-party tokens using the Ethereum infrastructure. As an informal document, it specifies the correctness properties that token contracts must satisfy. Unfortunately, however, it leaves several corner cases unspecified, making it less than ideal to use in the formal verification of token contracts.

As part of verifying ERC20 token implementations, we implemented the semantics of ERC20 in $\mathbb{K}$. Our definition clarifies what data (e.g., balances and allowances) are handled by the various ERC20 functions and the precise meaning of those functions on such data. It also clarifies the meaning of all the corner cases that the ERC20 standard omits to discuss, such as transfers to itself or transfers that result in arithmetic overflows, following the most natural implementations that aim at minimizing gas consumption.

To evaluate the effectiveness of both the EVM semantics, and the associated ABI DSL, we verified three popular smart contracts: the Vyper ERC20 token³, the HackerGold (HKG) ERC20 token⁴, and OpenZeppelin’s ERC20 token⁵. Of these, the Vyper ERC20 token was written in Vyper, and the others are written in Solidity. We compiled the source code down to the EVM bytecode using each language compiler, and executed our verifier to verify that the compiled EVM bytecode satisfies the ERC20 standard. During this verification process, and found divergent behaviors across these contracts that do not conform to the standard.

²Joint work with Park et al.

³https://github.com/ethereum/vyper/blob/master/examples/tokens/ERC20_solidity_compatible/ERC20.vy

⁴<https://github.com/ether-camp/virtual-accelerator/blob/master/contracts/StandardToken.sol>

⁵<https://github.com/OpenZeppelin/openzeppelin-solidity/blob/master/contracts/token/ERC20/StandardToken.sol>

6.2.2 Discussion

Note that for a more thorough introduction to our DSL and corresponding verification efforts, we refer the reader to [44, 45]. Here, we only mention it in the context of answering the questions we posed at the start of this chapter. All previous attempts at defining semantics in $\mathbb{K}$, such as C, Java and EVM, had relied on existing informal or semi-formal semantics documentation. Moreover, existing test suites could be relied upon to *quantifiably test* the completeness of the semantics. The DSL described in this section, however, represents a break from traditional $\mathbb{K}$ development as: (a) the DSL had to be implemented from scratch without an existing language manual, and, (b) tests had to be written as the language was developed. Moreover, as discussed briefly in 6.2.1, our DSL can be used to verify *real-world smart contracts* at the level of the EVM-bytecode.

At the beginning of this chapter (see [chapter 6](#), we attempted to evaluate whether $\mathbb{K}$ would be a good-choice for a *solely* semantics-first approach to clinical decision support using the following questions:

- (Q1) Can the $\mathbb{K}$ generated tools be performant enough to serve as the sole tools?
- (Q2) Can the $\mathbb{K}$ semantics replace an informal language description/manual?
- (Q3) Can the $\mathbb{K}$ -based semantics first approach be used to develop a semantics from scratch, without even an informal description of the language, and, comprehensive tests to go along with the same? Can the semantics serve as a comprehensive document to implement other tools for the language?

By *solely*, we meant that we intend the semantics-based tools to be the *de-facto* tools for our language. In other words, the semantics-derived tools should supplant any language specific ones.

In [section 6.1](#), we presented a formalization of the Ethereum Virtual Machine (EVM) in $\mathbb{K}$, called $\mathbb{K}$ EVM. $\mathbb{K}$ EVM was developed using an informal semantics of the EVM described in a document known as the Yellow Paper. We demonstrated that the $\mathbb{K}$ semantics-derived interpreter not only passed all tests in EVM’s conformance test suite, but also compared favorably in terms of performance to the de-facto C++ language-specific implementation, while offering stronger correctness guarantees, addressing (Q1). Moreover, we found inconsistencies in the original Yellow Paper, to which our semantics has since become a popular alternative, addressing (Q2).

Next, we implemented a DSL based on the Ethereum ABI to make verification of smart contracts easier. Unlike earlier $\mathbb{K}$ -based efforts, the language was designed and implemented

entirely using $\mathbb{K}$, without the help of any existing documentation or tests. Although it is much smaller in scope and less complex than other $\mathbb{K}$ languages (such as C or Java), our DSL could be utilized to verify real-world smart contracts against desired non-trivial specs (such as ERC20). Thus, our work demonstrated that $\mathbb{K}$ can be used to implement languages from scratch, with *performant* and *useful* semantics-derived tools that can function as the *sole* tools for the language, addressing (Q3).

CHAPTER 7: SEPARATING CONCERNS: MODULAR AND SAFE CLINICAL DECISION SUPPORT

In [subsection 2.2.3](#), we briefly mentioned components that conceptually make up a CDSS. This chapter describes the motivations behind decomposing the system as such. First, we describe where CDSSs may loosely fit within the larger context of control theory and motivate the need for tailored architectures for CDSSs. Then we briefly identify some unique challenges of encoding medical knowledge. Finally, we talk about encoding medical knowledge directly through rewriting, and associated issues.

7.1 CONTROL THEORY AND CDSSS

Control theory deals with the design and analysis of *closed-loop* systems, i.e., systems where the inputs are affected at-least in part by outputs. Typically, such systems are characterized by:

- The *plant* represents the part of the system *to be controlled*.
- The *control* or *compensator* is the part of the system that provides *satisfactory characteristics* or *regulation* [\[174\]](#).

When viewed from the lens of a closed-loop system, the patient behaves as the plant, and the goal is optimizing patient outcomes [\[175\]](#)¹. The CDSSs collects patient data through sensor, assessments and electronic health records and recommends treatment strategies to optimize patient outcomes. But, unlike traditional closed-loop systems CDSSs must additionally account for:

- **Patient Physiology:** While typical systems have plants whose dynamics can be mathematically described, doing so in case of CDSSs has traditionally been very challenging. This can partially be attributed to factors such as diversity among patients, age, etc. Moreover, unlike traditional systems, patient state is almost always *partially observable*. For instance, while data about a patient’s blood counts might be a few hours old, the heart rate might be almost real-time.
- **Absence of Direct Control:** While some medical devices (such as insulin pumps) may directly administer treatment, most complex CDSSs rely on HCPs to perform recommended treatment. But there is no guarantee of the recommendation being

¹Joint work with Song et al.

followed, leading to a *divergence* between expected and actual scenarios. Thus, the CDSS must reconcile said divergences over time.

- **Customizations:** CDSSs rely on User Interfaces (UIs) for a significant part of their interaction with the physical world. The effectiveness of these interactions directly affects quality of care. To maximize effectiveness, CDSSs need to accommodate hospitals divergences in specializations and degrees of expertise, adaptations of BPGs, available pharmacy and medical devices, and User Interface (UI) design.

7.2 A GENERIC TREATMENT OF BEST PRACTICE GUIDELINES

A DSL for representing BPGs must accommodate the diversity in BPGs. Thus, we need characteristics that our DSL must support that are common across guidelines. To arrive upon these, in [175], we attempted to come up with a convenient semi-formal *abstraction* that can describe most guidelines.

We observed that most guidelines consisted of statements that fit into the form “if φ do α ”. Note statements without a condition can be represented simply as if *true* do α . Thus, we can represent each statement as a tuple (φ, α) . Given a BPG B , we say:

$$Statements(B) = \{(\varphi_1, \alpha_1), (\varphi_2, \alpha_2), \dots, (\varphi_n, \alpha_n)\} \quad (7.1)$$

to refer to all statements within B . Next, we observed that:

- Statements were often required to be carried out *concurrently*.
- There may be *ordering* between statements.

To capture this, first, we define the concept of a *workflow* $W = (S_W, \leq_W)$ as a *lower semi-lattice* on a partition $S_W \subseteq Statements(B)$. Specifically, for any pair $((\varphi_1, \alpha_1), (\varphi_2, \alpha_2))$, there exists some $(\varphi, \alpha) \in S_W$ s.t.

$$(\varphi, \alpha) \leq_W (\varphi_1, \alpha_1) \quad \bigwedge \quad (\varphi, \alpha) \leq_W (\varphi_2, \alpha_2) \quad (7.2)$$

and there is no $(\varphi', \alpha') \in S_W$ s.t.

$$(\varphi', \alpha') \leq_W (\varphi_1, \alpha_1) \quad \bigwedge \quad (\varphi', \alpha') \leq_W (\varphi_2, \alpha_2) \quad \text{and} \quad (\varphi, \alpha) \leq_W (\varphi', \alpha') \quad (7.3)$$

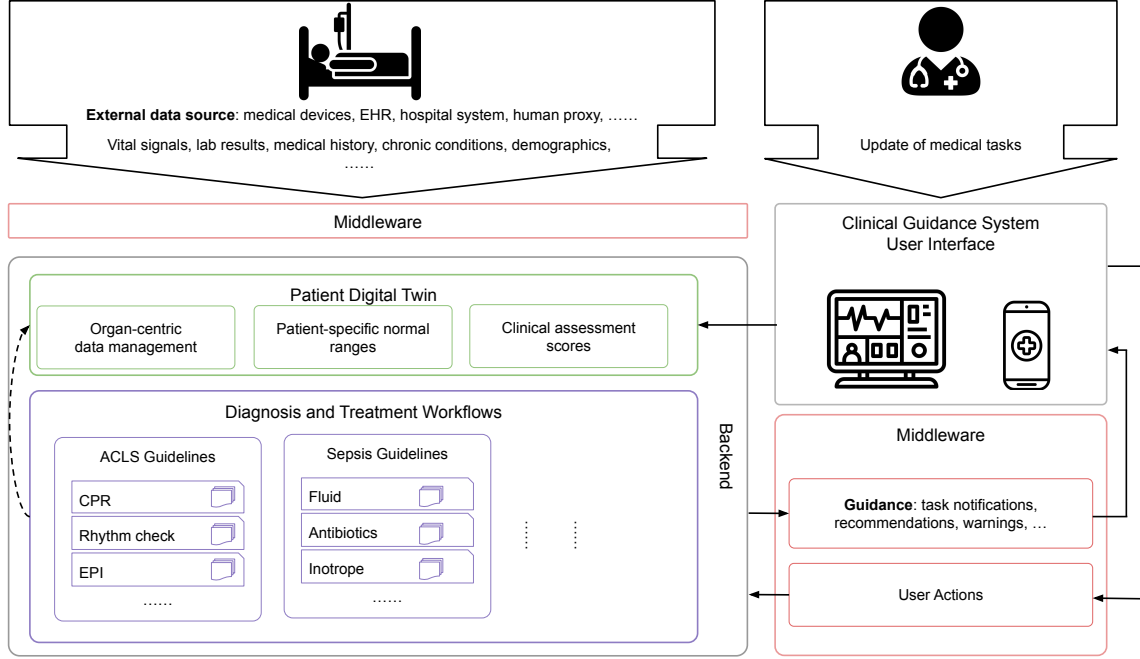


Figure 7.1: CDSSs Components

We can then define the guideline B as the *partial order*:

$$(\{W_1, W_2, \dots, W_n\}, \leq) \quad (7.4)$$

where, W_i is the workflow $(S_{W_i}, <_{W_i})$ and $S_{W_1} \cup S_{W_2} \cup \dots \cup S_{W_n} = \text{Statements}(B)$. Note that a workflow can contain a single statement to enable partitioning $\text{Statements}(B)$ into disparate workflows. Thus, given a workflow $W = (S_{W_i}, <_{W_i})$ an execution $\alpha_1, \alpha_2, \dots, \alpha_n$ is a valid execution of actions of a workflow iff:

$$(\varphi_1, \alpha_1) \leq_W (\varphi_2, \alpha_2) \leq_W \dots \leq_W (\varphi_n, \alpha_n) \quad \text{and} \quad \varphi_1 \wedge \varphi_2 \wedge \dots \wedge \varphi_n \text{ is satisfiable.} \quad (7.5)$$

7.3 A MODULAR ARCHITECTURE FOR CDSSS

This section describes the architecture for CDSSs that our work adopts. As shown in Figure 7.1 and subsection 2.2.3, we conceptually decompose CDSSs into three main components: (i) a *frontend* or a UI that forms the user-facing part of the system, (ii) a *backend* that encapsulates encoded medical knowledge, and, (iii) a *middleware* that connects the aforementioned two components. Next, we describe each of these components in detail.

7.3.1 Frontend

The frontend is the user-facing component of a CDSS, and handles interactions with users by collecting user actions, delivering guidance, and displaying data and information. Ideally a CDSS frontend must be customizable according to user preferences. As these changes don't directly affect the underlying medical knowledge, they should remain *localized* to the frontend itself. Thus, the architecture shown in [Figure 7.1](#), and many implementation from [chapter 4](#), compartmentalize the frontend into a standalone component that can permit adaptations to accommodate user and hospital-specific preferences that remain isolated to the frontend itself.

Having a modular frontend is also necessitated by increasing adoption of devices such as phones and tables at medical establishments. Thus, frontends for modern CDSSs further need to accommodate diversity in devices, such as phones, portals at patient's bedside, etc., and the associated data entry methods, such as digital keyboards, voice assistants, etc. Thus, a modular, self-contained frontend is critical to solving problems with modern CDSS implementations.

7.3.2 Backend

Conceptually, the part of the system that encodes the BPG constitutes what we call the *backend*. [Figure 7.1](#) depicts these as treatment workflows. In practice, however, the backend can be intertwined with other components in a monolithic system, be implemented as a shareable, standalone component, or be something in between. Moreover, the *computable encoding* may not resemble a traditional non-executable BPG.

As mentioned in [chapter 3](#), having high-quality, guidelines with mechanism that enable systematic validation of content and ease of sharing is vital to wider adoption. This work argues that this is enabled by computer-interpretable guidelines that can serve as both textual BPGs and their computer interpretable counterparts. Thus, ideally, the backend should comprise of a computer-interpretable guideline.

In [chapter 4](#), we introduced existing approaches for expressing guidelines in a computer-interpretable manner. As discussed in [section 4.3](#), several approaches also stress on using formal methods-based techniques to further ensure system correctness. Thus, good BPGs, and their computer-interpretable translations, require significant effort to develop and verify. This cost can be offset by ensuring that the backend can be shared across CDSSs implementations. Thus, in this work, we build CDSSs using an architecture where the backend is a *standalone* component that be composed with setting-specific frontends to build customizable but safe

CDSSs. By standalone, we specifically mean that the backend should be isolated such that any changes to any other component of the CDSSs should not percolate to the backend itself.

7.3.3 Middleware/Additional Infrastructure

[subsection 7.3.1](#) and [subsection 7.3.2](#) talk about the importance of ensuring that conceptually different components are also implemented and maintained as completely *independent* entities in practice. This modularity necessitates the need for additional *glue code* to tie components in a functioning system. For example, a frontend and backend implemented completely independently of each other, in unrelated languages, require standardized communication protocols. Moreover, as CDSSs rely on a diverse spectrum of devices for data, such as patient health records, real-time monitors and sensors, additional code must be written to connect the data to places where it’s used in the backend and frontend.

In this work, we refer to any additional code outside the frontend and backend as the “middleware” or “additional infrastructure”. Note that we use either term interchangeably throughout this work. Additionally, as shown in [Figure 7.1](#), we try to *organize* patient data into *digital twins* of organ systems whenever applicable. Notionally, we use “digital twins” as virtual entities that encapsulate actual physical processes or objects, as is typical in modeling and simulation [176]. In practice, however, we recognize that such abstractions may be imprecise, due to differences between CDSSs, and traditional closed-loop systems discussed at the start of this chapter. Thus, digital twins in our context may simply be simple structures that hold linked data. For example, while we may notionally refer to an abstract data type used to record a patient’s heart rate and blood pressure as a digital twin of the patient’s cardiovascular system, such a twin would be a very incomplete representation of the patient’s actual cardiovascular system.

CHAPTER 8: $\mathbb{K}$ -BASED COMPUTER INTERPRETABLE GUIDELINES

In [chapter 7](#), we described an architecture for building safe and modular clinical decision support systems. We split CDSSs into three separate components: (a) a frontend that facilitates interaction with HCPs, (b) a backend that serves as a computer-interpretable encoding of the BPG, and, (c) additional infrastructure that wires components together. This, and upcoming chapters, focus on building backends for CDSSs utilizing the *semantics-first* approach described in [chapter 5](#). By semantics-first, we mean that: (a) the semantics of medical knowledge in the BPG is accurately captured, and, (b) the semantics of the language used to describe the BPG is formally defined.

In [section 7.2](#), we briefly described characteristics of BPGs that a CIG language must accommodate. To this end, we attempted to come up with a framework that can accommodate expressing a large number of diverse BPGs. Thus, we broke BPGs into smaller statements that we organized into:

- a workflow containing statements that are executed sequentially, and,
- workflows within a guideline may be executed concurrently.

In this chapter, we describe a methodology to encode real-world BPGs as $\mathbb{K}$ definitions. Execution in $\mathbb{K}$ is inherently concurrent— if more than one $\mathbb{K}$ rule can apply, then $\mathbb{K}$ non-deterministically chooses one. Thus, we describe a way of systematically encoding medical knowledge in BPGs as $\mathbb{K}$ definitions. First, in [section 8.1](#), we introduce a real world BPG that will be utilized as running-example in the rest of this chapter. Note that we intentionally choose real-word examples, instead of small toy cases, to highlight that our philosophy scales to work in the real-world. Next, [section 8.2](#) describes $\mathbb{K}$ -ACLS, a $\mathbb{K}$ -based tool to assist HCPs conform to ACLS guidelines that attempts to follow the *semantics-first* approach. Specifically, we come up with an abstract representation that captures the semantics of the BPG from [section 8.1](#) with enough details to enable computer-interpretation. Finally, in [subsection 8.2.2](#), we describe a way to embed said abstract representation into a concrete $\mathbb{K}$ definition for execution.

8.1 ADVANCED CARDIOVASCULAR LIFE SUPPORT GUIDELINES (ACLS)

Advanced Cardiovascular Life Support (ACLS) are a set of BPGs periodically published by the American Heart Association (AHA) for management of life-threatening cardiac conditions that will cause or have caused cardiac arrest. The conditions that the guidelines treat range

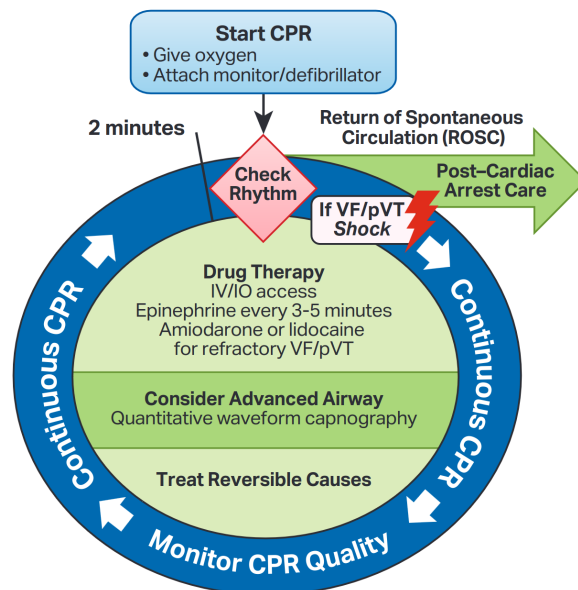


Figure 8.1: Advanced Cardiac Life Support Guidelines

from dangerous arrhythmias, i.e., irregularities in heart’s rhythm, to cardiac arrest—a cardiac emergency where the heart stop pumping [177]. The guidelines, according to the AHA, “are based on the most current and comprehensive review of resuscitation science, systems, protocols, and education” [178].

Figure 8.1 shows the AHA’s guidelines for advanced life support (ACLS) for managing cardiac arrest in adults [179]. AHA publishes guidelines for basic life support (BLS), and pediatric counterparts of both. While BLS is meant for a first responder to provide treatment with limited resources, such as an automatic emergency defibrillator (AED), ACLS is supposed to be followed by teams of trained professionals with advanced equipment for airway management, drug delivery, etc.

Why build CDSSs for ACLS?

Cardiac arrest treatment is extremely time-critical, and outcomes become significantly worse with every passing minute. This gathering relevant data about the patient infeasible. Moreover, as the seriousness of the condition necessitates multiple, simultaneous treatments to be administered rapidly, intervention is usually executed following standardized ACLS algorithms [177].

Prompt ACLS guidelines-conformant treatment has been shown to improve patient outcomes [180]. Moreover, deviations from the guidelines in in-hospital cases has been associated

with decreased rates of return of spontaneous circulation (ROSC), survival to discharge, and survival to discharge with favorable neurological outcomes [181]. Studies have also found that inadequate ACLS training can lead to poor retention, resulting in reduced ACLS conformance [182]. Thus, a CDSS that can provide situation-specific guidance about the next steps, especially in cases where HCP training might be inadequate, can potentially improve compliance, and consequently outcomes.

A Brief Overview of Advance Life Support Guidelines for Cardiac Arrest:

As we aren't concerned with intricacies of medical knowledge in the guideline, we present a brief and simplified description of the guideline-prescribed treatment. As ACLS is supposed to be performed in settings where necessary equipment is available, a electrocardiogram (EKG) machine is utilized to monitor the patient's cardiac rhythm. Certain cardiac arrhythmias, characterized by cardiac rhythms referred to as *shockable*, must be treated by delivering a therapeutic electric shock [183]. As shown in Figure 8.1, treatment has several parallel workflows such as:

- Periodic monitoring of the cardiac rhythm, and in case of a *shockable* rhythm, using a defibrillator.
- Continuous cardiopulmonary resuscitation (CPR).
- Administration of vital drugs such as epinephrine every few minutes.

As Figure 8.1 suggests, these workflows must be performed rapidly and periodically. As the duration of the intervention is typically a few minutes, making optimal use of available time is critical to outcomes.

8.2 THE $\mathbb{K}$ -ACLS SYSTEM

Figure 8.2 shows a snapshot of our CDSS for enabling compliance to the advanced life support guidelines described in section 8.1. Advanced life support is supposed to be performed by a team of HCPs, that take on roles such as a leader, CPR-provider, airway/respiratory specialist, Intravenous access (IV) and drug administration specialist, pharmacist, defibrillator attendant and members that serve as backups [177]. The leader is responsible for coordinating treatment, our tool render support through a handheld tablet held by the leader. Figure 8.2 show a snapshot of said tablet's main screen. Decision support, in the form of time-sensitive reminders, warnings when procedures are stopped prematurely, or exceed stipulated

Figure 8.2: Snapshot of $\mathbb{K}$ -ACLS

limitations, and confirmations regarding the patient rhythm’s are provided on the screen through popups and progress bars. For example, when the leader instructs the team to start CPR, and presses the “Start” button under the “CPR” section of the screen. A timer displays the duration and appropriate warnings about prematurely stopping CPR or exceeding the time for a CPR cycle are displayed.

In [section 7.3](#), we described conceptual components of CDSSs, and argued in favor of developing CDSSs using independently developed and maintained codebases aligned with these components. The $\mathbb{K}$ -ACLS system follows this philosophy and has: (i) a simple *frontend* written in Javascript using React [\[184\]](#) that handles user interaction, and, (ii) a $\mathbb{K}$ -based HTTP server that encodes the ACLS guideline and interacts with the frontend. The use of the Javascript based frontend allows our application to run on any modern web-browser. As shown in [figure 8.2](#), our frontend consists of forms and buttons that correspond to procedures in the algorithm in [figure 8.1](#). For example, the “Start” button in the CPR box results in a two-minute timer corresponding to the “2 minute continuous CPR” procedure in the informal description. The frontend is simple and doesn’t contain any guideline conformance related logic. Interaction with the frontend simply results in dispatch of *events* to the backend. For example, when the “Start” button is pressed, a “StartCpr” event is sent to the backend.

8.2.1 Capturing Execution-specific Details

A computer-interpretable version of a guidelines, unlike its textual counterpart, may require specification of additional details for execution. Such details can often be gathered from context in case of textual guidelines or may be unintentionally missing. This was also discussed in context of related approaches in [chapter 4](#), especially in [section 4.1.7](#).

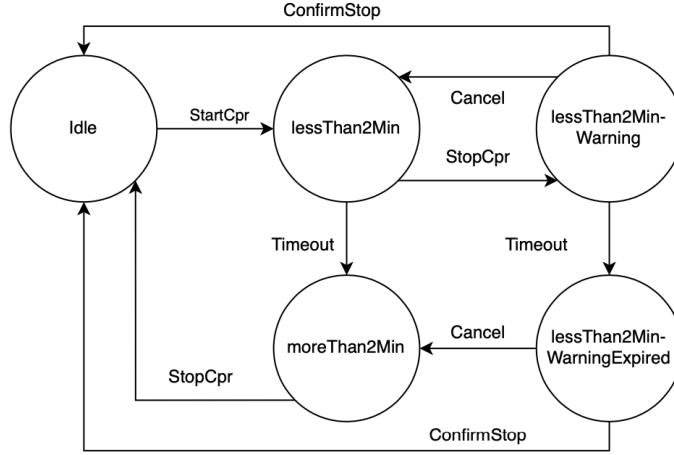
As discussed in [section 7.2](#), BPGs can be notionally represented using a collection of workflows. Each workflow has steps executed sequentially but may be concurrently executed with steps from other workflows. Consider the ACLS guidelines discussed in [section 8.1](#). Several procedures, such as administering drugs, and performing CPR occur concurrently, where each procedure has a set of tasks that need to be performed sequentially. There may also be some implicit order across workflows, but we address that in later chapters.

Modeling BPGs as State Machines

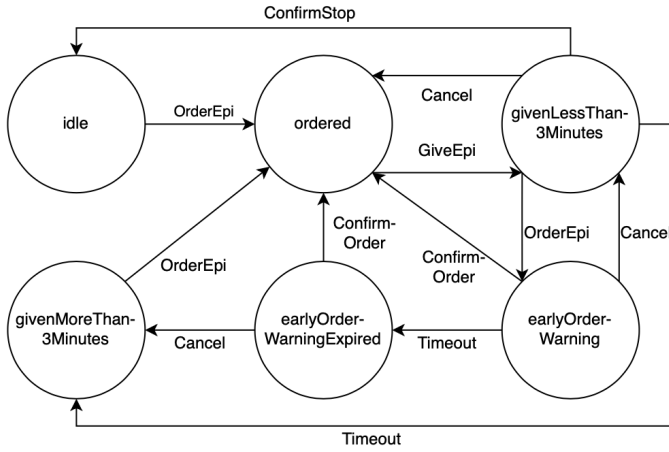
In this work, we choose to express medical logic using concurrently executing state machines with implicit queues for inter-machine communication. Communicating state machines (CSMs) is a well-understood model of concurrency [185] and is well-suited for representing medical guidelines in a computer-interpretable format that is also HCP-friendly. We shall discuss the motivations behind using CSMs in upcoming chapters. For now, it suffices to describe how our choice addresses issues mentioned in [subsection 8.2.1](#). Given a guideline represented as a collection of workflows, we describe each workflow via a state machine. The fact that CSMs can execute concurrently conveniently captures the inherent concurrency in workflows.

ACLS Workflows as State Machines

At the start of [section 8.2](#), we mentioned that the *frontend* of our CDSS is a simple Javascript-based application to facilitate user interaction through buttons and popups. We can now elaborate what we mean by *simple*. Button presses on the frontend correspond to *events* that trigger transitions in state machines shown in [Figure 8.3](#). For instance, pressing the “Start” button under the CPR part of the tool shown in [Figure 8.2](#) results in a **StartCPR** event that triggers the corresponding transition in the machine in [Figure 8.3a](#). Similarly, certain machine states result in events being dispatched to the frontend that cause popups or messages to be displayed on associated parts of the screen. Thus, the frontend itself contains no medical logic.



(a) CPR Machine



(b) Epinephrine Machine

Figure 8.3: Formal Machine Definitions

8.2.2 $\mathbb{K}$ -ACLS Backend

As described at the start of [section 8.2](#), the backend receives events from the frontend, processes them and dispatches the results back to the frontend. In [subsection 8.2.1](#), we described the process of details required for execution by expressing the BPG using communicating state machines. For a functioning backend through, we need to translate the abstract state machines into an executable medium. We do this by encoding the state machines in a $\mathbb{K}$ definition. Before we describe the encoding, we describe some additional challenges that we need to address in order to use $\mathbb{K}$ as a CDSS backend.

Additional Challenges

Asynchronous External Communication: Communication with external process in $\mathbb{K}$ is facilitated by built-in I/O support. $\mathbb{K}$ has built-in functions for many operations that map directly to their POSIX counterparts [186]. However, execution in $\mathbb{K}$ is single-threaded. Thus, to make $\mathbb{K}$ behave as an HTTP [187] server that accept messages and respond asynchronously, we need to write additional C++ initialization code and link it against $\mathbb{K}$'s LLVM backend [188].

Handling Timers: The ACLS workflows shown in Figure 8.1 require tasks being performed periodically. For instance, the algorithm calls for continuous CPR administration for two minutes between checking rhythm and performing defibrillation if needed. Similarly drugs like epinephrine must be administered every three minutes. $\mathbb{K}$ does not support setting timers that can interrupt the $\mathbb{K}$ process on expiration. Thus, we rely on external “timers” that, instead of interrupting the $\mathbb{K}$ process, place a `timeout` event at the head of the `<inputBuffer>` cell on expiration, which trigger corresponding transitions in state machines. For example, consider the CPR machine in Figure 8.3a. When CPR is started through the press of a button on the frontend, an external two-minute timer is set, and the machine transition from state `idle` to `lessThan2Min`. When the timer expires, the external timer sends a `Timeout` event to the $\mathbb{K}$, makes the CPR machine to transition from state `lessThan2Min` to `morethan2Min`.

$\mathbb{K}$ -ACLS Backend

Recall from subsection 5.2.1 that in $\mathbb{K}$ computation is described via rewrite *rules* that operate over a *configuration* that organizes data in labeled and potentially nested units called cells. Listing 8.1 shows the initial configuration for the definition $\mathbb{K}$ of our state machines-based representation. Note that unlike a typical $\mathbb{K}$ definition, the initial `configuration` does not have a `<k>` cell containing a `$PGM` variable replaced by the program AST at runtime, as the $\mathbb{K}$ definition is the program itself.

Lines 2-8 declare a cell `<machines>` that will be used to multiple `<machine> ... </machine>` cells, declared between Lines 3 and 7, where each cell stores all data related to a particular state machine. The `<id>` and `<state>` cells store the machine's name and active state respectively. The `<store>` cell is a map used to store any additional data. Note the following:

- (i) The `<machine>` cell is declared with an attribute `multiplicity=*`, signifying that the configuration can contain multiple instances of such cells that are added or removed

```

1 configuration
2   <machines>
3     <machine multiplicity="*" type="Set">
4       <id> .K </id>
5       <state> .K </state>
6       <store> .Map </store>
7     </machine>
8   </machines>
9   <inputBuffer> .JSONs </inputBuffer>
10  <outputBuffer> .JSONs </outputBuffer>

```

Listing 8.1: Initial Configuration

dynamically during execution. At the start of execution, the configuration has zero such cells.

- (ii) As the number of machines to represent is known and remains constant during execution, one might ask why they are not declared as a part of the initial configuration itself. We clarify that we deliberate not to do, due to reasons we explain shortly.

Lines 9 and 10 declare cells `<inputBuffer>` and `<outputBuffer>` respectively that hold comma-separated lists of JSON objects. As their name suggested, they are used as buffer to communicate with the frontend. Incoming events from the frontend are placed at the end of the `<inputBuffer>` and outgoing events intended for the frontend are consumed from head of the `<outputBuffer>`.

Initialization

Initialization involves populating the configuration with the initial state of each state machine. Listing 8.2 depicts the template of a rule for initializing machine any machine $\mathcal{M}$, with initial state u . Instead of starting `<machines>` cell that is pre-populated with `<machine>` cells, we dynamically initialize machines once the corresponding frontend component is rendered and ready. When the frontend component associated with machine $\mathcal{M}$ is successfully loaded, it sends a " $\mathcal{M}.init$ " event, which is added to the head of the input buffer. For instance, when the CPR part of the tool from Figure 8.2 has successfully loaded, it sends a "`cprMachine.init`", which is added to the head of the input buffer, where `cprMachine` is the name of the machine in the encoded $\mathbb{K}$ definition, stored under the `<id>` cell. Lines 2-6 of the rule introduce a new `<machine>` cell with name $\mathcal{M}$ and initial state u . The `.Bag` in $\mathbb{K}$, seen on Line 2, represents the absence of a cell, which the rule rewrites to a `<machine>` cell, resulting in its addition to the configuration under the `<machines>` cell. On Line 9, the $\mathbb{K}$

```

1 <machines>
2   .Bag => ( <machine>
3             <id>  $\mathcal{M}$  </id>
4             <state> u </state>
5             ...
6           </machine> )
7   ...
8 <machines>
9 <inputBuffer>  $\mathcal{M}.init$ , IN => IN </inputBuffer>

```

Listing 8.2: Template For Machine Initialization Rule

variable IN matches any list of JSON elements. Thus, " $\mathcal{M}.init$ " , IN matches any list with " $\mathcal{M}.init$ " at the head of the list (IN matches the tail of the list). When the rule fires, the list is rewritten to just the tail, resulting in the removal " $\mathcal{M}.init$ " event from the `<inputBuffer>` cell. For instance, in case of the CPR machine, the initialization rule is show in [Listing 8.3](#).

```

<machines>
  .Bag => ( <machine>
            <id> cprMachine </id>
            <state> idle </state>
            ...
          </machine> )
  ...
<machines>
<inputBuffer> cprMachine.init, IN => IN </inputBuffer>

```

Listing 8.3: cprMachine Initialization Rule

For each machine $\mathcal{M}$ in the guideline, the $\mathbb{K}$ definition contains a corresponding rule of the form shown in [Listing 8.2](#) in the definition. Next, we discuss each machine in detail to explain how we encode their transition systems in $\mathbb{K}$.

CPR Machine

The *CPR Machine* encodes the intended CPR procedure shown in figure [8.3a](#). As mentioned earlier, when the user clicks the “Start” button in the CPR pane in figure [8.2](#), an external two-minute timer is started. If the user decides to stop CPR before two minutes are up, a message is displayed to the user warning him of deviation from the intended guidelines. If the user stops CPR after two minutes, no warning message is displayed. [Listing 8.4](#) shows the rule that fires when the “Start” button is clicked. The rule does the following:


```

1 rule <machine>
2     <id> cprMachine </id>
3     <state> idle => lessThan2Min </state>
4     <store> ( "cprRunning" |-> ( _ => true ) ) ... </store>
5     ...
6 </machine>
7 <inputBuffer> "StartCpr" , IN => IN </inputBuffer>
8 <outputBuffer>
9     OUT => OUT +JSONs jsonResponse( cprMachine
10                                     | lessThan2Min
11                                     | startTimer( .JSONs ) )
12 </outputBuffer>

```

Listing 8.4: CPR Machine in $\mathbb{K}$

- (a) Consumes the "StartCpr" event from the <inputBuffer> in [Line 7](#).
- (b) Move the machine from state `idle` to state `lessThan2Min` in [Line 3](#).
- (c) Sends a `startTimer` action to the frontend via the <outputBuffer> (encoded as JSON) in [Lines 8-12](#), which results in the start of a two-minute timer in the frontend.

Note how features of $\mathbb{K}$ discussed in [section 5.2](#) make the definition *concise* yet *descriptive*. Specifically:

- The *configuration abstraction* mechanism enables only parts of the configuration that are not used in the rule to be easily ignored. The ... (dots) on [Line 5](#) contents of cell not used in the rule to be safely ignored. Other parts of the configuration, such as the <machines> cell and the <outputBuffer> cell need not be mentioned at all.
- Localized rewriting enables the rewrite symbol `=>` to apply on deeply nested subterms. This enables de-duplication, as parts of the term that remain unchanged don't have to be specified both on the LHS and RHS of the rewrite. For instance, the <store> cell on [Line 4](#) contains is a map that holds information about the machine, such as execution status. Recall that a map with n key-value pairs is depicted in $\mathbb{K}$ as $(k_1 \text{ |- } v_1), (k_2 \text{ |- } v_2), \dots, (k_n \text{ |- } v_n)$. Localized rewriting enables us to only rewrite the value mapped to key "cprRunning" from any value to `true` and ignore other parts of the map using ... notation.

Epinephrine Machine

The *Epinephrine Machine* encapsulates instructions for administering the drug Epinephrine

```

1 rule <machine>
2   <id> epiMachine </id>
3   <state> givenLessThan3Min
4     => earlyOrderWarningNotification
5   </state>
6   <store> .Map => ("tentativeOrder" |-> ORDERED) ... </store>
7 </machine>
8 <inputBuffer> { "event" : "orderEpi"
9               , "data"  : [ ORDERED:Int ] } , IN
10      => IN
11 </inputBuffer>
12 <outputBuffer> OUT
13   => OUT +JSONs jsonResponse( epiMachine
14                               | earlyOrderWarningNotification
15                               | showEarlyOrderWarning( .JSONs ))
16 </outputBuffer>

```

Listing 8.5: Epi Machine in $\mathbb{K}$

to the patient. The corresponding machine is shown in figure 8.3b. The user orders the drug using the “Order 1mg” button and can administer it using the “Give” button in Epinephrine pane in figure 8.2. Giving the drug results in the start of a three-minute timer, during which if a new order is placed, a warning is issued.

Listing 8.5 encodes the transition $\text{givenLessThan3Min} \xrightarrow{\text{orderEpi}} \text{earlyOrderWarningNotification}$ from Figure 8.3b. The rule fires when it has been less than three minutes before the last dose and a fresh order for Epinephrine is put in (Lines 8-11). Note that the input event here also has a payload corresponding to the dosage, depicted by a JSON object with fields "event" for the event's name and "data" for the dosage. As the guidelines dictate that Epinephrine must be administered every three minutes, ordering fresh Epinephrine is a deviation from the guidelines. Thus, an appropriate warning message is presented to the user (Lines 12-15).

```

<machine>
  <id>  $\mathcal{M}$  </id>
  <state>  $u \Rightarrow v$  </state>
  ...
  <machine>
  <inputBuffer>  $\sigma$ , IN => IN </inputBuffer>

```

Listing 8.6: Transitions as $\mathbb{K}$ -Rules

Encoding Remaining Transitions

Note that we only show a single rule from the *CPR* and *Epinephrine* Machines as other rules resemble the ones shown, and have a one-to-one correspondence to transitions in the state machines in [Figure 8.3](#). For instance, the rule in [Listing 8.4](#) encodes the transition “idle $\xrightarrow{\text{StartCpr}}$ lessThan2Min” in the CPR state machine in [Figure 8.3a](#). To encode the entire machine, along with the initialization rule, we encode every transition $u \xrightarrow{\sigma} v$ of machine $\mathcal{M}$ as a rewrite rule of the form shown in [Listing 8.6](#)

Shock Machine

We now describe the *Shock Machine*. Intuitively, this machine ensures: (a) shocks are only administered if the rhythm is shockable, (b) CPR is administered every two minutes, and, (c) drugs (such as Epinephrine) are periodically administered. In case the of deviation appropriate warnings are issued. The *Shock* machine differs from other machines, as its transitions depend on the status of both the *CPR* and *Epinephrine* machines.

[Listing 8.7](#) shows a rule from the *Shock* machine. The rule fires when the rhythm is *shockable*, and the user wants to administer a shock. Additionally, CPR hasn’t been administered

```
1 rule <machine>
2     <id> shockMachine </id>
3     <state>
4         shockableRhythm => checkRhythm
5     </state> ...
6 </machine>
7 <machine>
8     <id> cprMachine </id>
9     <store> ( "cprRunning" |-> false ) ... </store> ...
10 </machine>
11 <machine>
12     <id> epiMachine </id>
13     <store> ( "epiGiven" |-> true ) ... </store> ...
14 </machine>
15 <inputBuffer> "administerShock" , IN => IN </inputBuffer>
16 <outputBuffer> OUT
17 => OUT +JSONs jsonResponse( shockMachine
18                               | checkRhythm
19                               | confirmShock("CPR for
20                                              2 Minutes" ) )
21 </outputBuffer>
```

Listing 8.7: Shock Machine in $\mathbb{K}$

(Line 9), but Epinephrine has been administered (Line 13). In such a situation, the user is informed that it is safe to administer a shock, which should be followed by two minutes of CPR (Lines 16-20). However, it is not safe to assume that the rhythm would remain *shockable* for subsequent shocks. Thus, the machine moves from state `shockableRhythm` to `checkRhythm`. If the user tries to administer another shock, a prompt to confirm the rhythm will be displayed to ensure a shock is not administered in case of an *unshockable* rhythm.

We draw the user’s attention to the *succinctness* and *simplicity* of the rule. Despite expressing the interaction between three different machines, $\mathbb{K}$ ’s configuration abstraction mechanism ensures that only relevant parts of each machine are mentioned, making the transition *comprehensible*.

8.3 DISCUSSION

This chapter describes the process of systematically encoding BPGs expressed using informal flowchart-like notations as communicating state machines (CSMs). The CSMs can then be encoded in a $\mathbb{K}$, where, for every machine, $\mathcal{M}$, every transition $u \xrightarrow{\sigma} v$ can be encoded as a rewrite rule of the form shown in Listing 8.6. We discussed how *configuration abstraction*, *localized rewriting* and the inherent concurrency of rewriting enable the $\mathbb{K}$ encoding of the guideline to be both *succinct* and *simple*. But, comprehending such $\mathbb{K}$ based guidelines precludes comprehending the $\mathbb{K}$ language, which is unreasonable to expect not only from domain experts in medicine, but also from software developers in general. The $\mathbb{K}$ language is designed for development and implementation of programming languages, and can have an associated learning curve, especially if one is unfamiliar with the theory of defining formal semantics. Thus, understanding $\mathbb{K}$ may require additional effort for most software developers, who are more comfortable with popular general purpose programming languages such as Java, C and python. Moreover, the $\mathbb{K}$ definition can become harder to comprehend as more complex guidelines are modeled, requiring rules that encode transitions where multiple steps must be performed and complex conditions to determine when the transitions can occur.

In upcoming chapters, we address shortcomings of using $\mathbb{K}$ directly to model guidelines by coming up with a novel DSL specifically designed to encode medical logic. This language, called Medi $\mathbb{K}$, differs from $\mathbb{K}$ in many aspects. Notably Medi $\mathbb{K}$:

- Uses syntax that borrows from other languages for CSMs, notably P [49], to improve comprehensibility and readability, albeit at the expense of succinctness.
- Provides support for interacting with heterogeneous external devices, such as monitors, patient sensors, etc.

CHAPTER 9: MediK: TOWARDS SAFE GUIDELINES-BASED CDSSS

In [chapter 8](#), we discussed the process of encoding best practice guidelines (BPGs) as $\mathbb{K}$ definitions to describe a systematic way of building CDSSs. Specifically, we described the process of encoding medical knowledge in guidelines notionally via workflows that can be expressed abstractly as concurrently executing communicating state machines. Next, we described the process of expressing state machines as $\mathbb{K}$ definition, where $\mathbb{K}$ features such as configuration abstraction and local rewriting enable *conciseness*. We then combined the $\mathbb{K}$ definition with a simple Javascript-based frontend to develop a CDSS for assisting healthcare practitioners (HCPs) follow the Advanced Cardiac Life Support (ACLS) guidelines for managing cardiac arrest published by the American Heart Association.

In [chapter 3](#), we described that wider CDSS adoption is incumbent on a having a systematic way of developing guidelines with validated content. The semantics-first approach we described in [section 5.1](#) attempts to enable such a way. It dictates that: (i) the semantics of the language be formally defined, from which tools are derived from a correct-by-construction fashion, and, (ii) ensuring that semantics of medical knowledge is accurately described. As is typically the case, the $\mathbb{K}$ -ACLS system’s functionality in [section 8.2](#) was informed by collaboration between HCPs and computer scientists. As discussed in [section 8.3](#), while the $\mathbb{K}$ -based representation of the ACLS guideline was concise, it was not easily comprehensible to HCPs, or to other software engineers without prior experience of using $\mathbb{K}$. This chapter addresses limitations of our work from [chapter 8](#). Specifically, we describe MediK (pronounced “Medi-kay”¹) a novel domain-specific language (DSL) for expressing medical knowledge that is designed from the ground-up with HCP-comprehensibility in mind. As the name suggest, MediK has a formal $\mathbb{K}$ semantics, from which all tools for it are derived, including its interpreter. MediK has been used to implement a complex CDSS for management of sepsis in pediatric cases that has multiple concurrent workflows. Our MediK-based system has been shown to satisfy desired safety properties, and to the best of our knowledge, is the first such system with formal safety guarantees.

The rest of this chapter is organized as follows. In [section 9.1](#) we recall the example BPG from [section 2.1](#) for management of sepsis, to illustrate common requirements that DSL for modeling BPGs should satisfy. In [section 9.2](#), we introduce the MediK DSL and illustrate how it has been specifically designed to address said requirements. In [section 9.8](#), we evaluate the effectiveness of our approach by utilizing it to build a CDSSs intended for real-world use. In [section 9.9](#), we discuss how MediK builds on existing approaches in [chapter 4](#) to advance

¹MediK is a portmanteau of Medicine and $\mathbb{K}$

the state-of-art in addressing challenges from [chapter 3](#).

9.1 PEDIATRIC SEPSIS MANAGEMENT BPG

In [section 2.1](#), we presented a best practice guideline for management of sepsis in pediatric cases. We briefly describe the guidelines here. Recall that sepsis is life-threatening condition caused by the body’s extreme response to an infection and is a major cause of morbidity and mortality in children. Evidence indicates that timely identification and prompt treatment with antibiotics and intravenous (IV) fluids is *vital* for avoiding adverse outcomes [[53](#), [54](#)]. The BPG has several concurrent workflows with inter-workflow dependencies, making it suitable to study common characteristics of BPGs, which we recall here. Specifically, BPGs:

- Involve concurrent workflows, such as administering drugs, monitoring vitals, performing treatment, etc. There may also be inter-workflow interactions. For instance, a diagnosis of sepsis during the screening may require modifications to an ongoing course antibiotics.
- Often specified in a flowchart-like notation. See [[47](#)] and [[48](#)] for other flowchart-based BPGs for management of *cardiac arrest*, and screening, risk-reduction, treatment and survivorship in cancer care respectively.
- Require communication between heterogeneous agents such as monitors and Electronic Health Records (EHRs).
- Often use tables indexed by parameters such as age, weight, etc. to present normal/abnormal ranges for measurements, or recommended dosages for drugs.

In the following sections, we describe how these characteristics dictate the design philosophy behind MediK. We argue that this philosophy makes MediK both intuitive to HCPs, and suitable for expressing complex guidelines.

9.2 DESIGN CONSIDERATIONS

This section introduces the MediK DSL through its $\mathbb{K}$ syntax and semantics. We developed MediK to realize the semantics-first approach discussed in [section 5.1](#). Thus, it has:

- A formally defined, unambiguous semantics.

- A correct-by-construction interpreter derived from the semantics, as discussed in [chapter 6](#).
- A comprehensive suite of formal program analysis tools.
- The ability to quickly adapt physician feedback, as only the semantics need to be changed and all tools evolve automatically.

The remainder of this section introduces the MediK DSL and describes how it's designed to accommodate characteristics of BPGs discussed in [section 2.1](#).

9.2.1 HCP Comprehensibility

Our approach stresses that HCPs must be able to comprehend the guideline, and if necessary, making changes to it on their own. As discussed in [chapter 3](#), this has several hurdles, chief among which is the fact that HCPs are typically not trained to understand conventionally programming languages. Thus, HCPs need to work collaboratively to translate the BPG into a computable medium. In essence, the BPG serves as a functional specification for implementing the CDSS. But this may lead to a gap in the HCPs' understanding of the system, and the actual behavior. To address this, we designed MediK s.t. encoded guidelines resemble their physical, non-executable counterparts, with the intention that familiarity with non-executable guidelines would also translate to computable MediK ones.

Recall from [section 7.2](#) that BPGs typically involve concurrent workflows, often expressed using a flowchart-like notation that may involve inter-workflow interactions. In [subsection 8.2.2](#), we discussed the merits and suitability of concurrently executing state machines for modeling medical guidelines. Thus, we looked at state-of-art languages for modeling large concurrent systems using state machines, such as P [49], but made adaptations to make expressing BPGs easier.

In MediK, like in P, programs are expressed as concurrently executing instances of state machines that communicate via passing messages. Given a BPG where each workflow is expressed as a flowchart, we express said flowcharts as State Machines in MediK. Each flowchart node in the BPG is represented as a state in a state machine, and edges are represented as state transitions. During execution, instances of these machines are created, which interact with each other by passing events. Note the distinction between machine and its instance. A machine is analogous to an Object-Oriented Programming (OOP) class, whereas its instance is analogous to an OOP object.

Next, we describe MediK using its $\mathbb{K}$ -framework definition. Recall from [subsection 5.2.1](#), the $\mathbb{K}$ definition of a language has two components. The first is the language's syntax, which

```

1 [init] machine <IDENTIFIER>
2   receives <IDENTIFIER_LIST> {
3     vars <IDENTIFIER_LIST>;
4
5     [init] state <IDENTIFIER> {
6       entry [(IDENTIFIER_LIST)] {
7         <STMT> // entry block
8       }
9       on <IDENTIFIER> [(IDENTIFIER_LIST)] do {
10        <STMT> // event handler
11      }
12    }
13 }

```

Listing 9.1: Skeleton of a MediK Machine

is defined using a BNF-like notation. $\mathbb{K}$ utilizes this grammar to generate a parser for program in the language. We describe MediK’s syntax in depth in Section 9.3. The second is the semantics, which is defined using a $\mathbb{K}$ -configuration and rewrite rules. The $\mathbb{K}$ -configuration organizes the program’s execution state. Rewrite rules that operate over said configuration dictate the evolution of program state during execution. We describe the semantics in greater depth in Sections 9.4 and 9.5²

9.3 MediK SYNTAX

We use the skeleton of a MediK machine in Listing 9.1, and use it to describe the syntax. Note that we use [...] to denote optional constructs, <...> for mandatory constructs, lowercase for terminals, and uppercase for non-terminals.

A MediK program consists of a set of machine definitions, where a machine definition consists of:

- (Lines 1-2) The keyword **machine**, followed by the name and a comma-separated list of identifiers signifying events that it **receives** via the broadcast mechanism. One state in every machine, and one machine in a program can be prefixed with the keyword **init**. On execution, an implicit instance of this machine the created, and the **entry** block of initial state executed.
- (Line 3) A set of instance variables.

²The complete executable semantics is available at [189].


```

1 syntax StandaloneExp
2     ::= "new" Id "(" Exps ")" [strict(2)]
3     | "createFromInterface" "(" Id "," String ")" [strict(2)]
4     | Id "(" Exps ")" [strict(2)]
5 syntax Exp ::= Id
6     | Val
7     | Rat
8     | FloatLiteral
9     | "this"
10    |.UndefExp
11    | "obtainFrom" "(" Exp "," Exp ")" [seqstrict]
12    | "(" Exp ")" [bracket]
13    > Exp "." Exp [strict(1), left]
14    > Exp "+" Exp [seqstrict, left]
15    | Exp "-" Exp [seqstrict, left]
16    | Exp "*" Exp [seqstrict, left]
17    | Exp "/" Exp [seqstrict, left]
18    | Exp ">" Exp [seqstrict, left]
19    | Exp "<" Exp [seqstrict, left]
20    | Exp ">=" Exp [seqstrict, left]
21    | Exp "<=" Exp [seqstrict, left]
22    | "!" Exp [seqstrict, left]
23    | Exp "&&" Exp [strict(1), left]
24    | Exp "||" Exp [strict(1), left]
25    > Exp "==" Exp [seqstrict, left]
26    | "interval" "(" Exp "," Exp ")"
27    > Exp "in" Exp [macro]
28    | "parseInt" "(" Exp ")" [strict]
29    | StandaloneExp

```

Listing 9.2: MediK Expressions Syntax

- (Lines 5-12) A set of state declarations. Each state has a name, an optional entry block (Lines 6-8), and a set of event handlers (Lines 9-11). The entry block begins with the keyword **entry** and may contain a list of variables that are bound to values when an instance enters the state during execution. One state in the machine may be prefixed with **init**, specifying the initial state that an instance starts execution in.
- Event handlers begin with **on** followed by the event name, an optional list of variables bound to data in the event, followed by the keyword **do** a block of the handler's code.

Each entry and event handler block contains statements defined by syntax shown in Listing 9.3. The statements are written over expressions given by the syntax in Listing 9.2.

Recall from subsection 5.2.1, in $\mathbb{K}$, productions are defined using the keyword **syntax**, where terminals are enclosed in quotes (""), and non-terminals begin with an uppercase character.

MediK uses statement over expressions resemble counterparts in many commonly used

```

1 syntax Stmt ::=
2     StandaloneExp ";" [strict]
3     | "sleep" "(" Exp ")" ";" [strict(1)]
4     | "send" Exp "," ExtId ";" [macro]
5     | "send" Exp "," ExtId "," "(" Exps ")" ";" [seqstrict(1, 3)]
6     | "broadcast" Id ";" [macro]
7     | "broadcast" Id "," "(" Exps ")" ";" [strict(2)]
8     | "goto" Id ";" [macro]
9     | "goto" Id "(" Exps ")" ";" [strict(2)]
10    | "print" "(" Exp ")" ";" [strict]
11    > "return" ";" [macro]
12    | "return" Exp ";" [strict(1)]
13    | "var" Id "=" Exp ";" [macro]
14    > Exp "=" Exp ";" [strict(2)]
15    > "var" Id ";"
16    | "vars" Ids ";" [macro-rec]
17    | Block
18    > "if" "(" Exp ")" Block [strict(1)]
19    | "if" "(" Exp ")" Block "else" Block [strict(1)]
20    | "while" "(" Exp ")" Block
21    | "entry" Block [macro]
22    | "entry" "(" Ids ")" Block
23    | "on" ExtId "do" Block [macro]
24    | "on" ExtId "(" Ids ")" "do" Block
25    | "fun" Id "(" Ids ")" Block
26    | NonDetStmt
27    | Exp "in" "{" CaseDecl "}" [macro-rec]
28    | StateDecl
29    > "machine" Id Block [macro]
30    | "machine" Id "receives" Ids Block
31    | "interface" Id Block [macro]
32    | "interface" Id "receives" Ids Block
33    | "init" "machine" Id Block [macro]
34    | "init" "machine" Id "receives" Ids Block
35    | "yield" ";"
36    > Stmt Stmt [right]

```

Listing 9.3: MediK Statement Syntax

programming languages. For instance, Lines 5-9 enable one to write expressions using program identifiers (denoted by the builtin $\mathbb{K}$ production `Id`), and values such as booleans, or rationals, or “this”, which enables an instance to refer to itself. Line 13 defines the usual dot operator (`.`), which can be used to access members of an instance. Lines 14-25 declare common expressions such as `+`, `-`, `>`, `>=` over rationals and `&&`, `||`, `!` over

booleans. We use the production `StandaloneExp` to define certain expressions that are typically not used in conjunction with other expressions, such as `new (...)` on [Line 2](#) that resembles constructs in OOP languages used to create object instances. Note however, that MediK also has several constructs not commonly found in other languages, such as `createFromInterface(...)` ([Line 3](#)) and `obtainFrom(...)` ([Line 11](#)). We shall describe their need and purpose in upcoming sections.

As reflected in [Listing 9.3](#), MediK support many statements commonly found in conventional programming languages, such as variable assignment ([Line 14](#)), `if-else` (Lines [18](#) and [19](#)) and `while` ([Line 20](#)). Others, such as `broadcast` (Lines [6](#) and [7](#)), `goto` and `interface` declarations have nuanced meanings, and will be explained in upcoming sections.

Next, we use the $\mathbb{K}$ definition to describe MediK's semantics. Recall the semantics in $\mathbb{K}$ has two components:

- (1) Description of program state expressed as a configuration.
- (2) Rewrite rules over configuration segments that define the program's transition system.

We discuss MediK's configuration in [section 9.4](#), and corresponding semantics rules in [section 9.5](#).

9.4 MediK CONFIGURATION

Recall that $\mathbb{K}$ represents program execution state using $\mathbb{K}$ -configurations. A $\mathbb{K}$ -configuration is an unordered list of (potentially nested) *cells*, specified using an XML-like notation. When declaring rules (as rewrites) over this state, any subset of the cells present in the configuration can be mentioned. This allows specifying only necessary parts of the state for a given rule, letting $\mathbb{K}$ assume that the rest of the configuration remains unchanged. The configuration in [Listing 9.4](#) defines the initial state of any MediK program. For brevity, we only show a part of the configuration to illustrate organization of data in the semantics.

Recall that the keyword `configuration` ([Line 2](#)) defines a $\mathbb{K}$ -configuration, followed by xml-like notation for the $\mathbb{K}$ -cells. For example, `<foo> ... </foo>` corresponds to a $\mathbb{K}$ -cell with the name `foo`.

A MediK program consists of instances of concurrently executing state machines that communicate by passing events, where every instance manages its own environment. This is clearly reflected in MediK's configuration in [Listing 9.4](#). The configuration consists of the following cells, each with a similar composition:

- (a) An `<instances>` cell (Lines [2-12](#)) that encapsulates `<instance>` cells.

(b) A `<machines>` cell (Lines 13-33) that holds `<machine>` cells.

(c) An `<instances>` cell (Lines 34-40) that holds `<interface>` cells.

The sub-cells under each of the aforementioned cells the attribute `multiplicity=*` (Lines

```
1 configuration
2   <instances>
3     <instance multiplicity="*" type="Map">
4       <id> 0 </id>
5       <k> populateCells($PGM:Stmt) ~> createInitInstances </k>
6       <env> .Map </env>
7       <genv> .Map </genv>
8       <activeState> . </activeState>
9       <callerId> . </callerId>
10      <inBuffer> .List </inBuffer>
11    </instance>
12  </instances>
13  <machines>
14    <machine multiplicity="*" type="Map">
15      <machineName> . </machineName>
16      <declarationCode> nothing; </declarationCode>
17      <isInitMachine> false </isInitMachine>
18      <receiveEvents> .Set </receiveEvents>
19      <states>
20        <state multiplicity="*" type="Map">
21          <stateName> . </stateName>
22          <stateDeclarations> nothing; </stateDeclarations>
23          <entryBlock> nothing; </entryBlock>
24          <eventHandlers>
25            <eventHandler multiplicity="*" type="Set">
26              <eventId> . </eventId>
27              <handlerCode> nothing; </handlerCode>
28            </eventHandler>
29          </eventHandlers>
30        </state>
31      </states>
32    </machine>
33  </machines>
34  <interfaces>
35    <interface multiplicity="*" type="Map">
36      <interfaceName> . </interfaceName>
37      <interfaceDeclarations> nothing; </interfaceDeclarations>
38      <interfaceReceiveEvents> .Set </interfaceReceiveEvents>
39    </interface>
40  </interfaces>
41  <executorAvailable> true </executorAvailable>
42  <stuck> false </stuck>
43  <epoch> 0 </epoch>
44  <shouldAdvanceEpoch> false </shouldAdvanceEpoch>
```

Listing 9.4: MediK Configuration

3, 14, 35), denoting that zero-or-more copies of the cell may exist in the configuration. We briefly describe the purpose of each such sub-cell.

`<instance>` cells (Lines 3-11) hold the state of each MediK machine instance during execution. As is the case with instances in OOP languages, each instance manages its instance variables using a map in the `<env>` cell (Line 7). Additionally, each instance maintains a buffer of incoming events in the `<inBuffer>` cell (Line 10) and the currently executing code in the `<k>` cell (Lines 5). Note that unlike the example single-threaded language in subsection 5.2.1, MediK is inherently concurrent, as reflected by the presence of multiple `<k>` cells. At runtime, if multiple rewrite rules can apply on the configuration, $\mathbb{K}$ non-deterministically chooses a rule and applies it, allowing one to specify concurrency through interleavings. However, for MediK, we utilize a structured scheduling algorithm, which will be discussed in upcoming sections.

The `<machine>` cell (Lines 14-32) holds information relevant to a machine definition, such as the name in the `<machineName>` cell (Line 15) and states in the `<states>` cell (Lines 19-31). The `<state>` cell holds information relevant to a state, such as the entry block in cell `<entryBlock>` (Line 23) and event handlers in cell `<eventHandlers>` (Lines 25-28). Similarly, an `<interface>` cell holds information regarding interface definitions. In MediK, interfaces facilitate communication with the external world and will be discussed in upcoming sections.

Recall that on execution, $\mathbb{K}$ replaces `$PGM` (Line 5) with the Abstract Syntax Tree (AST) of the program, obtained by parsing the program using the syntax defined in section 9.3. In case of a MediK program, $\mathbb{K}$ adds a `<instance>` cell that has a `<k>` cell containing `populateCells($PGM) ~> createInitInstances` to the initial configuration. This instance, also referred to as the *implicit* or *initial*, is responsible for populating the configuration with relevant information from the program AST before execution can begin. The `populateCells(...)` construct at the top of the `<k>` cell of the implicit instance is defined (using rewrite rules) to recursively traverse the program AST translate each `machine` in the MediK program to a corresponding `<machine> ... </machine>` sub-configuration. Next, the `createInitInstances` creates an instance for the machine with the `init` keyword and schedules its `entry` block for execution. Recall that `~>` symbol (Line 5) is interpreted by $\mathbb{K}$ as “followed-by”, i.e., execution of `populateCells` is followed by execution of `createInitInstances`. During execution, as instances are created using the `new` keyword, the configuration grows dynamically to accommodate them. In MediK, interaction with the external world is handled through `interface` definitions that make up `<interface>` under the `<interfaces>` cell (Lines 34-40). Interface definitions will be discussed in detail in upcoming sections. Also note cells `<executorAvailable>`, `<epoch>`, `<stuck>` and `<shouldAdvanceEpoch>` (Lines 41-44). These cells are utilized in implementing the scheduling strategy MediK uses that shall be discussed

in subsequent sections.

9.5 RULES

We briefly recall $\mathbb{K}$ rules from [section 5.2.1](#). A rule begins with the keyword `rule`, and is a statement of the form $\varphi \Rightarrow \psi$, where φ and ψ are patterns over configuration terms and $\mathbb{K}$ -variables. We say φ is the LHS and ψ is the RHS of the rule. Let substitution θ be a map from $\mathbb{K}$ -variables to terms. Say, for given pattern φ and substitution θ , $\varphi\theta$ be the term obtained by replacing each variable v in φ with $\theta(v)$. During execution, if the current configuration C , i.e. program execution state, matches φ with substitution θ , then it is rewritten to $\psi\theta$. We say pattern φ matches configuration C iff there exists a substitution θ s.t. $C = \varphi\theta$. For example, consider the rule in [Listing 9.5](#) for updating the value of a local program variable.

```
rule <k> I:Id = V:Val => V ... </k>
      <env> (I |-> Loc) ... </env>
      <store> Store => Store[Loc <- V] </store>
```

Listing 9.5: Variable Assignment

In the rule, `I`, `V`, `Loc`, and `Store` are $\mathbb{K}$ -variables. Note the distinction between program variables and $\mathbb{K}$ -variables: while program variables are simply identifiers, $\mathbb{K}$ -variables have logical meaning. The `...` is used to denote parts of the configuration not relevant to the rule. Typically, the top of the `<k>` cell contains the statement currently being executed. Suppose we're executing the statement `i = 2;`. In this case, the current configuration will have a `<k>` cell of the form `<k> i = 2 ... </k>`, an environment cell `env` where variable `i` maps to some pointer pt , and cell `<store>` containing a map ρ with some value pointed-to by pt . The LHS matches with substitution $\theta = (I \mapsto i, V \mapsto 2, Loc \mapsto pt, Store \mapsto \rho)$, resulting in the top of the `<k>` cell to be rewritten to the value 2, and pointer pt updated to point to 2 in ρ . Execution is a sequence of rule applications that continues until no rule can match the current configuration.

Next, we explain several features of Medi $\mathbb{K}$ relevant to defining BPGs using their corresponding $\mathbb{K}$ -rules. Note that we only discuss rules related to idiosyncrasies of Medi $\mathbb{K}$. Other rules, that implement statement that resemble other definitions are omitted for brevity. For instance, the rules for Arithmetic Expressions and boolean expressions, that resemble counterparts in other languages are ignored.

9.5.1 Instance Creation

In MediK, machine definitions are analogous to OOP classes, and instances created using keyword `new` behave as objects. The rule in [Listing 9.6](#) depicts the semantics of the `new` operator. The rule will match any `<k>` cell that has `new` at the top of the cell and rewrite it to a temporary `wait` construct ([Line 2](#)). Note that the same $\mathbb{K}$ variable, `MName`, is used at the following places in the rule’s LHS:

- On [Line 2](#) to match against the name of the machine whose instance is being created.
- On [Line 12](#) to identify the `<machine>` (Lines [11-18](#)) with the same name as the one being initialized.

```
1 rule <id> SourceId </id>
2   <k> new MName (Args) => wait ... </k>
3   ( .Bag => <instance>
4     <id> Loc </id>
5     <k> MachineDecls ~> enterState(InitState | Args ) </k>
6     <callerId> SourceId </callerId>
7     <class> MName </class> ...
8     </instance> )
9   <nextLoc> Loc => Loc +Int 1 </nextLoc>
10  <store> ( .Map => (Loc |-> instance(Loc))) ... </store>
11  <machine>
12    <machineName> MName </machineName>
13    <declarationCode> MachineDecls </declarationCode>
14    <state>
15      <stateName> InitState </stateName>
16      <isInitState> true </isInitState> ...
17    </state> ...
18  </machine>
19  <activeInstances> ... (.Set => SetItem(Loc)) </activeInstances>
```

Listing 9.6: New Instance Creation

By matching a `<machineName>` cell that has the same name as the one being using `new` on [Line 2](#), appropriate instance variable and the state marked with the `init` keyword can be determined by matching against the contents of the `<declarationCode>` and `<state>` cells, as shown on [Line 13](#) and Lines [14-17](#) respectively. Finally, a new `<instance>` cell can be added to the configuration (Lines [3-8](#)) containing:

- An `<id>` cell containing a unique identifier ([Line 4](#)) that is also added to the set of active instances ([Line 19](#)).

- A `<k>` cell (Line 5) containing code to initialize instance variables and enter the initial state, obtained by matching on appropriate parts of the `<machine>` cell as described earlier.

Note that when `new` is executed, control is transferred from the caller, i.e., the original instance, to the callee, i.e., the created instance. Once the entry block of the callee completes execution, control is transferred back to the caller along with a *reference* to the newly created instance. We chose this behavior for the `new` operator to maintain consistency with most OOP languages where an object is considered instantiated and ready for use only after all relevant constructors have been executed. While MediK machines do not have direct analogues to OOP constructors, we treat the `entry` blocks of initial states as such, and typically place code commonly found in constructors, such as initialization of instance variables, under them. Upcoming sections shall discuss MediK’s scheduling semantics, and our motivations behind them in greater depth.

9.5.2 Sending and Receiving Messages

MediK machines communicating by passing messages. This is facilitated by the `send` statement that enables a machine to deliver an event to a particular machine through a reference to the receiver machine. This is captured by the rule in Listing 9.7.

```

1 rule <instance>
2   <k> send instance(RecvId) , EventName:Id , ( Args )
3   => done ...
4   </k>
5   ...
6 </instance>
7 <instance>
8 <id> RecvId </id>
9 <inBuffer> ...
10  (.List => ListItem( eventArgsPair(EventName | Args | Epoch + 1)))
11 </inBuffer>
12  ...
13 </instance>
14 <epoch> Epoch </epoch>

```

Listing 9.7: MediK Send Statement Semantics

When an instance the top of the `<k>` cell has `send`, the rule:

- Obtains the `id` of the receiver instance, the event name and the event arguments by matching the variables `RecvId`, `EventName` and `Args` against the current configuration (Lines 2-4) respectively.

- (ii) Rewrites the top of the `k` cell to `done`, marking the completion of execution for the construct.
- (iii) Adds the event and associated arguments to the buffer of incoming events of the instance with `id RecvId` (Lines 7-10).
- (iv) The *epoch* decides when the machine can run and is discussed in [section 9.6](#).

While the `send` construct enables communication between pairs of machines, MediK also supports *multicast* communication, that enables a machine to send to multiple machines at the same time. The `broadcast` construct that facilitates this is described in [Listing 9.8](#). Recall from [Listing 9.1](#) ([Line 2](#)) that a machine specifies the list of events it accepts as part of its definition, which we refer to as the “receives-list” of the machine. To define the semantics of `broadcast` for some event `EventName` ([Line 1](#)), we utilize a helper function symbol `getReceivers` ([Line 2](#)), to obtain all machines that declare `EventName` in their “receives-lists”, and use `send` to add the `EventName` to each machine’s `<inputBuffer>` ([Line 6](#)).

```

1 rule broadcast EventName:Id , ( Args ) ;
2   => performBroadcast ( EventName | Args | getReceivers(EventName))
3
4 rule performBroadcast ( EventName | Args | ListItem(Id) Recievers)
5   =>   send instance(Id), EventName, ( Args ) ;
6       ~> performBroadcast ( EventName | Args | Recievers)
7
8 rule performBroadcast(_ | _ | .List) => .

```

Listing 9.8: Broadcast Mechanism

9.5.3 External Agents

Recall from [section 9.1](#) that CDSSs need to interact with heterogeneous external agents, such as sensors and monitors for various patient parameters and EHR systems. Ideally, we would like a MediK program to be agnostic to specific details of external agents it interacts with, and for machines to utilize a *uniform* communication mechanism that serves both external and internal communication.

We achieve this by treating all external agents as state machines with transition systems defined outside the MediK program, like other languages, such as Object-Oriented Maude, [\[190\]](#). These external agents are modeled in MediK as **interfaces**. We define an interface as a state machine that has its transition system defined externally. In case of CDSSs, it is

```

1 interface HeartRateSensor { }
2
3 machine TreatmentMachine { ...
4     var hrSensor = createFromInterface(HeartRateSensor, "heartRateSensor");
5     var heartRate = obtainFrom(hrSensor, "heartRate");
6 }

```

Listing 9.9: MediK External Agents

common to utilize external sensors to obtain various patient parameters. Listing 9.9 shows such as example, where data is obtained from an external Heart Rate sensor.

Since we don't have the transition system for the heart rate sensor, we declare it as an interface (Line 1). Next, instead of using `new` to create an instance, we use a builtin MediK construct `createFromInterface`, which takes as arguments (a) the interface name (b) a unique identifier string used to identify the instance outside the MediK process Line 4. All other MediK machines can interact with external sensor using variable `hrSensor` (Line 5). There is no need to make any distinction between external, and MediK-based machines. To deal with external interactions, input and output pipes are provided to the MediK process at launch. When the `send` construct is used on an external machine, MediK will write a JSON [191] message with the event data, the identifier from Line 4, and a unique transaction id to the *write-end* of the output pipe. At the *read-end*, we need to write external code (in any programming language) to handle the JSON message. In the case of the example in Listing 9.9, this involves reading from an external heart rate sensor and sending the data back to the MediK process that made the request. To send data to MediK, a JSON message in a pre-specified format that also contains the unique transaction id of the request needs to be written to the *write-end* of the input pipe. Note that the `obtainFrom` construct implements a *request-response* mechanism, where a request initiated from MediK is responded to by an external agent. But external agents can also initiate communicate using the `broadcast` construct described in subsection 9.5.2.

9.5.4 Tables

Recall from section 9.1 that tables indicating normality/abnormality for parameters indexed by age and weight occur commonly in BPGs. For instance, in case of the sepsis BPG from section 9.1, once a measurement, such as the heart rate has been obtained from a sensor, we need to use a table, such as Table 2.1 to check if the measurement is within a normal range. In MediK, we can write a function that does the required check, as shown in Listing 9.10. While the table in Listing 9.10 could have also been implemented using simple `if-else`

```

1 fun isHeartRateNormal() {
2   days(age) in {
3     interval(days(0) , months(1)): return hr > 205;
4     interval(months(1), months(3)): return hr > 205;
5     // omitting other cases
6     default                      : return hr > 100;
7   }
8 }

```

Listing 9.10: Vital Signs Tables

constructs, we chose to present data in form of a table to improve overall readability and HCP comprehensibility, as HCPs are already familiar with them from paper-based guidelines. If the heart rate lies any of the intervals (closed on the left, open on the right) on Lines 3-4, the corresponding statement to the right of the colon (:) is run. Otherwise, the default clause (Line 6) is run. We can then define the semantics of tables in terms of **if-else** as shown in Listing 9.11.

```

1 rule E in interval(L, R) => (E >= L) && (E < R)
2 rule E in { interval(L, U): S:Stmt Cs:CaseDecl }
3   => if ((E >= L) && (E < U)) { S } else { E in { Cs } }
4 rule E in { interval(L, U): S:Stmt }
5   => if ((E >= L) && (E < U)) { S }
6 rule E in { interval(L, U): S1:Stmt default: S2:Stmt }
7   => if ((E >= L) && (E < U)) { S1 } else { S2 }

```

Listing 9.11: Implementing Tables

Note the syntax declarations for tables on Line 27 of Listing 9.2 and Line 27 of Listing 9.3 have the attributes `[macro]` and `[macro-rec]` respectively. These attributes specify that the corresponding constructs are merely syntactic sugar, and rules written over such constructs “de-sugar” them into other symbols. Rules that operate over symbols with attribute `[macro]` are applied statically at parse time, before execution begins, allowing the productions to be used as convenient syntactic sugar.

9.6 MediK SCHEDULING SEMANTICS

Since the $\mathbb{K}$ -generated interpreter is single-threaded, MediK employs interleaving-semantics for concurrency, using a single **executor** thread shared between machine instances. A machine instance that is either at the start of an entry block or has an event in the input buffer that

Algorithm 9.1: MediK Scheduling Semantics

```
1  $epoch \leftarrow 0$ 
2  $scheduled \leftarrow \{\mathcal{I}_{\mathcal{M}_0,0}^0\}$ 
3 while  $scheduled \neq \emptyset$  do
4    $\mathcal{I}_{\mathcal{M}_i,j}^\tau \leftarrow \text{choose}(scheduled) \text{ s.t. } \tau \leq epoch \wedge \text{enabled}(\mathcal{I}_{\mathcal{M}_i,j}^\tau)$ 
5    $\text{execute}(\mathcal{I}_{\mathcal{M}_i,j}^\tau, scheduled)$ 
6   if  $\nexists i', j', \tau' \text{ s.t. } (\mathcal{I}_{\mathcal{M}_{i'},j'}^{\tau'} \in scheduled) \wedge (\tau' \leq epoch) \wedge (\text{enabled}(\mathcal{I}_{\mathcal{M}_{i'},j'}^{\tau'}))$  then
7      $epoch \leftarrow epoch + 1$ 
8   end
9 end
```

it can handle is said to be *enabled*, i.e. one that can run once the **executor** becomes available. Recall from [section 5.2.1](#) that if multiple $\mathbb{K}$ rules can match the current configuration, then for execution, $\mathbb{K}$ will choose one non-deterministically. This, in case of a program comprising of concurrent MediK machines may lead to *unfairness*. Specifically, there may be situations where a machine instance is *enabled* but is never chosen for execution. Therefore, to ensure fairness, we use a scheduling strategy based on a monotonically increasing global counter called the *epoch*. Instead of just tracking *enabled* machines, we also track the *epoch* value at which said machines became enabled. We show this execution strategy in [algorithm 9.1](#).

Recall from [section 9.3](#) that a MediK program consists of a set of machines, of which one, prefixed with the keyword **init**, is the *initial* machine. Each machine has one state prefixed with **init**, referred to as the *initial* state. Let $P = \{\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_{n-1}\}$ be a program with n machines, where $\mathcal{M}_0$ is prefixed with **init**. MediK allows instances of a machine to be created dynamically at runtime. For machine $\mathcal{M}_i \in P$, let $\mathcal{I}_{\mathcal{M}_i,j-1}$ be its j -th instance.

Execution begins in *epoch* zero with the implicit (first) instance of the initial machine, denoted by $\mathcal{I}_{\mathcal{M}_0,0}^0$. We use $\mathcal{I}_{\mathcal{M}_i,j-1}^\tau$ to say that the j -th instance of machine $\mathcal{M}_i$ is scheduled for execution in *epoch* τ . Recall that a state definition may have an entry block, containing code that is executed when the state is entered, or event handlers containing code that is executed when an event is dequeued from the input buffer. When execution begins, the entry block of the *initial state* of the *implicit instance* of the *initial machine* $\mathcal{I}_{\mathcal{M}_0,0}$ becomes scheduled ([line 2](#)) at *epoch* 0. On [line 4](#), an instance $\mathcal{I}_{\mathcal{M}_i,j}^\tau$ is non-deterministically chosen from all machines that are both scheduled to run when $\tau \leq epoch$ and *enabled*. We use $\text{execute}(\mathcal{I}_{\mathcal{M}_i,j}^\tau, scheduled)$ on [line 5](#) to denote this execution process. Execution of the entry or event handler block is atomic, i.e., a context-switch can only occur at the end of the block. Note that when a new instance of a machine is created using the keyword **new**, the *entry* block of the *initial* state of the *target* machine is executed synchronously before control returns

to the source machine, and the instance is added to the multiset of *scheduled* machines. A context switch only occurs in three cases: `goto`, `sleep`, and `obtainFrom`, which we describe later.

During execution, if an instance $\mathcal{I}_{\mathcal{M}_i,j}$ sends an event to another instance $\mathcal{I}_{\mathcal{M}_{i'},j'}$, then the event is scheduled to be handled by $\mathcal{I}_{\mathcal{M}_{i'},j'}$ in or after the next epoch, i.e., $scheduled \leftarrow scheduled \cup \{\mathcal{I}_{\mathcal{M}_{i'},j'}^{epoch+1}\}$. Similarly, if a `goto` statement is encountered, the entry block of the target state is scheduled for execution at $epoch + 1$. If no other machine is both *scheduled* to run in the current *epoch*, and *enabled*, then the *epoch* advances by one (line 7).

9.7 TIMER SEMANTICS

Next, we discuss how MediK handles temporal aspects of BPGs. For instance, consider the Fluid resuscitation guideline BPG from Figure 2.2. After administering fluids, the BPG recommends waiting for 15 minutes before evaluating their effectiveness. This waiting behavior in MediK is implemented using a `sleep(duration)` statement. Formalizing the execution semantics of such a statement in $\mathbb{K}$ presents a challenge as $\mathbb{K}$ does not provide builtin support for timers. Therefore, in MediK, `sleep(duration)` is described by the rule in Listing 9.12.

```

1 rule <k> sleep(Duration: Int) ;
2     =>    jsonWrite( { "action"      : "sleep"
3                      , "duration"   : Duration
4                      , "tid"        : TId }
5                      , ... )
6     ~> releaseExecutor
7     ~> waitForSleepResponse(TId) ...
8 </k>
9 <tidCount> TId => TId +Int 1 </tidCount>

```

Listing 9.12: Semantics of `sleep`

The `sleep` statement results in a JSON message being sent to a remote endpoint (Lines 2-5) specified when the MediK process is launched, mimicing sending an event to an external *timer* machine, with the desired duration as the payload. At the remote endpoint, code must be provided (in any programming language) to parse the message and respond with a JSON message indicating the expiration of the timer once the desired duration has passed. A unique transaction-id (Lines 4, 7 and 9), which the code at the endpoint is expected to provide in the response, uniquely identifies the machine instance being responded to. `sleep` causes a

```

1 rule <k> waitForSleepResponse(TId) => . ... </k>
2   <inBuffer>
3     (ListItem(event($SleepDone | TId | Tau )) => .List) ...
4   </inBuffer>
5   <executorAvailable> true => false </executorAvailable>
6   <epoch> Epoch </epoch>
7   requires Tau <=Int Epoch

```

Listing 9.13: Sleep Completion

context-switch to occur on [Line 6](#), releasing the executor lock to process other *scheduled* machines.

When a message signifying the expiration of the timer is sent to the MediK process, along with the transaction id of source instance, the corresponding event signaling the completion of the sleep statement is placed at the *beginning* of the source machine instance’s input buffer, and the instance is scheduled to resume execution in the next epoch. The rule that handles the external response is shown in [Listing 9.13](#). The `waitForSleepResponse(TId)` ([Line 1](#)) blocks execution until the external response indicating the expiration of the sleep timer is received in the input buffer ([Line 2-4](#)). Once the response is received, the machine instance resumes execution when (a) the execution lock becomes available (indicated by `true` on [Line 5](#)), and, (b) the epoch the instance was scheduled to resume execution in ([Line 3](#)) is less than or equal to the current epoch ([Lines 6-7](#)).

An `obtainFrom` statement also results in a context switch as shown in [Listing 9.14](#). Just as in the case of sleep, a JSON message is sent to the remote endpoint with the *name* of the parameter being requested and a unique transaction ([Lines 2-3](#)), while the machine instance releases the execution lock ([Line 4](#)), and waits for a response, as shown in [Listing 9.14](#).

```

1 rule <k> obtainFrom(instance(Id:Int), Field:String)
2   =>   jsonWrite( { "name"           : "Obtain"
3               , "args"           : [ Field , .JSONs ] } ...)
4   ~> releaseExecutor
5   ~> waitForObtainResponse(TId) ...
6   </k>
7   <tidCount> TId => TId +Int 1 </tidCount>

```

Listing 9.14: Obtain From Request

As depicted in [Listing 9.15](#), once data for the requested field is available, it’s communicated as an event to the MediK process as a JSON message, where the response consists of a JSON message with the requested data and the unique `transactionId`, placed at the *head* of the

<inBuffer> cell and scheduled for execution in the next epoch (Lines 2-6).

```

1  rule <k> waitForObtainResponse(_) => V ... </k>
2    <inBuffer>
3      (ListItem(event($ObtainResponse | V:Val | Epoch ))
4        => .List)
5      ...
6    </inBuffer>
7    <executorAvailable> true => false </executorAvailable>
8    <epoch> Epoch </epoch>

```

Listing 9.15: Obtain From Response

9.8 MediK PEDIATRIC SEPSIS SYSTEM

9.8.1 Sepsis Management CDSS

To evaluate our approach, we collaborated with the Children’s Hospital of Illinois at OSF St. Francis Medical Center to develop a MediK-based CDSS for screening and management of Pediatric Sepsis ³.

Recall from Figure 2.1 the guideline for sepsis screening. In Listing 9.16, we show MediK code corresponding to the sepsis screening guideline. When modeled in MediK, a flowchart in the guideline is represented using a MediK machine. *Nodes* in the flowchart are represented as *states* in a MediK machine, while flowchart *edges* as *state-transitions*. Note that we use *node* to refer to constructs in the flowchart, and *state* to refer to counterparts in MediK. Also, while it’s desirable to represent each flowchart *node* as a state machine *state*, the task in the flowchart *node* may warrant using multiple state-machine *states*. For example, in Figure 2.1, the step “Obtain Patient Age, Weight, and High Risk Conditions” is translated to states `ObtainAge` (Lines 7-13), `ObtainWeight` (Line 14), and `ObtainHighRiskConditions` (Line 15) in Listing 9.16. Within each of these states, the code permits communication with heterogeneous external agents for obtaining required parameters. For instance, on Line 9, an `Instruct` event is sent to an external `tablet` machine with the payload “get age”. The recipient process runs on a tablet held by the Healthcare Practitioner and handles the event by prompting the provider to enter the patient’s age. A `ConfirmAgeEntered` event, emitted once the age is obtained, enables the screening machine to proceed to the next step. Once all appropriate measurements have been obtained, they are checked for abnormality (Lines 16-26) using tables shown in to arrive upon a diagnosis.

³the entire CDSS for sepsis management is available at [192].

```

1 machine SepsisScreening receives .. {
2   init state Start {
3     on StartScreening do {
4       goto ObtainAge;
5     }
6   }
7   state ObtainAge {
8     entry {
9       send tablet, Instruct, ("get age");
10    } on ConfirmAgeEntered do {
11      goto ObtainWeight;
12    }
13  }
14  state ObtainWeight { ... }
15  state ObtainHighRiskConditions { ... }
16  state CalculateScore {
17    var hrAbnormal = !isInNormalRange("HR", ...);
18    var bucket1    = hrAbnormal || ...
19    var bucket3    = mentalStatusAbnormal || ...
20
21    var sepsisSuspected
22      = bucket1 && bucket2 && bucket3;
23
24    send tablet, SepsisDiagnosis
25      , (sepsisSuspected);
26  }
27 }

```

Listing 9.16: Sepsis Screening in MediK

Apart from structurally resembling the workflow, the code permits communication with heterogenous external agents, and *obtains* patient parameters such as age and weight, and uses them to evaluate the patient for sepsis. For instance, on [Line 9](#), we send an event to the external machine that is the tablet held by the HCP, to obtain the patient’s age. Once the age is entered, an event `ConfirmAgeEntered` is sent to the machine, prompting the machine to move on to collecting the patient’s weight. When evaluating whether the patient meets the criteria for sepsis, we use functions, such as the one on [Line 17](#), that obtain the value from a sensor, and check whether it lies in acceptable ranges using tables indexed by the patient’s age and weight, similar to the paper-based BPG. Treatment machines *receive* events from the screening machine, and are triggered if the diagnosis is *sepsis positive*, suggesting commencement of treatment.

Recall from [section 2.1](#) that once a sepsis diagnosis has been arrived upon, one of the

guideline suggested actions include administering fluids as shown in [Figure 2.2](#). In [Listing 9.17](#),

```
1 machine FluidTherapy
2   receives StartFluidTherapy, ... {
3
4     init state Start {
5       on StartFluidTherapy do {
6         goto ObtainRisks;
7       }
8     }
9     state ObtainRisks {
10      // Obtain fluid overload related risks
11    }
12    state SuggestFluidDosage {
13      // Suggest a dosage based on risks
14    }
15    state WaitForAdministerFluidConfirmation {
16      // Handler for Normal Saline Administration
17      on ConfirmNormalSalineAdministered do {
18        sleep(900);
19        goto EvaluateResponsiveness;
20      }
21    }
22    state EvaluateResponsiveness {
23      entry {
24        send tablet
25          , Instruct
26          , ("get responsiveness to fluids");
27      }
28      on FluidResponsivenessEntered(responsiveness) do {
29        isResponsiveToFluids = responsiveness;
30        goto ObtainFluidOverloadSigns;
31      }
32    }
33    state ObtainFluidOverloadSigns {
34      // Obtain signs of fluid overload
35    }
36    state AskNextStep {
37      entry {
38        var recommendation;
39        if (this.fluidOverload) {
40          recommendation = "handle fluid overload";
41        } else {
42          // obtain total saline dose
43          if ((totalSalineDose >
44            measurementBounds.salineDosageUpperBound) {
45            if (isResponseiveToFluids) {
46              recommendation = "maintainence fluids"
47            } else {
48              recommendation = "consider inotropic support";
49              broadcast ConsiderInotropicSupport;
50            }
51          } else {
52            recommendation = "repeat fluid bolus";
53          }
54        }
55        // Send recommendation to tablet
56        // Wait for HCP response
57      }
58    }
59  }
```

Listing 9.17: Fluid Therapy Guidelines

we show the corresponding MediK code for administering fluids. The process starts when an external `StartFluidTherapy` event, corresponding to a button press by the HCP is received (Line 5). The next steps include:

- (a) Obtaining any *risks* associated with administering fluids (Lines 9-11).
- (b) Suggesting an appropriate *dose* to administer based on the risks, if any (Lines 12-14).
- (c) Waiting for the HCP to confirm that the suggested dose was administered (Lines 15-21).

Once the dose is administered, the machine waits for the for 15 minutes as specified by the guideline (Line 18), before prompting the HCP to evaluate the patient’s responsiveness to the administered fluid dose (Lines 23-31), and check for any signs of evaluate overload (Lines 33-35). If the patient exhibits any signs of fluid overload, then a recommendation to handle the overload is made (Line 40). Otherwise, the total dose of administered fluid is obtained from an external source (Line 42). If the total dose is above the maximum allowed dose, then a recommendation based on the patient’s responsiveness to administered fluids is made to either (a) reduce the fluid flow to maintenance levels (Line 46), or, (b) switch to inotropic support to address circulatory issues is made (Lines 47-50). If the total dose of administered fluids is less than the maximum allowed limit, then a recommendation to administer one more fluid bolus made (Lines 51-53)).

Note that both the `SepsisScreening` and `FluidTherapy` machines structurally resemble their paper based counterparts in Figure 2.1 and Figure 2.2 respectively, making it easier for Healthcare Providers to comprehend and validate the code.

9.8.2 Formal Analysis using MediK

During execution of a MediK program, a machine may be considered *stuck* if an event at the head of its input buffer does not have an associated handler, rendering said machine non-responsive. For this reason, languages for modeling large concurrent systems, such as P [49] raise an exception for unhandled events. To mitigate such exceptions, we can enforce every machine to define event handlers for all possible events in all states, and use static analysis to detect possible violations. But, for MediK programs, we found that for complex CDSSs, such as the one for screening and management of sepsis: (a) it’s tedious and error prone to define handlers for every event in every state, and, (b) it reduces the comprehensibility of the program, as many spurious event handlers that may never fire during execution have to be specified.

Thus, for MediK, we employ a weaker notion of responsiveness. We verify that every event that a state may possibly receive during execution must have a handler defined for it. This presents a challenge for reactive systems, or systems involve interactions with the external world, such as MediK-based CDSSs, as exploring the system’s state space requires modeling the external components. In MediK, we address this by specifying external components as *ghost* machines - a technique also used by other state machine formalisms such as P [49]. For program analysis, *ghost* machines substitute external agents, permitting exploration of the state space. During execution, *ghosts* are discarded and replaced by actual external agents. Due to this, ghost machines may have statements to express non-determinism in processes. Consider, for instance, on a positive sepsis diagnosis, an HCP may choose to either administer fluids first, followed by antibiotics, or vice-versa. MediK supports such non-determinism using *either-or* statements as shown in Listing 9.18.

```

1 either {
2   broadcast StartFluidTherapy;
3   broadcast StartAntibioticTherapy;
4 } or {
5   broadcast StartAntibioticTherapy;
6   broadcast StartFluidTherapy;
7 }

```

Listing 9.18: Modeling Non-Determinism

When writing ghosts, values of measurements need to be abstract, to encompass all possible values that may be encountered during execution. For instance, when modeling entering a parameter such as the Heart Rate, we need to use an abstract value, representing all possible concrete values. To this end, we allow using an abstract value `#nondet` in ghost machines, that have semantics shown in Listing 9.19.

```

1 rule #nondet + _ : Val => #nondet
2 rule _ : Val <= #nondet => #nondet
3 rule #nondet && _ => #nondet
4 rule if (#nondet) Block => Block
5 rule if (#nondet) _ => .

```

Listing 9.19: Abstract Semantics

The use of abstract encodings leads to a reduction of the state space. Recall from Section 2.1 that we needed to: (1) utilize patient’s basic information such as age and weight to calculate normal ranges for clinical measurements such as blood pressure and heart rate, and, (2) calculate abnormality in clinical measurements using aforementioned ranges. For example,

determining whether the patient’s heart rate is abnormal is performed using the `in-interval` construct as shown in [Listing 9.10](#). Recall that the `in-interval` construct is merely syntactic sugar for nested `if-else` statements. When using ghost machines for model checking, since the actual measurement is an abstract value, we know the result of this abnormality checking operation is an abstract boolean value. Thus, instead of exploring each branch of `if-else` statements corresponding to `in-interval` constructs in [Listing 9.10](#), we replace the entire checking process with a final abstract boolean value. This reduces the state space but still allows us to explore all treatment options for both the normal and abnormal cases.

9.8.3 Model Checking the Sepsis CDSS

To verify responsiveness of the Sepsis CDSS, we implemented ghost machines for the external components using support for non-determinism and abstract values. We then added the rule in [Listing 9.20](#), that takes a machine in an active state with an unhandled event at the head of the input buffer to a terminal `stuck` state.

```

1 rule <k> handleEvents ~> _ => stuck </k>
2   <activeState> ActiveState </activeState>
3   <class> MachineName </class>
4   <inBuffer>
5     ListItem(event(InputEvent | _ | _ )) ...
6   </inBuffer>

```

Listing 9.20: Unresponsive Machine Transition

We utilize the semantics-generated bounded model checker to search the state space to a depth of 300,000 for a `stuck` pattern, i.e., a machine that’s no longer responsive. This depth we used was adequate for a complete run of the both the fluid and antibiotics machines simultaneously. The search command was executed on a machine with 64 GB of memory, and took roughly 90 minutes, and reported no such state was possible. To the best of our knowledge, this makes ours the first system for screening and management of sepsis with some formal analysis.

9.9 DISCUSSION

In [section 9.1](#), we described several characteristics of BPGs that a language for expressing computer interpretable best practice guidelines should ideally accommodate. These include (i) comprehensibility for healthcare providers to enable them to verify correctness of medical

logic, (ii) ability to accommodate diverse external agents, (iii) execution and analysis tools with correctness guarantees.

In [chapter 4](#), we discussed progress made by existing approaches towards addressing them. We measured progress along the following lines:

- **Implementation-Specification Gap:** The textual BPG can also be used “as-is” in a CDSS implementation, eliminating any translation from a textual document to its computable encoding.
- **Formal Semantics:** The language has a complete formally defined semantics that serves as a manual for implementing tools and associated ecosystem.
- **Formal Analysis Tools:** Existence of a comprehensive set of analysis tools that can be used to analyze BPGs implemented in the language that evolve as the language evolves.
- **Holistic Safety:** Comprehensive support for formal analysis and tools, including execution engines, with correctness guarantees.

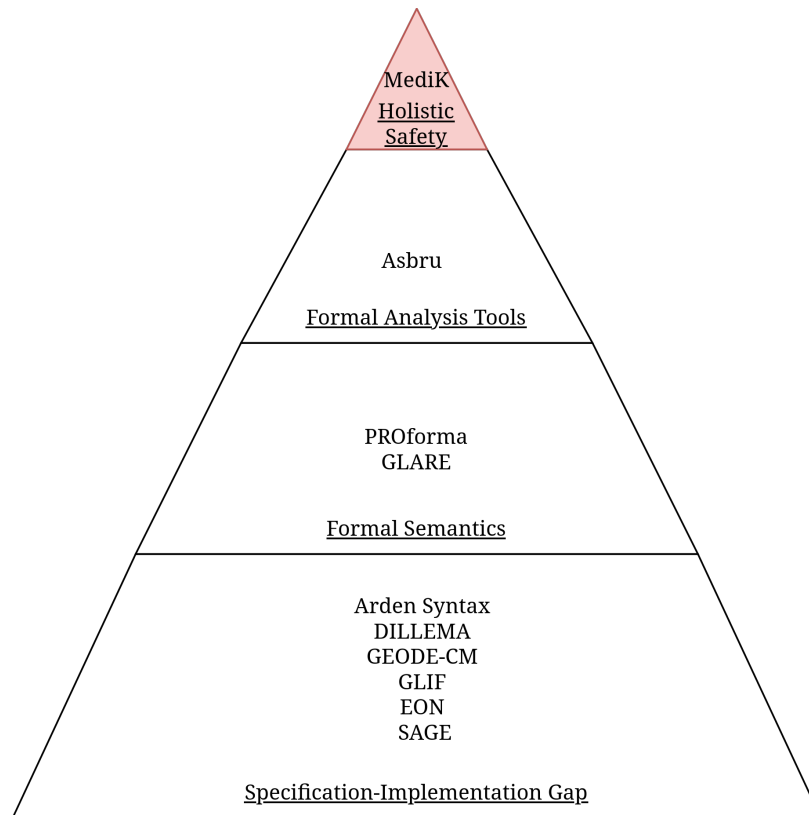


Figure 9.1: Our Contribution

When compared to existing approaches from [chapter 4](#), MediK is the first computer-interpretable guideline language that has a complete executable semantics. As MediK’s tools are derived from its semantics, they are, by definition, correct-by-construction. As the tools don’t have to be maintained independently, updates to the language to accommodate HCP feedback only require corresponding changes to the semantics, and tools evolve automatically.

This work attempted to establish that the semantics-first approach embodied by MediK makes building safer CDSSs easier. We show that MediK’s communicating state machines-based approach provides a convenient medium for expressing medical knowledge by utilizing it to build a real-world complex CDSS for management of sepsis in pediatric cases in collaboration with OSF St. Francis Medical Center. MediK’s support for modeling external agents as *ghosts* provides a uniform way of formal analysis. We use this to model-check the sepsis CDSSs for responsiveness, To the best of our knowledge, knowledge, our is the first system for sepsis management with any form of formal analysis. We briefly summarize how MediK compares against existing approaches in [Table 9.1](#).

In [chapter 5](#), we described incremental progress made by existing approaches towards enabling greater CDSS adoption through *solution themes* discussed in [section 4.3](#) and shown in [Figure 9.1](#). Specifically, upper rungs of the pyramid incorporates features and learnings of the lower rungs. Our contribution can be described by the run marked in red, as our

	Implementation- Specification Gap	Complete Formal Semantics	Formal Analysis Tools	Holistic Safety
Arden Syntax	✓	✗	✗	✗
DILLEMA	✓	✗	✗	✗
GEODE-CM	✓	✗	✗	✗
EON	✓	✗	✗	✗
GLIF	✓	✗	✗	✗
Asbru	✓	○	✓	✗
PROforma	✓	✓	✗	✗
GLARE	✓	○	○	○
GPROVE	✓	✓	✗	✗
HELEN	✓	✗	✗	✗
PRODIGY	✓	✗	✗	✗
SAGE	✓	✗	✗	✗
Hierarchical Plans	✗	○	✓	○
MDA	✗	✗	✓	✗
MediK	✓	✓	✓	✓

Table 9.1: MediK vs Existing Approaches

work incorporates the ability to implement HCP-comprehensible, computable BPGs of earlier approaches with a complete formal semantics, comprehensive support for formal analysis, and correctness guarantees for all tools.

CHAPTER 10: FUTURE WORK

So far, we have described objectives that have already been completed as part of this work. This chapter, however, describes remaining objectives that are important for a comprehensive solution to the challenges outlined in [chapter 3](#).

10.1 SOUNDNESS OF MODEL CHECKING ABSTRACTION

In [subsection 9.8.2](#), we introduced an abstraction that enabled us to model check the system. Instead of interpreting MediK programs concretely, we introduced rules that operate over abstract values that hold semantic meaning. However, we did not establish that the analysis is indeed sound. In this section, we briefly outline the steps to prove the soundness of our abstraction-based analysis, and outline limitations that prevented us from establishing it.

Recall that in [subsection 5.2.2](#), we presented the matching logic: the theoretical foundations of the $\mathbb{K}$ framework. We explained that a $\mathbb{K}$ definition of a language L , compiles to a matching logic theory Γ^L . A symbol $\bullet \in \Sigma$ is then defined as a part of the signature to capture dynamic behavior of rewrite rules. Intuitively, given a configuration pattern γ , the pattern $\bullet\gamma$ matches all possible configurations that γ can rewrite to in one-step. Thus, the one-step rewrite relation can be defined as:

$$\varphi \Rightarrow^1 \psi \equiv \varphi \rightarrow \bullet\psi \quad (10.1)$$

Next, the reflexive, transitive closure of the one-step rewrite relation can be defined using fixed-points as:

$$\diamond\varphi \equiv \mu X. \varphi \vee \bullet X \quad (10.2)$$

and the rewrite $\varphi \Rightarrow \psi$ as:

$$\varphi \Rightarrow \psi \equiv \varphi \rightarrow \diamond\psi \quad (10.3)$$

In other words, if a configuration matches φ then it matches φ in zero or more steps. Note that the existence of a fixed point of $\diamond\varphi$ is guaranteed by the Knaster-Tarski theorem as long as the interpretation $\mathcal{F}_{X,\varphi}^\rho = |\varphi|_{\rho[A/X]}$ is monotone [144].

Typically, establishing soundness of abstraction-based static analysis requires coming up partial order over abstract states, $(Abs, \sqsubseteq)$, and, semantic rules defined over Abs . For $a_1, a_2 \in Abs$, $a_1 \sqsubseteq a_2$ typically means a_1 is *more precise* than a_2 . Recall that in [subsection 9.8.2](#), that we defined a value `#nondet`, which represents the definition of a the very simple abstract domain $\{\top, \perp\}$, where $\top$ represents *undetermined* or *all-possible* values, and $\perp$ represents

missing/no-possible value. We then define rewrite rules that operate over these abstract values. Let $\Rightarrow_{Abs}$ be the transition system defined by rules over the abstract state. We first need to establish that for any $a_1, a_2 \in Abs$, if $a_1 \Rightarrow_{Abs} a_2$ then $a_1 \sqsubseteq a_2$. In other words, we need to show that the transition relation $\Rightarrow_{Abs}$ is monotonic w.r.t. the ordering $\sqsubseteq$.

Next, we need to relate the concrete transition system $\Rightarrow$ to $\Rightarrow_{Abs}$. Typically, this is done by defining a homomorphism $\beta : Concrete \rightarrow Abstract$ from the set of concrete values *Concrete* to the set of abstract values *Abstract*. We can then utilize β to define a complete lattice over configuration terms. Recall from [section 9.4](#) that MediK's configuration comprises of an `<store>`-cell that stores map ρ of pointer-value bindings. Given configuration terms $\bar{\tau}_1, \bar{\tau}_2 \in Abs$ with `<store>` cells containing maps $\rho_{\bar{\tau}_1}$ and $\rho_{\bar{\tau}_2}$ respectively. We say $\bar{\tau}_1 \sqsubseteq \bar{\tau}_2$ iff they only differ in their `<store>` cell (all other cells are similar), and, for every pointer in $p \in \text{keys}(\rho_{\bar{\tau}_1} \cup \rho_{\bar{\tau}_2})$ we have $\rho_{\bar{\tau}_1}[p] \sqsubseteq \rho_{\bar{\tau}_2}[p]$. Note we say, given a map ρ of pointer-value bindings, $\rho[p] = \perp$ for any $p \notin \text{keys}(\rho)$.

To complete the soundness argument, we need to make the complete the connection between concrete and abstract executions. Given homomorphism $\beta : Concrete \rightarrow Abstract$ from concrete values to abstract values, and a concrete configuration term τ . We derive an abstract configuration term $\bar{\beta}(\tau)$ from τ s.t. every (`pointer` $\mapsto$ `value`) pair in τ 's `<store>`-cell is mapped to (`pointer` $\mapsto$ $\beta(\text{value})$) in $\bar{\beta}(\tau)$'s `<store>`-cell. We then need to show that for concrete terms τ, τ' and abstract term $\bar{\tau}$, if (a) $\tau \Rightarrow \tau'$, and, (b) $\bar{\beta}(\tau) \sqsubseteq \bar{\tau}$ then there exists an abstract state $\bar{\tau}'$ such that $\bar{\tau} \Rightarrow_{Abs} \bar{\tau}'$ and $\bar{\beta}(\tau') \sqsubseteq \bar{\tau}'$. This, along with the proof of monotonicity of $\Rightarrow_{Abs}$ w.r.t. the ordering $\sqsubseteq$ enables us to show that any trace in the concrete semantics is safely approximated by a corresponding trace in the abstract semantics, as shown in [\[193\]](#). However, while we briefly sketch out a proof outline in this work, a complete proof for a language as complex as MediK is a significant undertaking, which we leave to future work.

Note that instead of replacing a concrete value with a corresponding abstract value, we could have simply used a logical variable and symbolic execution to explore possible interleavings. This would have allowed us to 1. use satisfiability modulo theory (SMT) solvers to eliminate non-reachable branches, and, 2. eliminate the need to develop an maintain an abstract semantics, as existing rules would work for both concrete and symbolic execution.

$\mathbb{K}$ has multiple backends, including a fast execution LLVM backend that can only handle *ground* patterns, i.e. patterns with logical variables, and a haskell backend that can handle ML patterns in their full generality [\[188\]](#). Symbolic execution requires the use of $\mathbb{K}$'s haskell backend, which presented significant performance related challenges when handling MediK programs as large as the pediatric sepsis management guidelines presented in our work. Our abstraction-based model checking was in part motivated by said performance issues. We

leave moving to symbolic execution-based program analysis as part of future work.

10.2 SEMANTICS BASED VISUAL REPRESENTATION GENERATION

Extremely vital to the success of MediK, the ability to generate visual representation of MediK programs to aid comprehensibility to medical domain experts, enabling them to verify accuracy of encoded medical knowledge. Even though MediK code structurally resembles paper-based guidelines, said representations aid comprehensibility for medical domain experts as the representations resemble their paper-based counterparts, imparting a sense of familiarity.

Control Flow Graphs (CFGs) form the basis of not only such visual representations, but other important analyses. For example, flow-sensitive analyzers can be used to verify that a medical procedure is performed on both treatment branches. CFGs find use in applications from compilers to model checkers. But, despite their importance, they are not well understood. To illustrate the lack of a *canonical* definition of a CFG, in [194], the authors used three different CFG generators on the same program, and obtaining three different CFGs, each tailored to the objectives of the generator.

Thus, generation of CFGs that present *all relevant* information, with *minimal* intervention during generation, in a *correct-by-construction* fashion, is indeed challenging. In [194] that authors argue that operational semantics are not suitable to be used directly for generation of CFGs by demonstrating that many control flow edges correspond to different halves of the same rule, while some correspond to none. To address this, the authors utilize an algorithm to construct an Abstract Machines (AM) style semantics from the SOS.

Consider for example, the following SOS rules for assignment:

$$\frac{e, \mu \rightsquigarrow e' \mu'}{x := e, \mu \rightsquigarrow x := e', \mu'} \text{ AssnCong} \quad \frac{}{x := v, \mu \rightsquigarrow (\text{skip}, \mu[x \rightarrow v])} \text{ AssnEval} \quad (10.4)$$

The authors then automatically generate an AM, written $\langle (t, \mu) \mid K \rangle$ where K is the *context* or *continuation* composed of frames of the form $[x := (\Box_t, \Box_\mu)]$ indicating holes for an evaluated value and environment. Thus, the AM rules are:

$$\begin{aligned} \langle (x := e, \mu) \mid k \rangle &\rightarrow \langle (e, \mu) \mid k \circ [x := (\Box_t, \Box_\mu)] \rangle \\ \langle (v, \mu) \mid k \rangle &\rightarrow \langle (\text{skip}, \mu[x \rightarrow v]) \mid k \rangle \end{aligned} \quad (10.5)$$

Next, as it's possible for the AM to have an infinitely-many states, an abstraction is required to collapse the states and make the transition system finite. To this end, the

authors utilize a *value irrelevant* abstraction that replaces all values with a single one $\star$ that representing any of them. With this, the abstract states can be treated as nodes in the CFG. Thus, program execution with this abstraction will produce a CFG. For example, consider evaluation of statement $x := y$, under an infinite number of environments, can lead to an infinite number of states of the AM. But, converting all values to $\star$ results in one-possible execution: $\langle (x := y, [y \mapsto \star]) \mid k \rangle \rightarrow \langle (y, [y \mapsto \star]) \mid k \circ [x := (\Box_t, \Box_u)] \rangle \rightarrow \langle (\star, [y \mapsto \star]) \mid k \circ [x := (\Box_t, \Box_u)] \rangle \rightarrow \langle (\text{skip}, [x \mapsto \star, y \mapsto \star]) \mid k \rangle$

Note that expressions built over $\star$ -values can still pose a challenge. For instance, a statement of the form **while** e **do** s when evaluated under an environment $[\star \mapsto \star]$ requires handling $\langle (e, [\star \mapsto \star]) \mid k \rangle$, which can be matched by any expression rule. To handle this, $\langle (e, [\star \mapsto \star]) \mid k \rangle$ is over-approximated to $\langle (\star, [\star \mapsto \star]) \mid k \rangle$ as evaluating any such e would eventually result in $\star$.

To generate visual representations, we plan to utilize ideas from [194] in conjunction with the $\mathbb{K}$ -Summarizer toolchain. The $\mathbb{K}$ -Summarizer is an effort to improve performance of semantics-based tools by modifying the $\mathbb{K}$ -definition itself. One of the primary objectives of the summarizer is to enable semantics-based compilation. Semantics-based compilation can be considered a generalization of established techniques for established techniques for program optimization such as partial evaluation of programs [195].

For a given language L , let $\llbracket _ \rrbracket$ be the interpreter for L . Say, for given $p \in L\text{-Program}$, and inputs in_1, in_2 , $\text{output} = \llbracket p \rrbracket [\text{in}_1, \text{in}_2]$ if p terminates. We can now express partial evaluation of program p , using another program mix , called so since performs a mix of both execution and code generation. Now, we get a two-stage execution process:

$$\begin{aligned} p_{\text{in}_1} &= \llbracket \text{mix} \rrbracket [p, \text{in}_1] \\ \text{output} &= \llbracket p_{\text{in}_1} \rrbracket \text{in}_2 \end{aligned} \tag{10.6}$$

Now, for given program p , if in_1 is statically known (at stage 1), and in_2 is dynamically known (at stage 2), then, $\llbracket p_{\text{in}_1} \rrbracket$ can be computed *once*, and re-used for any new in_2 . Semantics-based compilation can be seen as a generalization of this idea. Say L is a $\mathbb{K}$ definition. We can generate a new definition $L^{p_{\text{in}_1}}$ that directly implements $\llbracket p_{\text{in}_1} \rrbracket$. At runtime, only the dynamically determined input in_2 to complete execution.

Conceptually, the summarization process results in a CFG, where each basic block is summarized as a $\mathbb{K}$ rule. But, in Medi $\mathbb{K}$'s case, the inherent concurrency presents additional challenges that we intend to tackle.

While SBC is one of the primary goals of the $\mathbb{K}$ -Summarizer project, it has also been used to optimize $\mathbb{K}$ definitions by *merging* multiple rules that take smaller steps into one

rule that *summarizes* all of them. This is similar to the translation from SOS to abstract machine presented in [194]. In the future, we plan to generate such *summaries* soundly and utilize them to generate visualizations of MediK programs. Note, that both $\mathbb{K}$ Summarizer and the work in [194] have been tried in *deterministic* settings, and we anticipate additional challenges in translating it to concurrent settings.

10.3 APPLICABILITY TO OTHER CDSSS

Throughout this work, we have used the sepsis management BPG as a case study for clinical decision support systems. While the BPG is a real-world, complex CDSSs for management of sepsis in pediatric cases, it remains to be seen if MediK can act as a convenient medium for building CDSSs in general. This can most conveniently be answered through implementing more CDSSs using our approach.

Recall in [chapter 8](#) we presented a CDSS for management of cardiac arrest in adult cases based on the advanced cardiac life support (ACLS) guidelines published by the American Heart Association (AHA). In [section 8.2](#), we presented the $\mathbb{K}$ -ACLS CDSS for management of cardiac arrest. The $\mathbb{K}$ -ACLS system medical logic has been expressed as $\mathbb{K}$ definition, which, while great for conciseness, required prior experience with $\mathbb{K}$ to understand. Thus, as a first step towards standardizing more guidelines in MediK, and evaluating its applicability, we intend to translate embed the existing $\mathbb{K}$ -ACLS BPG into MediK. Our choice is motivated by the following reasons: (i) an effective CDSS for cardiac arrest can significantly improve outcomes, (ii) enables us to evaluate the MediK code against directly encoding the guideline in $\mathbb{K}$, as we had used the same BPG for a $\mathbb{K}$ -based CDSSs from [section 8.2](#).

CHAPTER 11: CONCLUSION

Chapter 1 explains the persistent challenge that preventable medical errors (PMEs) present in medicine. The seriousness of the problem can be gauged by the fact that President’s Council on Science and Technology (PCAST) declared the prevalence of PMEs to be an “urgent national public health issue”.

Among the solutions outlined in the PCAST report include: (i) Utilizing *evidence-based* medicine. (ii) Investing in technology that assists healthcare practitioners (HCPs) in decision making. Clinical best practice guidelines (BPGs) exemplify evidence-based medicine. Produced by hospital and medical associations, these guidelines are evidence-based statements that codify recommended treatment for various clinical scenarios, and growing evidence supports their use in medicine to reduce PMEs. But their effectiveness in practice is often limited by challenges in their uptake and adoption. As outlined in chapter 3, integrating BPGs into HCP workflows and following them in practice is often difficult, resulting in treatment that deviates from prescribed best-practices, and as a result, leads to worse outcomes. Guidelines-based clinical decision support systems that assist HCPs with situation-specific advice to enhance guideline adherence have hence been shown to improve patient outcomes.

Despite many apparent benefits, the prevalence of CDSSs in practice remains low, due to factors outlined in chapter 3. In this work, we utilize a semantics-first approach to building safe clinical decision support systems.

By semantics-first, we mean that:

- The semantics of medical knowledge are accurately captured in the system.
- The language for expressing medical knowledge has a well-defined, formal semantics from which tools for execution and formal analysis are derived.

CDSSs are typically developed by software developers lacking expertise in medicine. Thus, it is possible for medical knowledge in a CDSS to be incorrectly encoded during development. As CDSSs operate in safety critical settings, such bugs can lead to adverse outcomes for patients. To address this, we developed a domain specific language for describing medical knowledge called MediK. Developed as an academic endeavor, MediK has been designed to be comprehensible to HCPs. This allows HCPs to examine and validate the accuracy medical knowledge in a MediK-based CDSS, enabling greater transparency in the source of the CDSS’s recommendations.

As outlined in chapter 4, the need for DSLs for expressing BPGs to have formal semantics and support for comprehensive support for formal analysis has already been recognized.

MediK builds on the state-of-art in modeling concurrent systems and medical knowledge to enable complex medical guidelines with multiple concurrent workflows and interdependencies to be conveniently expressed. To demonstrate this, we collaborate with OSF St. Francis Hospital in Peoria, Illinois—a major pediatric hospital to develop a CDSS for their pediatric sepsis management guidelines. During the development process, bugs in medical logic were uncovered by collaborating physicians by examining MediK code, and learnings from our endeavor were incorporated into the design of the language.

We believe the work outlined in this thesis presents the first steps towards addressing challenges inhibiting broader CDSS uptake. But a comprehensive solution involves addressing several more challenges, some of which are outlined [chapter 10](#). Some of these challenges demand collaboration from multiple stakeholders, from both computer science and medicine. Thus, they remain unaddressed for now. But we hope addressing them, as part of building on top of work presented here, can lower PMEs, leading to better quality healthcare at reduced costs.

REFERENCES

- [1] T. L. Rodziewicz, B. Houseman, and J. E. Hipskind, *Medical Error Reduction and Prevention*. StatPearls Publishing, Treasure Island (FL), 2022. [Online]. Available: <http://europepmc.org/books/NBK499956>
- [2] M. S. Donaldson, J. M. Corrigan, L. T. Kohn et al., *To err is human: building a safer health system*. National Academies Press, 2000.
- [3] M. A. Makary and M. Daniel, “Medical error—the third leading cause of death in the us,” *The BMJ*, vol. 353, 2016. [Online]. Available: <https://www.bmj.com/content/353/bmj.i2139>
- [4] President’s Council on Science and Technology , “ PCAST Releases Report on Transforming Patient Safety ,” <https://www.whitehouse.gov/pcast/briefing-room/2023/09/07/pcast-releases-report-on-transforming-patient-safety/>.
- [5] C. Andel, S. L. Davidow, M. Hollander, and D. A. Moreno, “The economics of health care quality and medical errors,” *Journal of health care finance*, vol. 39, no. 1, p. 39, 2012.
- [6] “Commercial Aviation Safety Team,” <https://www.cast-safety.org/apex/f?p=102:1>.
- [7] Commercial Aviations Safety Team (CAST), “Improving Aviation Safety.”
- [8] “Fly-By-Wire,” <https://skybrary.aero/articles/fly-wire>.
- [9] “Cockpit Automation - Advantages and Safety Challenges,” <https://skybrary.aero/articles/cockpit-automation-advantages-and-safety-challenges>.
- [10] “Automation and ATM,” <https://skybrary.aero/articles/automation-and-atm>.
- [11] National Business Aviation Association, “Traffic Alert and Collision Avoidance System (TCAS),” <https://nbaa.org/aircraft-operations/communications-navigation-surveillance-cns/tcas/>.
- [12] “CAST Safety Enhancements,” <https://skybrary.aero/enhancing-safety/cast-safety-enhancements>.
- [13] C. R. Gerstle, “Parallels in safety between aviation and healthcare,” *Journal of pediatric surgery*, vol. 53, no. 5, pp. 875–878, 2018.
- [14] M. J. Field, K. N. Lohr et al., *Clinical practice guidelines*. National Academies Press (US) Washington, DC, USA, 1990.
- [15] E. Steinberg, S. Greenfield, D. M. Wolman, M. Mancher, R. Graham et al., *Clinical practice guidelines we can trust*. National Academies Press, 2011.

- [16] A. R. Panchal, J. A. Bartos, J. G. Cabañas, M. W. Donnino, I. R. Drennan, K. G. Hirsch, P. J. Kudenchuk, M. C. Kurz, E. J. Lavonas, P. T. Morley et al., “Part 3: adult basic and advanced life support: 2020 american heart association guidelines for cardiopulmonary resuscitation and emergency cardiovascular care,” *Circulation*, vol. 142, no. 16_Suppl.2, pp. S366–S468, 2020.
- [17] A. A. Topjian, T. T. Raymond, D. Atkins, M. Chan, J. P. Duff, B. L. Joyner Jr, J. J. Lasa, E. J. Lavonas, A. Levy, M. Mahgoub et al., “Part 4: Pediatric basic and advanced life support: 2020 american heart association guidelines for cardiopulmonary resuscitation and emergency cardiovascular care,” *Circulation*, vol. 142, no. 16_Suppl.2, pp. S469–S523, 2020.
- [18] J. P. Ornato, M. A. Peberdy, R. D. Reid, V. R. Feeser, H. S. Dhindsa, N. investigators et al., “Impact of resuscitation system errors on survival from in-hospital cardiac arrest,” *Resuscitation*, vol. 83, no. 1, pp. 63–69, 2012.
- [19] H. A. Wolfe, R. W. Morgan, B. Zhang, A. A. Topjian, E. L. Fink, R. A. Berg, V. M. Nadkarni, A. Nishisaki, J. Mensinger, R. M. Sutton et al., “Deviations from aha guidelines during pediatric cardiopulmonary resuscitation are associated with decreased event survival,” *Resuscitation*, vol. 149, pp. 89–99, 2020.
- [20] C. P. Crowley, J. D. Saliccioli, and E. Y. Kim, “The association between acls guideline deviations and outcomes from in-hospital cardiac arrest,” *Resuscitation*, vol. 153, pp. 65–70, 2020.
- [21] K. Honarmand, C. Mepham, C. Ainsworth, and Z. Khalid, “Adherence to advanced cardiovascular life support (acls) guidelines during in-hospital cardiac arrest is associated with improved outcomes,” *Resuscitation*, vol. 129, pp. 76–81, 2018.
- [22] M. D. McEvoy, L. C. Field, H. E. Moore, J. C. Smalley, P. J. Nietert, and S. H. Scarbrough, “The effect of adherence to acls protocols on survival of event in the setting of in-hospital cardiac arrest,” *Resuscitation*, vol. 85, no. 1, pp. 82–87, 2014.
- [23] C. Rand, N. Powe, A. Wu, and M. Wilson, “Why don’t physicians follow clinical practice guidelines,” *Journal of the American Medical Association (JAMA)*, vol. 282, p. 14581465, 1999.
- [24] D. A. Davis and A. Taylor-Vaisey, “Translating guidelines into practice: a systematic review of theoretic concepts, practical experience and research evidence in the adoption of clinical practice guidelines,” *Canadian Medical Association Journal (CMAJ)*, vol. 157, no. 4, pp. 408–416, 1997.
- [25] S. H. Woolf, R. Grol, A. Hutchinson, M. Eccles, and J. Grimshaw, “Potential benefits, limitations, and harms of clinical guidelines,” *The BMJ*, vol. 318, no. 7182, pp. 527–530, 1999.

- [26] A. X. Garg, N. K. Adhikari, H. McDonald, M. P. Rosas-Arellano, P. J. Devereaux, J. Beyene, J. Sam, and R. B. Haynes, “Effects of computerized clinical decision support systems on practitioner performance and patient outcomes: a systematic review,” *Journal of the American Medical Association (JAMA)*, vol. 293, no. 10, pp. 1223–1238, 2005.
- [27] K. Kawamoto, C. A. Houlihan, E. A. Balas, and D. F. Lobach, “Improving clinical practice using clinical decision support systems: a systematic review of trials to identify features critical to success,” *The BMJ*, vol. 330, no. 7494, p. 765, 2005.
- [28] P. Bennett and N. R. Hardiker, “The use of computerized clinical decision support systems in emergency care: a substantive review of the literature,” *Journal of the American Medical Informatics Association (JAMIA)*, vol. 24, no. 3, pp. 655–668, 12 2016.
- [29] N. Sahota, R. Lloyd, A. Ramakrishna, J. Mackay, J. Prorok, L. Weise-Kelly, T. Navarro-Ruan, N. Wilczynski, and b. Haynes, “Computerized clinical decision support systems for acute care management: A decision-maker-researcher partnership systematic review of effects on process of care and patient outcomes,” *Implementation Science (IS)*, vol. 6, p. 91, 08 2011.
- [30] B. C. James, “Making it easy to do it right,” *New England Journal of Medicine (NEJM)*, vol. 345, no. 13, pp. 991–993, 2001.
- [31] R. T. Sutton, D. Pincock, D. C. Baumgart, D. C. Sadowski, R. N. Fedorak, and K. I. Kroeker, “An overview of clinical decision support systems: benefits, risks, and strategies for success,” *npj Digital Medicine*, vol. 3, no. 1, p. 17, 2020.
- [32] Office of the National Coordinator for Health Information Technology (ONC), “About ONC,” <https://www.healthit.gov/topic/about-onc>.
- [33] Office of the National Coordinator for Health Information Technology (ONC), “Clinical decision support,” <https://www.healthit.gov/topic/safety/clinical-decision-support>.
- [34] “Optimizing Strategies for Clinical Decision Support,” in *The Learning Health Systems Series*, T. J. Lopez MH, Weingarten S et al., Eds., National Academy of Medicine. National Academies Press (US), 2017.
- [35] T. J. Bright, A. Wong, R. Dhurjati, E. Bristow, L. Bastian, R. R. Coeytaux, G. Samsa, V. Hasselblad, J. W. Williams, M. D. Musty et al., “Effect of clinical decision-support systems: a systematic review,” *Annals of internal medicine*, vol. 157, no. 1, pp. 29–43, 2012.
- [36] M. Peleg, “Computer-interpretable clinical guidelines: A methodological review,” *Journal of Biomedical Informatics (JBI)*, vol. 46, no. 4, pp. 744–763, 2013.
- [37] P. A. De Clercq, J. A. Blom, H. H. Korsten, and A. Hasman, “Approaches for creating computer-interpretable guidelines that facilitate decision support,” *Artificial Intelligence in Medicine (AIM)*, vol. 31, no. 1, pp. 1–27, 2004.

- [38] P. Kubben, M. Dumontier, and A. Dekker, *Fundamentals of clinical data science*. Springer, 2019.
- [39] D. R. Sutton and J. Fox, “The syntax and semantics of the PROforma guideline modeling language,” *Journal of the American Medical Informatics Association (AMIA)*, vol. 10, no. 5, pp. 433–443, 2003.
- [40] Y. Shahar, S. Miksch, and P. Johnson, “An intention-based language for representing clinical guidelines.” in *Proceedings of the AMIA Annual Fall Symposium*. American Medical Informatics Association, 1996, p. 592.
- [41] “The K Framework,” <https://kframework.org>.
- [42] C. Hathhorn, C. Ellison, and G. Roşu, “Defining the undefinedness of c,” in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI ’15. New York, NY, USA: Association for Computing Machinery, 2015, p. 336–345.
- [43] D. Bogdănaş and G. Roşu, “K-Java: A complete semantics of Java,” in *Proceedings of the 42nd Symposium on Principles of Programming Languages (POPL’15)*. ACM, 1 2015, pp. 445–456.
- [44] E. Hildenbrandt, M. Saxena, N. Rodrigues, X. Zhu, P. Daian, D. Guth, B. Moore, D. Park, Y. Zhang, A. Stefanescu, and G. Rosu, “KEVM: A complete formal semantics of the Ethereum Virtual Machine,” in *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, 2018, pp. 204–217.
- [45] D. Park, Y. Zhang, M. Saxena, P. Daian, and G. Roşu, “A formal verification tool for ethereum vm bytecode,” in *Proceedings of the 2018 26th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2018, pp. 912–915.
- [46] A. Stăfănescu, D. Park, S. Yuwen, Y. Li, and G. Roşu, “Semantics-based program verifiers for all languages,” *ACM SIGPLAN Notices*, vol. 51, no. 10, pp. 74–91, 2016.
- [47] American Heart Association (AHA), “CPR and Emergency Cardiovascular Care Algorithms,” <https://cpr.heart.org/en/resuscitation-science/cpr-and-ecc-guidelines/algorithms>.
- [48] MD Anderson Cancer Center, “Clinical Practice Algorithms,” <https://www.mdanderson.org/for-physicians/clinical-tools-resources/clinical-practice-algorithms.html>.
- [49] A. Desai, V. Gupta, E. Jackson, S. Qadeer, S. Rajamani, and D. Zufferey, “P: Safe asynchronous event-driven programming,” in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI ’13. New York, NY, USA: Association for Computing Machinery, 2013, p. 321–332.

- [50] M. Clavel, F. Durán, S. Eker, P. Lincoln, N. Martí-Oliet, J. Meseguer, and C. Talcott, *All about maude-a high-performance logical framework: how to specify, program, and verify systems in rewriting logic*. Springer, 2007, vol. 4350.
- [51] A. Rhodes, L. E. Evans, W. Alhazzani, M. M. Levy, M. Antonelli, R. Ferrer, A. Kumar, J. E. Sevransky, C. L. Sprung, M. E. Nunnally et al., “Surviving sepsis campaign: international guidelines for management of sepsis and septic shock: 2016,” *Intensive Care Medicine*, vol. 43, pp. 304–377, 2017.
- [52] M. Eisenberg, E. Freiman, A. Capraro, K. Madden, M. C. Monuteaux, J. Hudgins, and M. Harper, “Comparison of manual and automated sepsis screening tools in a pediatric emergency department,” *Pediatrics*, vol. 147, no. 2, 2021.
- [53] S. L. Weiss, J. C. Fitzgerald, F. Balamuth, E. R. Alpern, J. Lavelle, M. Chilutti, R. Grundmeier, V. M. Nadkarni, and N. J. Thomas, “Delayed antimicrobial therapy increases mortality and organ dysfunction duration in pediatric sepsis,” *Critical Care Medicine*, vol. 42, no. 11, p. 2409, 2014.
- [54] I. V. Evans, G. S. Phillips, E. R. Alpern, D. C. Angus, M. E. Friedrich, N. Kissoon, S. Lemeshow, M. M. Levy, M. M. Parker, K. M. Terry et al., “Association between the new york sepsis care mandate and in-hospital mortality for pediatric sepsis,” *Journal of American Medicine (JAMA)*, vol. 320, no. 4, pp. 358–367, 2018.
- [55] F. Balamuth, E. R. Alpern, M. K. Abbadessa, K. Hayes, A. Schast, J. Lavelle, J. C. Fitzgerald, S. L. Weiss, and J. J. Zorc, “Improving recognition of pediatric severe sepsis in the emergency department: contributions of a vital sign–based electronic alert and bedside clinician identification,” *Annals of Emergency Medicine*, vol. 70, no. 6, pp. 759–768, 2017.
- [56] R. J. Sepanski, S. A. Godambe, C. D. Mangum, C. S. Bovat, A. L. Zaritsky, and S. H. Shah, “Designing a pediatric severe sepsis screening tool,” *Frontiers in Pediatrics*, vol. 2, p. 56, 2014.
- [57] M. J. Field, K. N. Lohr et al., *Clinical practice guidelines*. National Academies Press (US) Washington, DC, USA, 1990.
- [58] E. Guerra-Farfan, Y. Garcia-Sanchez, M. Jornet-Gibert, J. H. Nuñez, M. Balaguer-Castro, and K. Madden, “Clinical practice guidelines: The good, the bad, and the ugly,” *Injury*, vol. 54, pp. S26–S29, 2023.
- [59] Busse, Reinhard and Klazinga, Niek and Panteli, Dimitra and Quentin, Wilm and World Health Organization and others, *Improving healthcare quality in Europe: Characteristics, effectiveness and implementation of different strategies*. World Health Organization. Regional Office for Europe, 2019.
- [60] G. Guyatt, J. Cairns, D. Churchill, D. Cook, B. Haynes, J. Hirsh, J. Irvine, M. Levine, M. Levine, J. Nishikawa et al., “Evidence-based medicine: a new approach to teaching the practice of medicine,” *Jama*, vol. 268, no. 17, pp. 2420–2425, 1992.

- [61] D. L. Sackett and W. M. C. Rosenberg, "On the need for evidence-based medicine," *Journal of Public Health*, vol. 17, no. 3, pp. 330–334, 1995.
- [62] P. De Clercq, K. Kaiser, and A. Hasman, "Computer-interpretable guideline formalisms," *Studies in Health Technology and Informatics*, vol. 139, p. 22, 2008.
- [63] J. H. Barth, S. Misra, K. M. Aakre, M. R. Langlois, J. Watine, P. J. Twomey, and W. P. Oosterhuis, "Why are clinical practice guidelines not followed?" *Clinical Chemistry and Laboratory Medicine (CCLM)*, vol. 54, no. 7, pp. 1133–1139, 2016.
- [64] A. X. Garg, N. K. Adhikari, H. McDonald, M. P. Rosas-Arellano, P. J. Devereaux, J. Beyene, J. Sam, and R. B. Haynes, "Effects of computerized clinical decision support systems on practitioner performance and patient outcomes: a systematic review," *Jama*, vol. 293, no. 10, pp. 1223–1238, 2005.
- [65] S. Wells, S. Furness, N. Rafter, E. Horn, R. Whittaker, A. Stewart, K. Moodabe, P. Roseman, V. Selak, D. Bramley et al., "Integrated electronic decision support increases cardiovascular disease risk assessment four fold in routine primary care practice," *European Journal of Preventive Cardiology*, vol. 15, no. 2, pp. 173–178, 2008.
- [66] S. Rodriguez-Loya and K. Kawamoto, "Newer architectures for clinical decision support," *Clinical Decision Support Systems: Theory and Practice*, pp. 87–97, 2016.
- [67] E. Shortliffe, *Computer-based medical consultations: MYCIN*. Elsevier, 2012, vol. 2.
- [68] G. O. Barnett, J. J. Cimino, J. A. Hupp, and E. P. Hoffer, "DXplain: an evolving diagnostic decision-support system," *Jama*, vol. 258, no. 1, pp. 67–74, 1987.
- [69] K. Kawamoto, G. Del Fiol, C. Orton, and D. F. Lobach, "System-agnostic clinical decision support services: benefits and challenges for scalable decision support," *The open medical informatics journal*, vol. 4, p. 245, 2010.
- [70] T. Erl, *Service-oriented architecture*. Prentice Hall, 2005.
- [71] J. Solomon, K. Dauber-Decker, S. Richardson, S. Levy, S. Khan, B. Coleman, R. Persaud, J. Chelico, D. King, A. Spyropoulos et al., "Integrating clinical decision support into electronic health record systems using a novel platform (evidencepoint): developmental study," *JMIR Formative Research*, vol. 7, p. e44065, 2023.
- [72] A. Press, C. DeStio, L. McCullagh, S. Kapoor, J. Morley, S. N.-I. Team, J. Conigliaro et al., "Usability testing of a national substance use screening tool embedded in electronic health records," *JMIR human factors*, vol. 3, no. 2, p. e5820, 2016.
- [73] A. C. Li, J. L. Kannry, A. Kushniruk, D. Chrimes, T. G. McGinn, D. Edonyabo, and D. M. Mann, "Integrating usability testing and think-aloud protocol analysis with "near-live" clinical simulations in evaluating clinical decision support," *International journal of medical informatics*, vol. 81, no. 11, pp. 761–772, 2012.

- [74] R. Shiffman and A. Wright, “Evidence-Based Clinical Decision Support,” *Yearbook of medical informatics*, vol. 8, pp. 120–7, 08 2013.
- [75] Y. Shahar, O. Young, E. Shalom, A. Mayaffit, R. Moskovitch, A. Hessian, and M. Galperin, “The Digital electronic Guideline Library (DeGeL): a hybrid framework for representation and use of clinical guidelines,” in *Computer-based Support for Clinical Guidelines and Protocols*. IOS Press, 2004, pp. 147–151.
- [76] G. Hripcsak, “Writing arden syntax medical logic modules,” *Computers in biology and medicine*, vol. 24, no. 5, pp. 331–363, 1994.
- [77] M. Peleg, A. A. Boxwala, O. Ogunyemi, Q. Zeng, S. Tu, R. Lacson, E. Bernstam, N. Ash, P. Mork, L. Ohno-Machado et al., “Glif3: the evolution of a guideline representation format,” in *Proceedings of the AMIA Symposium*. American Medical Informatics Association, 2000, p. 645.
- [78] S. Bäumler, M. Balser, A. Dunets, W. Reif, and J. Schmitt, “Verification of medical guidelines by model checking – a case study,” in *Model Checking Software*, A. Valmari, Ed. Springer Berlin Heidelberg, 2006, pp. 219–233.
- [79] M. Samwald, K. Fehre, J. De Bruin, and K.-P. Adlassnig, “The arden syntax standard for clinical decision support: experiences and directions,” *Journal of Biomedical Informatics*, vol. 45, no. 4, pp. 711–718, 2012.
- [80] V. Anand, P. G. Biondich, G. C. Liu, M. B. Rosenman, S. M. Downs et al., “Child health improvement through computer automation: the chica system,” in *Medinfo*, 2004, pp. 187–191.
- [81] H. C. Karadimas, C. Chailloleau, F. Hemery, J. Simonnet, and E. Lepage, “Arden/j: an architecture for mlm execution on the java platform,” *Journal of the American Medical Informatics Association*, vol. 9, no. 4, pp. 359–368, 2002.
- [82] A. Blacky, H. Mandl, K.-P. Adlassnig, and W. Koller, “Fully automated surveillance of healthcare-associated infections with moni-icu,” *Applied Clinical Informatics*, vol. 2, no. 03, pp. 365–372, 2011.
- [83] E. Steinbrecher, D. Sohr, S. a. Hansen, A. Nassauer, F. Daschner, H. Rüden, and P. Gastmeier, “Surveillance postoperativer Wundinfektionen–Referenzdaten des Krankenhaus-Infektions-Surveillance-Systems (KISS),” *Der Chirurg*, vol. 73, pp. 76–82, 2002.
- [84] T. Engel, K. Fehre, A. Rappelsberger, and K. Adlassnig, “Qualitätssicherung in der geburtshilfe durch leitlinienbasierte entscheidungsunterstützung,” *Proceedings of eHealth2011, Austrian Computer Society*, pp. 313–318, 2011.
- [85] K. Boegl, N. Anastassova, K.-P. Adlassnig, A. Rappelsberger, M. Hayde, and A. Pollak, “Knowledge-based computer-aided decision support in prenatal toxoplasmosis screening (temptoxopert),” in *AMIA Annual Symposium Proceedings*, vol. 2005. American Medical Informatics Association, 2005, p. 897.

- [86] S. I. Herbert, C. J. Gordon, A. Jackson-Smale, and J.-L. R. Salis, "Protocols for clinical care," *Computer Methods and Programs in Biomedicine*, vol. 48, no. 1-2, pp. 21–26, 1995.
- [87] P. E. Stoufflet, L. Ohno-Machado, S. R. Deibel, D. Lee, and R. A. Greenes, "GEODE-CM: a state-transition framework for clinical management," in *Proceedings of the AMIA Annual Fall Symposium*. American Medical Informatics Association, 1996, p. 924.
- [88] L. Ohno-Machado, J. H. Gennari, S. N. Murphy, N. L. Jain, S. W. Tu, D. E. Oliver, E. Pattison-Gordon, R. A. Greenes, E. H. Shortliffe, and G. O. Barnett, "The guideline interchange format: a model for representing guidelines," *Journal of the American Medical Informatics Association*, vol. 5, no. 4, pp. 357–372, 1998.
- [89] S. W. Tu and M. A. Musen, "The eon model of intervention protocols and guidelines." in *Proceedings of the AMIA Annual Fall Symposium*. American Medical Informatics Association, 1996, p. 587.
- [90] M. A. Musen, S. W. Tu, A. K. Das, and Y. Shahar, "EON: A component-based approach to automation of protocol-directed therapy," *Journal of the American Medical Informatics Association*, vol. 3, no. 6, pp. 367–388, 1996.
- [91] A. A. Boxwala, R. A. Greenes, and S. Deibel, "Architecture for a multipurpose guideline execution engine." in *Proceedings of the AMIA Symposium*. American Medical Informatics Association, 1999, p. 701.
- [92] R. D. Zielstorff, J. M. Teich, M. D. Paterno, M. Segal, G. J. Kuperman, F. L. Hiltz, and R. L. Fox, "P-cape: a high-level tool for entering and processing clinical practice guidelines. partners computerized algorithm and editor." in *Proceedings of the AMIA Symposium*. American Medical Informatics Association, 1998, p. 478.
- [93] J. Choi, L. M. Currie, D. Wang, and S. Bakken, "Encoding a clinical practice guideline using guideline interchange format: a case study of a depression screening and management guideline," *International Journal of Medical Informatics*, vol. 76, pp. S302–S307, 2007.
- [94] D. Wang, "Translating arden mlms into glif guidelines—a case study of hyperkalemia patient screening," in *Computer-based Support for Clinical Guidelines and Protocols*. IOS Press, 2004, pp. 177–181.
- [95] "The Tallis Toolset." [Online]. Available: <http://www.acl.icnet.uk/lab/tallis/index.html>
- [96] R. Walton, C. Gierl, P. Yudkin, H. Mistry, M. Vessey, and J. Fox, "Evaluation of computer support for prescribing (capsule) using simulated cases," *BMJ*, vol. 315, no. 7111, pp. 791–795, 1997.
- [97] J. Bury, C. Hurt, A. Roy, L. Cheesman, M. Bradburn, S. Cross, J. Fox, and V. Saha, "Lisa: a web-based decision-support system for trial management of childhood acute lymphoblastic leukaemia," *British journal of haematology*, vol. 129, no. 6, pp. 746–754, 2005.

- [98] P. Taylor, J. Fox, and A. T. Pokropek, "The development and evaluation of cadmium: a prototype system to assist in the interpretation of mammograms," *Medical Image Analysis*, vol. 3, no. 4, pp. 321–337, 1999.
- [99] J. Fox, V. Patkar, R. Thomson, and E. Black, "Proforma guidelines and care pathways: performance in trials and future plans," in *The biennial European Conference on Artificial Intelligence (ECAI) Workshop: AI techniques in healthcare: computerized guidelines and protocols*. Riva del Garda, Italy, 2006.
- [100] P. E. Friedland and Y. Iwasaki, "The concept and implementation of skeletal plans," *Journal of automated reasoning*, vol. 1, pp. 161–208, 1985.
- [101] Y. Shahar, S. Miksch, and P. Johnson, "The asgaard project: a task-specific framework for the application and critiquing of time-oriented clinical guidelines," *Artificial intelligence in Medicine*, vol. 14, no. 1-2, pp. 29–51, 1998.
- [102] O. Young and Y. Shahar, "Applying hybrid-asbru clinical guidelines using the spock system," in *AMIA Annual Symposium Proceedings*, vol. 2005. American Medical Informatics Association, 2005, p. 854.
- [103] M. Balser, C. Duelli, and W. Reif, "Formal semantics of asbru-an overview," in *Proceedings of the International Conference on Integrated Design and Process Technology*, 2002, pp. 1–8.
- [104] O. Young and Y. Shahar, "The spock system: developing a runtime application engine for hybrid-asbru guidelines," in *Conference on Artificial Intelligence in Medicine in Europe*. Springer, 2005, pp. 166–170.
- [105] A. Ten Teije, M. Marcos, M. Balser, J. van Croonenborg, C. Duelli, F. van Harmelen, P. Lucas, S. Miksch, W. Reif, K. Rosenbrand et al., "Improving medical protocols by formal methods," *Artificial Intelligence in Medicine*, vol. 36, no. 3, pp. 193–209, 2006.
- [106] M. Balser, W. Reif, G. Schellhorn, K. Stenzel, and A. Thums, "Formal system development with kiv," in *Fundamental Approaches to Software Engineering: Third International Conference, FASE 2000 Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2000 Berlin, Germany, March 25–April 2, 2000 Proceedings 3*. Springer, 2000, pp. 363–366.
- [107] P. parameter: management of hyperbilirubinemia in the healthy term newborn, "American academy of pediatrics. provisional committee for quality improvement and sub-committee on hyperbilirubinemia," *Pediatrics*, vol. 94, pp. 558–565, 1994.
- [108] G. Rutten, "Nhg-standaard diabetes mellitus type 2 (eerste herziening)," *HUISARTS EN WETENSCHAP-NEDERLANDS-*, vol. 42, pp. 67–74, 1999.
- [109] D. Turi and G. Plotkin, "Towards a mathematical operational semantics," in *Proceedings of Twelfth Annual IEEE Symposium on Logic in Computer Science*. IEEE, 1997, pp. 280–291.

- [110] W. Damm, B. Josko, H. Hungar, and A. Pnueli, “A compositional real-time semantics of state machine designs,” in *Compositionality: The Significant Difference: International Symposium, COMPOS’97 Bad Malente, Germany, September 8–12, 1997 Revised Lectures*. Springer, 1998, pp. 186–238.
- [111] M. Balser, C. Duelli, and W. Reif, “Formal semantics of asbru—an overview,” in *Proceedings of the International Conference on Integrated Design and Process Technology*, 2002, pp. 1–8.
- [112] J. F. Allen and G. Ferguson, “Actions and events in interval temporal logic,” *Journal of logic and computation*, vol. 4, no. 5, pp. 531–579, 1994.
- [113] P. Terenziani, G. Molino, and M. Torchio, “A modular approach for representing and executing clinical guidelines,” *Artificial Intelligence in Medicine*, vol. 23, no. 3, pp. 249–276, 2001.
- [114] V. Brusoni, L. Console, P. Terenziani, and B. Pernid, “Later: Managing temporal information efficiently,” *IEEE Expert*, vol. 12, no. 4, pp. 56–64, 1997.
- [115] C. J. Date, *A Guide to the SQL Standard*. Addison-Wesley Longman Publishing Co., Inc., 1989.
- [116] P. Terenziani, S. Montani, A. Bottrighi, M. Torchio, G. Molino, and G. Correndo, “The glare approach to clinical guidelines: main features,” in *Computer-based Support for Clinical Guidelines and Protocols*. IOS Press, 2004, pp. 162–166.
- [117] F. Chesani, P. De Matteis, P. Mello, M. Montali, and S. Storari, “A framework for defining and verifying clinical guidelines: A case study on cancer screening,” in *Foundations of Intelligent Systems: 16th International Symposium, ISMIS 2006, Bari, Italy, September 27–29, 2006. Proceedings 16*. Springer, 2006, pp. 338–343.
- [118] M. Alberti, F. Chesani, M. Gavanelli, E. Lamma, P. Mello, and P. Torroni, “Verifiable agent interaction in abductive logic programming: the sciff framework,” *ACM Transactions on Computational Logic (TOCL)*, vol. 9, no. 4, pp. 1–43, 2008.
- [119] F. Chesani, E. Lamma, P. Mello, M. Montali, S. Storari, P. Baldazzi, and M. Manfredi, “Compliance checking of cancer-screening careflows: an approach based on computational logic,” in *Computer-based medical guidelines and protocols: A primer and current trends*. IOS Press, 2008, pp. 183–192.
- [120] A. Bottrighi, F. Chesani, P. Mello, M. Montali, S. Montani, S. Storari, and P. Terenziani, “Analysis of the glare and gprobe approaches to clinical guidelines,” in *Knowledge Representation for Health-Care. Data, Processes and Guidelines: AIME 2009 Workshop KR4HC 2009, Verona, Italy, July 19, 2009, Revised Selected and Invited Papers 1*. Springer, 2010, pp. 76–87.

- [121] S. Skonetzki, H.-J. Gausepohl, M. van der Haak, S. Knaebel, O. Linderkamp, and T. Wetter, "HELEN, a modular framework for representing and implementing clinical practice guidelines," *Methods of Information in Medicine*, vol. 43, no. 04, pp. 413–426, 2004.
- [122] J. H. Gennari, M. A. Musen, R. W. Fergerson, W. E. Grosso, M. Crubézy, H. Eriksson, N. F. Noy, and S. W. Tu, "The evolution of protégé: an environment for knowledge-based systems development," *International Journal of Human-computer studies*, vol. 58, no. 1, pp. 89–123, 2003.
- [123] P. C. on Quality Improvement and S. on Hyperbilirubinemia, "Practice parameter: management of hyperbilirubinemia in the healthy term newborn," *Pediatrics*, vol. 94, no. 4, pp. 558–565, 1994.
- [124] J. Bhatia, "Current options in the management of apnea of prematurity," *Clinical Pediatrics*, vol. 39, no. 6, pp. 327–336, 2000.
- [125] P. D. Johnson, S. Tu, N. Booth, B. Sugden, and I. N. Purves, "Using scenarios in chronic disease management guidelines for primary care." in *Proceedings of the AMIA Symposium*. American Medical Informatics Association, 2000, p. 389.
- [126] S. W. Tu, J. R. Campbell, J. Glasgow, M. A. Nyman, R. McClure, J. McClay, C. Parker, K. M. Hrabak, D. Berg, T. Weida et al., "The sage guideline model: achievements and overview," *Journal of the American Medical Informatics Association*, vol. 14, no. 5, pp. 589–598, 2007.
- [127] P. Ram, D. Berg, S. Tu, G. Mansfield, Q. Ye, R. Abarbanel, and N. Beard, "Executing clinical practice guidelines using the sage execution engine," in *MEDINFO 2004*. IOS Press, 2004, pp. 251–255.
- [128] P. L. Miller, "Domain-constrained generation of clinical condition sets to help test computer-based clinical guidelines," *Journal of the American Medical Informatics Association*, vol. 8, no. 2, pp. 131–145, 2001.
- [129] G. Duftschmid and S. Miksch, "Knowledge-based verification of clinical guidelines by detection of anomalies," *Artificial intelligence in Medicine*, vol. 22, no. 1, pp. 23–41, 2001.
- [130] A. D. Preece and R. Shinghal, "Foundation and application of knowledge base verification," *International journal of intelligent Systems*, vol. 9, no. 8, pp. 683–701, 1994.
- [131] B. Pérez and I. Porres, "Authoring and verification of clinical guidelines: A model driven approach," *Journal of Biomedical Informatics*, vol. 43, no. 4, pp. 520–536, 2010.
- [132] Object Management Group, "Omg system modeling language specification," <https://omgsysml.org>, 2006.

- [133] J. Whittle and J. Schumann, “statechart synthesis from scenarios: An air traffic control case study,” in *Proc. of” Scenarios and State-Machines: models, algorithms and tools” workshop at the 24th Int. Conf. on Software Engineering (ICSE 2002)*. ACM New York, 2002.
- [134] S. Efroni, D. Harel, and I. R. Cohen, “Toward rigorous comprehension of biological complexity: modeling, execution, and visualization of thymic t-cell maturation,” *Genome research*, vol. 13, no. 11, pp. 2485–2497, 2003.
- [135] E. Mikk, Y. Lakhnech, M. Siegel, and G. J. Holzmann, “Implementing statecharts in promela/spin,” in *Proceedings. 2nd IEEE Workshop on Industrial Strength Formal Specification Techniques*. IEEE, 1998, pp. 90–101.
- [136] M. B. Dwyer, G. S. Avrunin, and J. C. Corbett, “Property specification patterns for finite-state verification,” in *Proceedings of the second workshop on Formal methods in software practice*, 1998, pp. 7–15.
- [137] M. B. Dwyer, G. S. Avrunin, and J. C. Corbett, “Patterns in property specifications for finite-state verification,” in *Proceedings of the 21st International Conference on Software Engineering*, 1999, pp. 411–420.
- [138] F. Bitsch, “Safety patterns—the key to formal specification of safety requirements,” in *Computer Safety, Reliability and Security: 20th International Conference, SAFECOMP 2001 Budapest, Hungary, September 26–28, 2001 Proceedings 20*. Springer, 2001, pp. 176–189.
- [139] K. Ryndina, “Improving requirements engineering: an enhanced requirements modelling and analysis method,” <http://pubs.cs.uct.ac.za/id/eprint/201>, 2005.
- [140] I. Porres, E. Domínguez, B. Pérez, Á. Rodríguez, and M. A. Zapata, “A model driven approach to automate the implementation of clinical guidelines in decision support systems,” in *15th Annual IEEE International Conference and Workshop on the Engineering of Computer Based Systems (ECBS 2008)*. IEEE, 2008, pp. 210–218.
- [141] X. Chen and G. Rosu, “SETSS’19 lecture notes on K,” in *Engineering Trustworthy Software Systems*, ser. Lecture Notes in Computer Science, J. Bowen and Z. Liu, Eds. Springer, 2019.
- [142] D. Park, A. Stefănescu, and G. Roşu, “Kjs: A complete formal semantics of javascript,” in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI ’15. New York, NY, USA: Association for Computing Machinery, 2015, p. 346–356.
- [143] G. Roşu, “Matching logic,” *Logical Methods in Computer Science*, vol. 13, no. 4, pp. 1–61, 12 2017.
- [144] X. Chen and G. Rosu, “Matching mu-logic,” in *2019 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. Los Alamitos, CA, USA: IEEE Computer Society, 6 2019, pp. 1–13.

- [145] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [146] “Matching logic,” <https://matching-logic.org>.
- [147] X. Chen, D. Lucanu, and G. Roşu, “Initial algebra semantics in matching logic,” <http://hdl.handle.net/2142/107781>, University of Illinois at Urbana-Champaign, Tech. Rep., 2020.
- [148] X. Chen and G. Roşu, “A general approach to define binders using matching logic,” vol. 4, no. ICFP. ACM New York, NY, USA, 2020, pp. 1–32.
- [149] X. Chen and G. Roşu, “Applicative matching logic: Semantics of k,” <http://hdl.handle.net/2142/104616>, University of Illinois at Urbana-Champaign, Tech. Rep., 7 2019.
- [150] X. Chen, M.-T. Trinh, N. Rodrigues, L. Peña, and G. Roşu, “Towards a unified proof framework for automated fixpoint reasoning using matching logic,” in *PACMPL Issue OOPSLA 2020*. ACM/IEEE, 11 2020, pp. 1–29.
- [151] “The GNU Foundation”, “GCC, The GNU Compiler Collection”, <https://gcc.gnu.org/>.
- [152] “Google”, “v8”, <https://v8.dev>.
- [153] “The Mozilla Foundation”, “spidermonkey”, <https://firefox-source-docs.mozilla.org/js/index.html>.
- [154] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” 2014, (Updated for EIP-150 in 2017) <http://yellowpaper.io/>.
- [155] N. Szabo, “Smart contracts,” *Unpublished manuscript*, 1994, http://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart_contracts_idea.html.
- [156] G. W. Peters and E. Panayi, “Understanding modern banking ledgers through blockchain technologies: Future of transaction processing and smart contracts on the internet of money,” in *Banking Beyond Banks and Money*. Springer, 2016, pp. 239–278.
- [157] M. del Castillo, “Ethereum executes blockchain hard fork to return DAO funds,” 2016, <http://goo.gl/MpwrhS>.
- [158] P. Daian, “DAO attack,” <http://hackingdistributed.com/2016/06/18/analysis-of-the-dao-exploit/>, 2016.
- [159] Y. Hirai, “Defining the ethereum virtual machine for interactive theorem provers,” WSTC17, International Conference on Financial Cryptography and Data Security, 2017.

- [160] “cpp-ethereum,” <https://github.com/ethereum/cpp-ethereum>.
- [161] D. P. Mulligan, S. Owens, K. E. Gray, T. Ridge, and P. Sewell, “Lem: reusable engineering of real-world semantics,” in *ACM SIGPLAN Notices*, vol. 49, no. 9. ACM, 2014, pp. 175–188.
- [162] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, “Making smart contracts smarter,” Cryptology ePrint Archive, Report 2016/633, 2016, <http://eprint.iacr.org/2016/633>.
- [163] I. Grishchenko, M. Maffei, and C. Schneidewind, “A semantic framework for the security analysis of ethereum smart contracts. technical report.” 2018.
- [164] “Oyente,” <https://github.com/melonproject/oyente>.
- [165] “Trail of Bits”, “manticore: Symbolic execution for humans”, <https://github.com/trailofbits/manticore>, 2017.
- [166] The Ethereum Foundation, “Remix,” <https://remix.ethereum.org>.
- [167] “hevm,” <https://github.com/dapphub/hevm>.
- [168] “F*,” <https://www.fstar-lang.org/>.
- [169] I. Grishchenko, M. Maffei, and C. Schneidewind, “a semantic framework for the security analysis of ethereum smart contracts”, in *Principles of Security and Trust*, L. Bauer and R. Küsters, Eds. Springer International Publishing, 2018, pp. 243–269.
- [170] “Ethereum jello paper,” <https://yellowpaper.org>.
- [171] Ethereum, “Ethereum application binary support,” <https://docs.soliditylang.org/en/latest/abi-spec.html>, 2017.
- [172] “Solidity language,” <https://solidity-lang.org>.
- [173] F. Vogelsteller and V. Buterin, “Erc-20 token standard,” <https://ethereum.org/en/developers/docs/standards/tokens/erc-20/>, 2015.
- [174] S. Simrock, “Control theory,” <https://cds.cern.ch/record/1100534/files/p73.pdf>, 2008.
- [175] S. Song, M. Saxena, P.-H. Tsai, and L. Sha, “Towards modular and formally-verifiable software architecture for clinical guidance systems,” in *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 2023, pp. 4271–4276.
- [176] F. Tao, B. Xiao, Q. Qi, J. Cheng, and P. Ji, “Digital twin modeling,” *Journal of Manufacturing Systems*, vol. 64, pp. 372–389, 2022.
- [177] Wikipedia contributors, “Acls — Wikipedia, the free encyclopedia,” <https://en.wikipedia.org/w/index.php?title=ACLS&oldid=994958350>, 2020.

- [178] American Heart Association (AHA), “2020 American Heart Association Guidelines for CPR and ECC,” <https://cpr.heart.org/en/resuscitation-science/cpr-and-ecc-guidelines>.
- [179] American Heart Association (AHA), “Algorithms: Adult,” <https://cpr.heart.org/en/resuscitation-science/cpr-and-ecc-guidelines/algorithms>.
- [180] K. Honarmand, C. Mepham, C. Ainsworth, and Z. Khalid, “Adherence to advanced cardiovascular life support (acls) guidelines during in-hospital cardiac arrest is associated with improved outcomes,” *Resuscitation*, vol. 129, pp. 76–81, 2018.
- [181] C. P. Crowley, J. D. Saliccioli, and E. Y. Kim, “The association between ACLS guideline deviations and outcomes from in-hospital cardiac arrest,” *Resuscitation*, vol. 153, pp. 65–70, 2020.
- [182] T. Kidd and S. Kendall, “Review of effective advanced cardiac life support training using experiential learning,” *Journal of Clinical Nursing*, vol. 16, no. 1, pp. 58–66, 2007.
- [183] Wikipedia contributors, “Defibrillation — Wikipedia, the free encyclopedia,” <https://en.wikipedia.org/w/index.php?title=Defibrillation&oldid=1238353052>, 2024.
- [184] MetaOpenSource, “React: The library for web and native user interfaces,” <https://react.dev/>.
- [185] D. Brand and P. Zafiropulo, “On communicating finite-state machines,” *Journal of the ACM (JACM)*, vol. 30, no. 2, pp. 323–342, 1983.
- [186] “K Framework: Networking,” <https://kframework.org/k-distribution/include/kframework/builtin/domains/#i%2Fo-in-k>.
- [187] “HTTP,” <https://developer.mozilla.org/en-US/docs/Web/HTTP>.
- [188] “K Framework: Backends,” https://kframework.org/k-distribution/k-tutorial/1-basic/20_backends/#k-backends.
- [189] “The K semantics of MediK.” [Online]. Available: <https://github.com/fmcad-78/medik-semantics.git>
- [190] The Maude Manual, “External Objects and IO,” <https://maude.lcc.uma.es/maude30-manual-html/maude-manualch9.html>.
- [191] “Ecma-404 the json data interchange standard.” <https://www.json.org/json-en.html>.
- [192] “Sepsis screening and management application.” [Online]. Available: <https://github.com/fmcad-78/psepsis.git>
- [193] D. A. Schmidt, “Trace-based abstract interpretation of operational semantics,” *Lisp and symbolic computation*, vol. 10, pp. 237–271, 1998.

- [194] J. Koppel, J. Kearl, and A. Solar-Lezama, “Automatically deriving control-flow graph generators from operational semantics,” vol. 6, no. ICFP. ACM New York, NY, USA, 2022, pp. 742–771.
- [195] N. D. Jones, C. K. Gomard, and P. Sestoft, *Partial evaluation and automatic program generation*. Peter Sestoft, 1993.